AQEF Conformance Test Specification

Working Draft 9

Tests for AQEF Working Draft 0.41.4

Igor Akymenko

AQEF Conformance Test Specification

Status: Working Draft 9. Tests for AQEF Working Draft 0.41.4.

License: CC BY 4.0, as the specification.

0. Introduction

0.1 Purpose

The AQEF specification states what a conformant implementation MUST do. This document states how to check it. Each test names the requirement it checks, the configuration to start from, what to do, and what an implementation that meets the requirement does in response.

This document contains no requirements of its own. Where a test and the specification disagree, the specification is right and the test is wrong. Its normative words (MUST, SHOULD, MAY) appear only inside quotations from the specification.

0.2 What a test result means

A test that passes shows that an implementation behaved as the specification requires in that one situation. It does not show that the implementation is conformant: no finite set of tests can. A test that fails shows that it did not. An implementation’s Conformance Statement (AQEF §10) says which capabilities it claims; this document says how each claim can be checked, and a test report says how the checks came out.

Working Draft 8 was run against a reference implementation, built from the specification text by a party that had not seen these tests. Working Draft 9 revises what that run showed to be wrong in the tests, rather than in the implementation or the specification. Tests may be revised again as other implementations are run.

0.3 Scope of this draft

This draft covers the non-negotiable core of AQEF Conformance (§10): the five logical layers and the seven Fundamental Principles, and the path every conformant implementation takes from a configuration to a Release Decision. It also covers every scoped capability (§10, Conformance Statement), in Sections 20 to 26, each test tagged with the capability’s identifier; CI/CD tests are in Section 14. Section 90 shows, requirement by requirement, what is covered.

0.4 Versioning

This document is versioned separately from the specification. Each version names the specification version its tests were written against. A test is revised, not reused, when the requirement it checks changes.

0.5 Structure

Section	Content
1	Conventions: test format, outcomes, verification methods, test harness, isolation, test report, review of Inspection and Declaration
2	Fixtures: the base configuration, the scripted subject and Judge, postcondition fixtures, harness self-test
10	Configuration and Definition
11	Execution and Evidence
12	Assessment
13	Decision: aggregation, Quality Gates, Result Resolution
14	Release Decision and Governance
15	API behavior: pagination, deletion
16	Admission of Judges
20	Capability `human_review`
21	Capability `multi_judge`
22	Capability `seeded_controls`
23	Capability `action_boundary`
24	Capabilities `dataset:<source>` (`dataset_mechanism:<type>` has one case, in CT-CFG-025, and CT-CFG-028)
25	Capabilities `plugins:<type>`
26	Capabilities `cli`, `sdk:<language>`, `reference_ui`, `test_type:<name>`
90	Traceability: requirement → test
`harness/`	Harness Contract: a separate document specifying the test infrastructure (1.4)

0.6 Changes

Working Draft 9 follows the first run of these tests against an implementation, and covers what AQEF 0.41.2 to 0.41.4 added.

- **Tests added.**
  - CT-CFG-026: a Gate that no clause can feed is rejected at load.
  - CT-CFG-027: a lower level cannot loosen a higher level's `validator_composition`.
  - CT-CFG-028: a Dataset mechanism the implementation does not claim is rejected.
  - CT-ASM-020: a Custom Validator's `expected` in a form the implementation does not accept is rejected.
  - CT-DEC-024: a Gate with no `actionable` Result fails with `insufficient_basis` and `value: "not_computed"`.
  - CT-REL-038: Audit records actions that change state, and reading the audit log does not change it.

    - CT-REL-039: a Team binding, where an implementation has one, denies access outside the Team.
- **Tests revised where they asked more than the specification does.**
    - A rejection is no longer required to name its cause, since the specification says nothing about what an error contains (1.5). CT-CFG-025, CT-REL-010, and CT-REL-011 change with it.
    - CT-EXE-012 checks the subject's latency, not the request: the specification's "exact request" is a tool call's (Section 23). It no longer assumes that an implementation adds less than a fixed overhead.
    - CT-REL-019 and CT-REL-033 read a Principal's kind where the implementation records it. A Release Decision has no field for it.
    - CT-SCT-006 gives `reference_oracles` as JudgeConfigs (Appendix A §A.1).
    - CT-ASM-016 gives `cts-len`'s `expected` in the form the implementation documents (Appendix A §A.5, 0.41.2).
    - CT-ASM-019 uses a scripted Judge, `J-PASS-90-A3`, that reports how many assertions it assessed, where the implementation's Judge interface carries a count.
    - `CT-CFG-025.24` belongs to `dataset_mechanism:edge_cases` (0.41.4).
- **Conventions.**
    - Every test that expects a rejection first loads its unchanged starting configuration as a setup check (1.6).
    - The Principal that loads fixtures holds the Permissions loading needs (14.1).
    - The fields of a Release Decision that the implementation sets itself need not be sent (14.1).
    - `ADM-PASS` names registered Persons in `labelled_by`.
    - 1.2 adds a third ground for `NOT_APPLICABLE`.

Working Draft 8 covers what AQEF 0.40 to 0.41.1 added: clean controls, and the Admission of Judges.

- **Tests added.** CT-SCT-009 to CT-SCT-015 and CT-MJG-009, for clean controls: their configuration, a passed control, a false alarm and what it invalidates, confirmation of cleanness, requalification after a false alarm, `halt_run`, and the quorum. Section 16, CT-ADM-001 to CT-ADM-007, for Admission: who records it, what a record must hold, `admitted` against the threshold, the most recent record deciding at the Gate and in the Report, a changed Judge, a panel, and the reasons a Gate lists. Clean controls have tests of their own; CT-SCT-001 to CT-SCT-008 and CT-DEC-015 to CT-DEC-018 still test defect controls, unchanged.
- **Tests revised.** CT-REL-035 checks the sixth signal, `unadmitted_judges`. CT-SCT-008 quotes the revised Requalification rule. CT-SCT-004 belongs to `seeded_controls` alone: confirming a control needs a Person holding `review`, not the capability `human_review`, which is about Human Reviewers as Oracles.
- **Fixture corrected.** F-SEED-V confirmed its planted farewell with a `forbidden-content` Validator, which fails when the farewell is present, while Appendix A §A.4 makes a confirmation pass when the Evidence is as the control declares. It now uses a `schema` pattern that passes when the farewell is present, so CT-SCT-003 and CT-SCT-004 expect what the specification says.
- **Inventory.** `_inventory/must-inventory-v041-additions.md` adds R-0432 to R-0441; eight revised rows keep their IDs (Section 90).

Working Draft 7 answers a feasibility review of Working Draft 6: whether an independent developer can carry out each test without asking the author.

- **Review.** Inspection and Declaration tests are decided by a checklist with recorded evidence, and a disputed outcome goes to a second reviewer (1.8).
- **Waiting.** A test that waits for a timeout has a stated budget and watches for an event instead of sleeping (1.6); CT-HRV-003, CT-HRV-005, CT-REL-025, and CT-EXE-012 are bounded accordingly.

- **Tests revised.** CT-ASM-014 may use a documented fault hook; CT-DSR-003 names how regeneration is requested and is `NOT_APPLICABLE` where it is not offered; CT-PLG-005 calls its repeated runs a sampled check, not a proof.
- **Harness Contract.** The test infrastructure is specified in a separate document, `harness/` (1.4). Requirement states in a test report follow a stated rule: verified only when every applicable test or case traced to the requirement passed (1.7).

Working Draft 6 answers an external review of Working Draft 5 (20 findings). No requirement of the specification changed; the tests did.

- **Outcomes. Fails if** is no longer a closed list: any departure from **Expected** fails a test (1.1, 1.2). A test that needs an operation Appendix C specifies and the implementation lacks now fails instead of passing (13.1). A test of an optional feature the implementation does not offer is `NOT_APPLICABLE`, not `PASS`.
- **Isolation.** Every test runs in a Project and a harness session of its own (1.6). States that tests used to take over from other tests are built as named postcondition fixtures (2.7), and the harness is checked before a run (2.8).
- **Fixtures.** `J-CATCH` fails both `Goodbye.` and `Bye now.`, as CT-SCT-008 requires. CT-CFG-016 uses `cts-len` instead of an undefined Validator.
- **Tests revised.** CT-ASM-009, CT-ASM-014, CT-ASM-016, CT-ASM-018, CT-ASM-019, CT-DEC-012, CT-DEC-013, CT-DEC-014, CT-DEC-018, CT-REL-018, CT-REL-032, CT-REL-036, CT-API-001, CT-API-002, CT-HRV-002, CT-MJG-006, CT-PLG-003, CT-PLG-005, CT-SDK-001, CT-SDK-002; and the fixtures of the tests that took over another test's state.
- **Tests added.** CT-CFG-025, one case per required configuration field of Appendix A, so that no row of Section 90 is left as Field. Each case has an identifier and an outcome of its own, and Section 90 traces each row to its case (1.2, Cases).
- **Test reports** list every requirement whose tests did not run (1.7).

1. Conventions

1.1 Test format

Every test has the same fields:

Field	Meaning
ID	`CT-<area>-<nnn>`. Areas: `CFG` configuration, `EXE` execution, `ASM` assessment, `DEC` decision, `REL` release and governance, `ADM` Admission of Judges, `API` API behavior not tied to one layer; for scoped capabilities, `HRV` Human Review, `MJG` Multi Judge, `SCT` Seeded Controls, `ABD` Action Boundary, `DSR` Dataset sources, `PLG` Plugins, `SDK` SDKs and CLI, `RUI` Reference UI, `TTP` Test Types. IDs are never reused.
Capability	The Conformance Statement identifier the test belongs to (§10). `core` for the non-negotiable core.
Requirement	Where the requirement is stated, by Volume or Appendix and section, with the operative words quoted.
Method	How the outcome is observed (1.3).
Fixture	The configuration to start from: a fixture from section 2, and the changes to it.
Action	What the tester does.
Expected	What an implementation that meets the requirement does.
Fails if	The departures from **Expected** most likely to occur, named to make them easy to check. The list is not exhaustive: any observation that contradicts **Expected** fails the test (1.2).

1.2 Outcomes

Outcome	Meaning
`PASS`	The implementation behaved as **Expected**. Where **Expected** allows alternatives, any one of them is a `PASS`.
`FAIL`	The implementation did not behave as **Expected**: one of the **Fails if** observations was made, or any other observation that contradicts **Expected**.
`NOT_APPLICABLE`	The test's capability is not claimed in the implementation's Conformance Statement; or the test checks a feature the specification leaves optional (MAY), the implementation does not offer it, and the test says that this outcome applies; or the test checks what an implementation does when it does not claim a capability, and the capability is claimed.
`NOT_RUN`	The test could not be carried out, with the reason stated: for example, its setup could not be built (1.6), a fixture the tester supplies could not be provided or registered, or the harness failed its self-test (2.8). A `NOT_RUN` is not a `PASS`.

Cases. A test whose **Action** lists numbered cases gives each case an identifier of its own, the test's ID followed by the case number (`CT-CFG-025.07`), and each case has its own outcome, decided by the table above as if it were a test. The test report gives every case's outcome, and Section 90 traces requirements to cases, not to the test as a whole. The test's own outcome only summarizes its cases: `FAIL` if any case is `FAIL`; otherwise `NOT_RUN` if any case

is `NOT_RUN`; otherwise `PASS` if any case is `PASS`; otherwise `NOT_APPLICABLE`. A summary `PASS` therefore never stands for a case that did not run, and a test with any case `NOT_RUN` is not reported as passed.

1.3 Verification methods

Method	Meaning
Automated	The outcome is read from the implementation's own records or responses: the Results, Gate Results, Release Decisions, Execution records, and errors it produces.
Harness	The outcome is read from the test harness (1.4): which calls the subject and the Judge actually received, in what order, how many times.
Inspection	A reviewer, following 1.8, examines the implementation's documentation, configuration, or records and states what was found. Used where a requirement cannot be observed from outside, such as how scheduling decisions are recorded.
Declaration	The implementation states how it meets the requirement, and the reviewer records the statement and whether it is plausible, following 1.8. Used only where neither of the above can reach, such as whether a Validator is pure in the sense of Volume V. A Declaration is evidence of intent, not of behavior, and a report says so.

Tests are written against observable behavior. Where an implementation exposes it through the REST API (Appendix C), the API is the natural place to observe it; where it uses another surface, the test report names that surface.

1.4 Test harness

A test checks the engine, not a model. Every fixture therefore points the engine at two stand-ins the tester controls:

- a **scripted subject**: an endpoint the Environment's `model_version` resolves to, which returns a fixed Response chosen per test (section 2.2), or fails in a stated way;
- a **scripted Judge**: an endpoint a JudgeConfig's `model` resolves to, which returns a fixed verdict and Confidence chosen per test (section 2.3), or fails in a stated way.

Both record every call they receive. How an implementation maps a model identifier to an endpoint is its own concern; the tester configures it so that these identifiers reach the harness. Neither stand-in is a probabilistic subject or a real Judge, and the specification does not require one to be: the subject is whatever its Environment points to (Volume II), and a scripted Judge still satisfies Independent Oracles, since its identifier differs from the subject's.

The interfaces of the harness, the adapter between tester and implementation, the records a run keeps, and the format of the test report are specified in the **Harness Contract** (`harness/`), a separate, non-normative document versioned on its own. It binds a harness, never an implementation, and adds no test: where it and this document disagree, this document is right.

1.5 Notation

- Fixture changes are written as YAML fragments with a path, e.g. *in* `suites[0].scenarios[0]`, *remove* `contract`.

- `{{name}}` in a prompt template is illustrative. The specification leaves placeholder syntax to the implementation (Appendix A §A.4, PromptDef); the tester substitutes the implementation's own.
- "Loaded" means accepted at configuration load time, before any Execution. "Rejected" means refused at that point. The specification does not say what an error must contain, so no test requires an error to name its cause. A refusal for a reason that has nothing to do with the configuration is not a rejection of it: missing or failed authentication, a missing Permission, or an operation the implementation does not offer. Where an error does name a cause, the test report records it.

1.6 Isolation

Every test can be run on its own, in any order, and gives the same outcome whichever tests ran before it.

- **A Project of its own.** Each test runs in a Project no other test uses. The text calls it `cts-base`; the tester may give it a unique name per test, such as `cts-base-CT-DEC-012`. The Principals of Section 14.1 are set up for it. Where a **Fixture** gives variants, (a), (b), …, each variant runs in a Project of its own; the steps of one **Action** share one Project.
- **A harness session of its own.** Before the test starts, the scripted subject and Judge are reset: their call logs are emptied, their scripted behaviors set to the defaults of Sections 2.2 and 2.3 or to what the test names, and every counter cleared, so that a "first call" is the first call within this test. Each variant gets its own session. (Harness Contract §8.2 and §8.3.) Calls are logged against the test and variant they belong to, and a harness observation counts only those. A test that switches a behavior during its **Action** changes it from the next call on, without clearing the counters.
- **Setup.** A test whose **Fixture** names a postcondition fixture (2.7) builds it in its own Project as setup, then checks the fixture's setup checks. Calls made during setup are not counted in any harness observation of the test. If the setup cannot be built or a check fails, the test is `NOT_RUN`, with the step that failed as its reason: the behavior that failed belongs to the test that checks it, which reports it as a `FAIL` there.
- **Rejection tests.** A test whose **Expected** is that a configuration is rejected first loads the configuration it starts from, unchanged, in a Project of its own, as a setup check. If that is rejected, the rejection of the changed one shows nothing, and the test is `NOT_RUN`, with that reason. A test with variants or cases does this once for each starting configuration it uses.
- **Reusing a configuration.** "As CT-xxx" in a **Fixture**, without "after", reuses that test's configuration only. No test relies on another test having run.
- **Waiting.** Where a test waits for something to happen (a review timeout, a pipeline timeout), the tester states a budget before the run: the configured duration plus a grace period. The tester watches for the event the test names, such as a record or a change of state, rather than sleeping for a fixed time, and stops at the end of the budget. If the shortest duration the implementation accepts is longer than the run can wait, the test is `NOT_RUN`, with that reason. Durations are measured by the harness on a monotonic clock; the harness's time is never taken as the implementation's.

1.7 Test report

A test report gives, for each test and each case (1.2), its outcome and, for `NOT_RUN` and `NOT_APPLICABLE`, the reason. It also gives every requirement of Section 90 a state, from the tests and cases Section 90 traces it to, leaving out those `NOT_APPLICABLE`:

Tests and cases traced	State
Any `FAIL`	violated
Otherwise any `NOT_RUN`, or any the run left out	unverified
Otherwise at least one `PASS`	verified
Otherwise	not applicable

A requirement is therefore verified only when every applicable test or case traced to it passed: a `PASS` in one test never covers a `NOT_RUN` in another, and a requirement traced to a case is decided by that case, never by the outcome of the test it belongs to. A row of status Restates takes the state of the row it restates. A summary of the run lists every unverified requirement rather than counting it as covered. The report's format is the Harness Contract's (§14, §15).

1.8 Review of Inspection and Declaration tests

A test whose method is Inspection or Declaration is decided by a named reviewer, in a way a second reviewer can repeat:

- The reviewer turns **Expected** into a checklist, one item per statement, and records for each item whether it is met and the evidence used: an identifier of the record, document, response, or screen, kept with the test report.
- The test is `PASS` only when every item is met and has its evidence recorded. An item without evidence, or one the reviewer cannot decide, makes the test `NOT_RUN`, never `PASS`. An item shown not to be met makes it `FAIL`.
- Where the outcome is disputed, a second reviewer decides from the same evidence without seeing the first decision. Until both agree, the test is `NOT_RUN`, and the report keeps both records.

The record of a review is specified in the Harness Contract (§13).

2. Fixtures

2.1 F-BASE — the base configuration

Every test starts from F-BASE unless it says otherwise. F-BASE is complete and valid under Appendix A: one Environment, one Suite, one Scenario with one Constraint and one Expectation, and one blocking Quality Gate.

```
project:
  name: cts-base
  environments:
    - name: cts-env
      model_version: cts-scripted-subject@1
      prompt_version: cts-prompt@1
      tool_configuration: null
      infra_target: cts-harness
  quality_gates:
    - name: functional-gate
      aggregate: functional
      threshold: 1.0
      on_failure: block_release
  suites:
    - name: cts-suite
      scenarios:
        - name: s-greeting
          variables:
            query: "Hello"
          prompt:
            template: "{{query}}"
          contract:
            constraints:
              - validator: schema
                target: response
                expected: { type: string, minLength: 1 }
            expectations:
              - criteria: "The response greets the user."
                judge:
                  model: cts-scripted-judge@1
                  confidence_source: self_rating
                confidence_threshold: 0.8
```

F-BASE's clauses declare no `id` and no `dimension`, so the defaults of Appendix A §A.5 apply: the schema Constraint counts toward Functional, the Expectation toward Semantic.

2.2 Scripted subject responses

Code	Behavior
`SR-OK`	Returns the Response `"Hello! How can I help?"`.
`SR-EMPTY`	Returns the Response `""`, which fails the F-BASE schema Constraint (`minLength: 1`).
`SR-503-THEN-OK`	First call: HTTP 503 before any Response. Second call: as `SR-OK`.
`SR-ECHO`	Returns the Prompt it received, verbatim.
`SR-SLOW-200`	As `SR-OK`, after waiting 200 ms.

Unless a test says otherwise, the scripted subject behaves as `SR-OK`.

2.3 Scripted Judge responses

Code	Behavior
`J-PASS-90`	Verdict `pass`, Confidence `0.90`.
`J-PASS-60`	Verdict `pass`, Confidence `0.60` (below F-BASE's 0.8 threshold).
`J-FAIL-90`	Verdict `fail`, Confidence `0.90`.
`J-DOWN`	Refuses the connection on every call; no verdict.
`J-DOWN-THEN-PASS-90`	First call as `J-DOWN`; every later call as `J-PASS-90`.
`J-FAIL-THEN-PASS-90`	First call as `J-FAIL-90`; every later call as `J-PASS-90`.
`J-PASS-90-A3`	As `J-PASS-90`, and reports that it assessed three independent assertions, where the implementation's Judge interface carries such a count (CT-ASM-019).

Unless a test says otherwise, the scripted Judge behaves as `J-PASS-90`.

2.4 F-MULTI — several Scenarios

Used where a test needs counts. As F-BASE, with `cts-suite` holding three Scenarios, `s-1`, `s-2`, `s-3`, each identical to `s-greeting` except for its name and `variables.query` (`"One"`, `"Two"`, `"Three"`).

2.5 Custom Validators

Some tests need a Custom Validator (Volume V). The tester registers these through whatever means the implementation provides. Where it offers none, those tests are `NOT_RUN`, with that reason.

Name	Behavior
`cts-len`	`pass` if the Response is at least one character long, otherwise `fail`. Reads only the Response.
`cts-flaky`	Returns `pass` the first time it is called and `fail` every later time, whatever the input. Deliberately impure.
`cts-crash`	Raises an error on every call and returns no Result (CT-ASM-014).

A test that uses one registers it as setup, unless its **Action** is the registration itself, and the registration is a setup check (1.6).

2.6 Fixtures defined in later sections

Fixture or behavior	Defined in
`semantic-gate`, `J-MAP`, `J-CATCH`, F-SEED	13.1
Principals (`P-RELEASE`, `P-APPROVER`, `P-NOPERM`, `P-EDITOR`, `P-REVIEWER`, `P-ADMIN`, `SA-CI`), `RP-1`	14.1
F-HR, `P-REVIEWER-2`	20
F-PANEL	21
`ORACLE-1`, `ADM-PASS`, `ADM-FAIL`, `SA-REVIEW`, `gate-admitted`	16
`SR-BYE`, F-SEED-V, `SR-HI`, `J-ALARM`, F-CLEAN, F-CLEAN-V	22
F-ACT, `SR-CALL(url)`	23
`cts-probe`	25

2.7 Postcondition fixtures

Some tests start from the state another sequence of steps leaves behind: a reviewed Result, a missed Seeded Control, an override. Each such state is named here, with the steps that build it and the checks that confirm it was built. A test builds it in its own Project as setup (1.6); it never relies on another test having run. Words such as “evaluate the Gates”, “override”, and “a governed change” have the meanings of Sections 13.1 and 14.1. A machine form of each, for a harness, is in `harness/fixtures/`; where it differs from this table, this table is right.

Fixture	Built by	Setup checks
F-REVIEWED	F-BASE with `semantic-gate`; scripted Judge `J-PASS-60`. Execute `cts-suite`; evaluate the Gates. As `P-REVIEWER`, record a Human Reviewer Result with verdict `pass` for the Expectation, resolving the Judge's Result.	The Human Reviewer Result is recorded, with `resolves` naming the Judge's Result.
F-SEED-MISSED	F-SEED with `semantic-gate`; scripted Judge `J-PASS-90`. Execute `cts-suite`; evaluate the Gates.	`s-control`'s entry in `seeded_control_results` has `outcome: missed`.
F-OVERRIDDEN	F-BASE with `semantic-gate`; scripted Judge `J-FAIL-90`. Execute `cts-suite`; evaluate the Gates. Override `semantic-gate`. As `P-RELEASE`, record a Release Decision with `decision: overridden`.	The `semantic-gate` GateResult carries `override`; the Release Decision is accepted.
F-GATE-CHANGED	F-BASE with `semantic-gate`; scripted Judge `J-PASS-90`. Execute `cts-suite`; evaluate the Gates (evaluation 1). As a governed change, set `semantic-gate`'s `threshold` to `0.9`. Evaluate the Gates again for the same batch (evaluation 2).	Evaluation 2's `semantic-gate` GateResult has `threshold: 0.9`.
F-CONTRACT-CHANGED	F-BASE with `semantic-gate`; scripted Judge `J-PASS-90`. Execute `cts-suite`; evaluate the Gates. As a governed change, set the Expectation's `confidence_threshold` in `s-greeting`'s inline Contract to `0.85`.	`s-greeting`, retrieved, has `confidence_threshold: 0.85`.
F-SA-DECISIONS	F-BASE with `semantic-gate`; `RP-1` in place. Batch 1, with `J-PASS-90`: execute, evaluate the Gates, and as `SA-CI` record decision (b): `decision: approved`, `release_policy: RP-1`. Batch 2, with `J-FAIL-90`: execute, evaluate the Gates, and as `SA-CI` record decision (d): `decision: rejected`, `release_policy: RP-1`. Then, as `P-RELEASE`, revoke `RP-1`.	Decisions (b) and (d) and the revocation are accepted.
F-BASELINED	F-BASE; scripted Judge `J-PASS-90`. Execute `cts-suite` twice. As `P-APPROVER`, designate the first Execution of `s-greeting` as a Baseline. Evaluate the Gates for the first batch, so that a Snapshot references `cts-env`.	The Baseline is recorded; the evaluation returned a `snapshot_id`.
F-SIGNALS	CT-REL-035's configuration. Execute `cts-suite`; evaluate the Gates.	A Report for the batch can be retrieved.

2.8 Harness self-test

Before a test run, and after any change to the harness, the tester calls each scripted behavior directly, without the implementation (Harness Contract §8.6), and checks that it does what Sections 2.2, 2.3, 13.1, 22, and 23 say: each subject code returns its Response or fails as stated; each Judge code returns its verdict and Confidence, and `J-PASS-90-A3` its count; `J-MAP` follows its mapping; `J-CATCH` returns `fail` with Confidence `0.90` for `Goodbye.` and for `Bye now.`, and `pass` with `0.90` for `Hello! How can I help?`; `J-ALARM` returns `fail` with `0.90` for `Hi there! What do you need?` and `pass` with `0.90` for `Hello! How can I help?`; and every first-call behavior changes after its first call and starts afresh after a reset (1.6). The Custom Validators of 2.5 are checked the same way: `cts-len`

passes `"a"` and fails `""`; `cts-flaky` passes, then fails; `cts-crash` raises an error. Every test that uses a behavior that failed its self-test is `NOT_RUN`, with that reason.

10. Configuration and Definition

Tests in this section load a configuration and observe whether it is accepted. None of them needs an Execution.

CT-CFG-001 — A complete, valid configuration is accepted

- **Capability:** core
- **Requirement:** Appendix A, introduction: “A conformant implementation MUST accept configurations expressible within this specification.”
- **Method:** Automated
- **Fixture:** F-BASE, unchanged.
- **Action:** Load the configuration.
- **Expected:** Loaded without error.
- **Fails if:** Rejected, or loaded with any field of F-BASE ignored or altered.

CT-CFG-002 — A Scenario with no effective Contract is rejected

- **Capability:** core
- **Requirement:** Appendix B §B.3, Effective Contract: the composition of Project, Suite, and Scenario Contracts “MUST contain at least one clause”; Appendix A §A.4, `contract`.
- **Method:** Automated
- **Fixture:** F-BASE; in `suites[0].scenarios[0]`, remove `contract`. No `contracts` at Project or Suite level.
- **Action:** Load the configuration.
- **Expected:** Rejected; the error identifies `s-greeting`.
- **Fails if:** Loaded, or `s-greeting` can be executed.

CT-CFG-003 — A Contract with no clauses is rejected

- **Capability:** core
- **Requirement:** Appendix B §B.2, Contract: “at least one of `constraints` or `expectations` MUST be non-empty”; Appendix A §A.5.
- **Method:** Automated
- **Fixture:** F-BASE; in the Scenario’s `contract`, set `constraints: []` and `expectations: []`.
- **Action:** Load the configuration.
- **Expected:** Rejected.
- **Fails if:** Loaded.

CT-CFG-004 — An Expectation must bind exactly one Oracle

- **Capability:** core
- **Requirement:** Appendix B §B.2, Expectation: “Exactly one of `judge`, `multi_judge`, or `human_reviewer` MUST be present”; Volume VII, Contract Language.
- **Method:** Automated

- **Fixture:** Two variants of F-BASE. (a) In the Expectation, remove `judge`. (b) In the Expectation, add `human_reviewer: { role: reviewer }` alongside `judge`.
- **Action:** Load each variant.
- **Expected:** Both rejected.
- **Fails if:** Either is loaded.

CT-CFG-005 — A Judge-bound Expectation needs a Confidence threshold

- **Capability:** core
- **Requirement:** Appendix B §B.3, Expectation Oracle exclusivity: "`confidence_threshold` is REQUIRED" for `judge` and `multi_judge`.
- **Method:** Automated
- **Fixture:** F-BASE; in the Expectation, remove `confidence_threshold`.
- **Action:** Load the configuration.
- **Expected:** Rejected.
- **Fails if:** Loaded, or loaded with a default threshold supplied silently.

CT-CFG-006 — A Judge identical to the subject is rejected

- **Capability:** core
- **Requirement:** Volume I, Chapter 3, Independent Oracles: "An Oracle MUST NOT be the same system, weights, or decision process as the AI system it evaluates"; Appendix B §B.3, Independent Oracle — identity.
- **Method:** Automated
- **Fixture:** F-BASE; set the Judge's `model` to `cts-scripted-subject@1`.
- **Action:** Load the configuration.
- **Expected:** Rejected; the error names the identity of Judge and subject.
- **Fails if:** Loaded.

CT-CFG-007 — A composition conflict is surfaced, not resolved silently

- **Capability:** core
- **Requirement:** Volume VII, Contract Composition: "Composition MUST surface the conflict explicitly rather than silently resolve it"; Appendix B §B.3, Effective Contract: "A lower level MUST NOT remove or weaken a clause contributed by a higher one. Two clauses are the same clause only when they declare the same `id`; the same `id` with differing content is a composition conflict."
- **Method:** Automated
- **Fixture:** F-BASE, with `id: greets` added to the Scenario's Expectation, plus a Project-level Contract carrying an Expectation with the same `id` and `confidence_threshold: 0.9`:

```
project:
  contracts:
    - expectations:
        - id: greets
          criteria: "The response greets the user."
          judge:
            model: cts-scripted-judge@1
            confidence_source: self_rating
          confidence_threshold: 0.9
```

The Scenario keeps `confidence_threshold: 0.8`.

- **Action:** Load the configuration; if it loads, attempt to execute `cts-suite`.
- **Expected:** The conflict between 0.9 and 0.8 is reported, naming the clause and both values. Either the configuration is rejected, or `s-greeting` is not assessed under that clause until the conflict is resolved.
- **Fails if:** `s-greeting` is assessed with either threshold without the conflict having been reported.

CT-CFG-008 — A Prompt that references an unsupplied Variable is rejected

- **Capability:** core
- **Requirement:** Appendix B §B.3, Prompt presence: “Every Variable a `prompt.template` references MUST be supplied”; Appendix A §A.4, PromptDef.
- **Method:** Automated
- **Fixture:** F-BASE; set the template to `"{{query}} {{name}}"`. `name` is not in `variables`.
- **Action:** Load the configuration, or, if the implementation checks templates only when an Execution is requested, request one.
- **Expected:** Rejected before any call reaches the scripted subject.
- **Fails if:** The subject receives a call, or receives a Prompt with the placeholder unfilled or filled with an empty value.

CT-CFG-009 — Quality Gate names are unique across a Project and its Suites

- **Capability:** core
- **Requirement:** Appendix B §B.3, Gate name uniqueness; Appendix A §A.3, `quality_gates`.
- **Method:** Automated
- **Fixture:** F-BASE; add to `suites[0]` a `quality_gates` list with a Gate named `functional-gate` (same name as the Project’s) and `threshold: 0.5`.
- **Action:** Load the configuration.
- **Expected:** Rejected.
- **Fails if:** Loaded, whichever of the two Gates the implementation would use.

CT-CFG-010 — Retry may be configured only for infrastructure failure

- **Capability:** core
- **Requirement:** Appendix B §B.2, RetryConfig: `on` is `infrastructure_failure`; “A schema MUST reject any other value”; Appendix B §B.3, Retry scope.
- **Method:** Automated

- **Fixture:** F-BASE; add `execution: { retry: { max_attempts: 2, on: result_failure } }` to the Project.
- **Action:** Load the configuration.
- **Expected:** Rejected.
- **Fails if:** Loaded.

CT-CFG-011 — A Scenario cannot take both a Dataset and Variables

- **Capability:** core
- **Requirement:** Appendix A §A.4: a Scenario "MUST NOT specify both" `dataset` and `variables`.
- **Method:** Automated
- **Fixture:** F-BASE, plus a Static Dataset in the Project's `datasets`:

  ```
  project:
    datasets:
      - name: ds-1
        version: "1.0.0"
        source: static
        rows:
          - id: r1
            variables:
              query: "Hi"
  ```

  In the Scenario, add `dataset: ds-1@1.0.0` and keep `variables`.
- **Action:** Load the configuration.
- **Expected:** Rejected.
- **Fails if:** Loaded, whichever source the implementation would use.

CT-CFG-012 — The same clause contributed twice, identically, is one clause

- **Capability:** core
- **Requirement:** Appendix A §A.5, `id`: "Two contributed clauses are the same clause only when they declare the same `id`: identical content is one clause"; Volume VII, Contract Composition.
- **Method:** Automated, Harness
- **Fixture:** As CT-CFG-007, with the Project-level Expectation's `confidence_threshold` set to `0.8`, so that both are identical.
- **Action:** Load the configuration; execute `cts-suite`.
- **Expected:** Loaded with no conflict reported. `s-greeting` has exactly one Expectation Result, and the scripted Judge receives one call.
- **Fails if:** A conflict is reported, or two Expectation Results are produced.

CT-CFG-013 — Clauses without an `id` are never the same clause

- **Capability:** core
- **Requirement:** Appendix A §A.5, `id`: "A clause without `id` is never the same clause as another, and simply applies"; Volume VII, Contract Composition.
- **Method:** Automated, Harness

- **Fixture:** As CT-CFG-007, with `id` removed from both Expectations (thresholds stay `0.9` and `0.8`).
- **Action:** Load the configuration; execute `cts-suite` with scripted Judge `J-PASS-90`.
- **Expected:** Loaded with no conflict reported. `s-greeting` has two Expectation Results, both `actionable` (0.9 meets both thresholds), and the scripted Judge receives two calls.
- **Fails if:** A conflict is reported, or the two clauses are merged into one.

CT-CFG-014 — A Dataset reference without a version is rejected

- **Capability:** core
- **Requirement:** Appendix A §A.4, `dataset`: "The version MUST be given"; Appendix B §B.3, Dataset reference.
- **Method:** Automated
- **Fixture:** As CT-CFG-011, with `variables` removed from the Scenario and `dataset: ds-1` (no version).
- **Action:** Load the configuration.
- **Expected:** Rejected. The same configuration with `dataset: ds-1@1.0.0` is loaded.
- **Fails if:** The configuration without a version is loaded, or the one with it is rejected.

CT-CFG-015 — A clause `id` is unique within its Contract

- **Capability:** core
- **Requirement:** Appendix B §B.3, Clause identity: "A clause `id` MUST be unique within the Contract that declares it."
- **Method:** Automated
- **Fixture:** F-BASE; give the schema Constraint `id: c1`, and add a second Constraint to the same Contract with the same `id`:

```
- id: c1
  validator: forbidden-content
  target: response
  expected:
    patterns:
      - regex: "error"
```

- **Action:** Load the configuration.
- **Expected:** Rejected; the error names `c1`.
- **Fails if:** Loaded.

CT-CFG-016 — A Custom Validator's clause must declare its dimension

- **Capability:** core
- **Requirement:** Appendix B §B.3, Clause dimension: "A Constraint bound to a Custom Validator MUST declare `dimension`"; Appendix A §A.5.
- **Method:** Automated
- **Fixture:** F-BASE; add a Constraint `validator: cts-len` (Section 2.5), `target: response`, `expected: {}`, with no `dimension`. Setup check: `cts-len` is registered (2.5). If the implementation offers no Custom Validators, the outcome is `NOT_RUN`, with that reason.
- **Action:** Load the configuration. Then add `dimension: functional` to that Constraint and load it again.

- **Expected:** The first load is rejected; the second is accepted.
- **Fails if:** The first is loaded, or the second is rejected.

CT-CFG-017 — A Dataset without a version is rejected

- **Capability:** core
- **Requirement:** Volume IV, Versioning: “Every Dataset, regardless of source, MUST be versioned”; Appendix A §A.8, `version`; Appendix C §C.3: “Create a Dataset (version MUST be specified).”
- **Method:** Automated
- **Fixture:** As CT-CFG-011, with `variables` removed from the Scenario, `dataset: ds-1@1.0.0` kept, and `version` removed from `ds-1`.
- **Action:** (a) Load the configuration. (b) Separately, `POST /projects/cts-base/datasets` with the same Dataset and no `version`.
- **Expected:** Both rejected.
- **Fails if:** Either is accepted.

CT-CFG-018 — One name and version always means the same rows

- **Capability:** core
- **Requirement:** Appendix B §B.3, Dataset version immutability: “Two configurations referencing the same Dataset name + version MUST resolve to identical rows.”
- **Method:** Automated
- **Fixture:** CT-CFG-011’s `ds-1` at version `1.0.0`, registered with the implementation.
- **Action:** Register, or load in a second configuration, a Dataset `ds-1` at version `1.0.0` whose only row has `query: "Bye"`. Then retrieve the rows of `ds-1@1.0.0`.
- **Expected:** The second definition is rejected. The rows of `ds-1@1.0.0` are unchanged: one row, `query: "Hi"`.
- **Fails if:** The second definition is accepted, or the rows changed.

CT-CFG-019 — A Constraint names a Validator the implementation has

- **Capability:** core
- **Requirement:** Volume VII, Constraints: “A Constraint MUST name the Validator, built-in or custom (Volume V), it binds to”; Appendix A §A.5, `validator`.
- **Method:** Automated
- **Fixture:** Two variants of F-BASE. (a) Remove `validator` from the schema Constraint. (b) Set it to `cts-no-such-validator`, which is not registered.
- **Action:** Load each variant.
- **Expected:** Both rejected.
- **Fails if:** Either is loaded.

CT-CFG-020 — A Project needs at least one Environment

- **Capability:** core
- **Requirement:** Appendix A §A.1, `environments`: “At least one Environment MUST be defined.”
- **Method:** Automated

- **Fixture:** F-BASE with `environments: []`.
- **Action:** Load the configuration.
- **Expected:** Rejected.
- **Fails if:** Loaded.

CT-CFG-021 — A Suite needs at least one Scenario

- **Capability:** core
- **Requirement:** Appendix A §A.3, `scenarios`: "At least one Scenario MUST be present."
- **Method:** Automated
- **Fixture:** F-BASE with `suites[0].scenarios: []`.
- **Action:** Load the configuration.
- **Expected:** Rejected.
- **Fails if:** Loaded.

CT-CFG-022 — A Scenario needs a Prompt with a template

- **Capability:** core
- **Requirement:** Appendix B §B.3, Prompt presence: "A Scenario MUST carry `prompt` unless its `dataset` is a `production_replay` Dataset or it carries `evidence`"; Appendix A §A.4, PromptDef `template`.
- **Method:** Automated
- **Fixture:** Two variants of F-BASE. (a) Remove `prompt` from `s-greeting`. (b) Keep `prompt` but remove its `template`.
- **Action:** Load each variant.
- **Expected:** Both rejected.
- **Fails if:** Either is loaded.

CT-CFG-023 — A Reusable Contract needs a name

- **Capability:** core
- **Requirement:** Appendix A §A.5, `name`: "MUST (if reusable)."
- **Method:** Automated
- **Fixture:** None beyond F-BASE.
- **Action:** `POST /projects/cts-base/contracts` as `P-APPROVER` (Section 14.1) with F-BASE's Contract and no `name`.
- **Expected:** Rejected.
- **Fails if:** Accepted.

CT-CFG-024 — A minimum basis must state what it counts

- **Capability:** core
- **Requirement:** Appendix A §A.10, MinBasis: "At least one of the two MUST be present"; Appendix B §B.2, MinBasis.
- **Method:** Automated
- **Fixture:** F-BASE; give `functional-gate` `min_basis: {}`.
- **Action:** Load the configuration.

- **Expected:** Rejected.
- **Fails if:** Loaded, or loaded with a default minimum supplied silently.

CT-CFG-025 — A configuration missing a required field is rejected

- **Capability:** core; case `CT-CFG-025.24`: `dataset_mechanism:edge_cases`; case `CT-CFG-025.25`: `seeded_controls`
- **Requirement:** Appendix A, every field marked MUST in the tables of §A.1 to §A.10 that the cases below name; Appendix A, introduction: “A conformant implementation MUST accept configurations expressible within this specification”, which a configuration lacking a required field is not.
- **Method:** Automated
- **Fixture:** Four starting configurations, each a setup check: it must load (for F-BASE, CT-CFG-001). If a starting configuration is rejected, the cases that use it are `NOT_RUN`, with that reason; the other cases still run.
  - F-BASE.
  - F-RETRY: F-BASE with `execution: { retry: { max_attempts: 2, on: infrastructure_failure } }` on the Project.
  - F-DS: CT-CFG-011’s configuration with `variables` removed from the Scenario, so that `s-greeting` draws on `ds-1@1.0.0`; for case `CT-CFG-025.24`, `ds-1` also given `mechanisms: [ { type: edge_cases } ]`.
  - F-SEED (Section 13.1).
- **Action:** For each case, remove the one field named from its starting configuration, change nothing else, and load the result. Each case runs in a Project of its own and has its own outcome (1.2, Cases).

Case	Inventory row	Start	Field removed
CT-CFG-025.01	R-0129	F-BASE	`project.name`
CT-CFG-025.02	R-0137	F-BASE	`environments[0].name`
CT-CFG-025.03	R-0138	F-BASE	`environments[0].model_version`
CT-CFG-025.04	R-0139	F-BASE	`environments[0].prompt_version`
CT-CFG-025.05	R-0140	F-BASE	`environments[0].tool_configuration` (the key itself; with the value `null`, F-BASE loads)
CT-CFG-025.06	R-0142	F-BASE	`environments[0].infra_target`
CT-CFG-025.07	R-0151	F-BASE	`suites[0].name`
CT-CFG-025.08	R-0155	F-BASE	`suites[0].scenarios[0].name`
CT-CFG-025.09	R-0175	F-BASE	the schema Constraint's `target`
CT-CFG-025.10	R-0176	F-BASE	the schema Constraint's `expected`
CT-CFG-025.11	R-0177	F-BASE	the Expectation's `criteria`
CT-CFG-025.12	R-0185	F-BASE	the Expectation's `judge.model`
CT-CFG-025.13	R-0187	F-BASE	the Expectation's `judge.confidence_source`
CT-CFG-025.14	R-0216	F-BASE	`quality_gates[0].name`
CT-CFG-025.15	R-0217	F-BASE	`quality_gates[0].aggregate`
CT-CFG-025.16	R-0218	F-BASE	`quality_gates[0].threshold`
CT-CFG-025.17	R-0219	F-BASE	`quality_gates[0].on_failure`
CT-CFG-025.18	R-0213	F-RETRY	`execution.retry.max_attempts`

Case	Inventory row	Start	Field removed
`CT-CFG-025.19`	R-0195	F-DS	`datasets[0].name`
`CT-CFG-025.20`	R-0198	F-DS	`datasets[0].source`
`CT-CFG-025.21`	R-0199	F-DS	`datasets[0].rows` (the Dataset is `static`)
`CT-CFG-025.22`	R-0202	F-DS	`datasets[0].rows[0].id`
`CT-CFG-025.23`	R-0203	F-DS	`datasets[0].rows[0].variables`
`CT-CFG-025.24`	R-0211	F-DS	`datasets[0].mechanisms[0].type`
`CT-CFG-025.25`	R-0132	F-SEED	`defect_classes[0].name`

- **Expected:** In each case, the configuration is rejected (1.5). `CT-CFG-025.24` is `NOT_APPLICABLE` unless `dataset_mechanism:edge_cases` is claimed: an implementation that does not claim it rejects F-DS's mechanism itself (Appendix A §A.8), so the starting configuration does not load. `CT-CFG-025.25` is `NOT_APPLICABLE` unless `seeded_controls` is claimed. (The other branch of `CT-CFG-025.21`, a Dataset that is not `static` loading without `rows`, belongs to `dataset:<source>`, Section 24.)
- **Fails if:** A case is `FAIL` when its configuration loads, including with a default supplied silently for the missing field. The test's own outcome summarizes its cases (1.2).

CT-CFG-026 — A Gate that no clause can feed is rejected

- **Capability:** core
- **Requirement:** Appendix B §B.3, Gate coverage: "A Quality Gate whose `aggregate` is `functional`, `semantic`, `operational`, or `safety` MUST be rejected unless at least one clause in its scope, outside a Seeded Control, has that `dimension`, given or by default"; Appendix A §A.10, `aggregate`; Volume I, Chapter 6, Quality Gates.
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`. Variants:
  - (a) the Expectation removed from `s-greeting`;
  - (b) a further Gate `{ name: operational-gate, aggregate: operational, threshold: 0.5, on_failure: warn }`, with no `latency` or `cost` Constraint;
  - © F-BASE with `semantic-gate`, unchanged.

  As a governed change after © loads, remove the Expectation from `s-greeting`.

- **Action:** Load (a), (b), and ©; then make the change.
- **Expected:** (a) and (b) are rejected. © loads. The change is rejected, and `s-greeting`, retrieved, still has its Expectation.
- **Fails if:** (a) or (b) loads; © is rejected; or the change takes effect.

CT-CFG-027 — A lower level cannot loosen how Constraints combine

- **Capability:** core
- **Requirement:** Volume VII, Contract Composition: "The effective `validator_composition` … is the strictest one contributed, in the order `hard-veto`, then `AND`, then `OR`; a lower level that contributes a looser value than a higher one has contributed is a conflict, surfaced like any other"; Appendix B §B.3, Composition conflict surfacing; Appendix A §A.5, Policies.
- **Method:** Automated
- **Fixture:** F-BASE, with a Project-level Contract holding one Constraint (`validator: schema`, `target: response`, `expected: { type: string }`) and `policies: { validator_composition: <P> }`, and `s-greeting`'s Contract given `policies: { validator_composition: <S> }`. Variants, as (P, S): (a) (`AND`, `OR`); (b) (`hard-veto`, `AND`); © (`OR`, `AND`).
- **Action:** Load each variant.
- **Expected:** (a) and (b) are rejected as conflicts. © loads.
- **Fails if:** (a) or (b) loads; or © is rejected.

CT-CFG-028 — A Dataset mechanism the implementation does not claim is rejected

- **Capability:** core
- **Requirement:** Appendix A §A.8, `mechanisms`: "Each `type` is a scoped capability, `dataset_mechanism: <type>` …; an implementation that does not claim it MUST reject a Dataset naming that `type` when the configuration is loaded."
- **Method:** Automated
- **Fixture:** F-DS (CT-CFG-025). Variants: `ds-1` given `mechanisms: [ { type: <t> } ]`, one variant for each `t` of `data_mutation`, `fuzzing`, `edge_cases`, and `adversarial`.
- **Action:** Load each variant.
- **Expected:** Each variant whose `dataset_mechanism:<t>` the Conformance Statement does not claim is rejected. Where every one is claimed, the test is `NOT_APPLICABLE` (1.2).
- **Fails if:** A variant whose mechanism is not claimed loads.

11. Execution and Evidence

Tests in this section run `cts-suite` against `cts-env` and observe the Executions, Conversations, and Artifacts that result, and the calls the scripted subject received.

CT-EXE-001 — Every Execution runs in exactly one Environment

- **Capability:** core
- **Requirement:** Volume II, Environment: “every Execution MUST run inside exactly one Environment.”
- **Method:** Automated
- **Fixture:** F-BASE.
- **Action:** Execute `cts-suite` against `cts-env`.
- **Expected:** The Execution record names exactly one Environment, `cts-env`.
- **Fails if:** The record names no Environment, more than one, or a different one.

CT-EXE-002 — Variable interpolation is deterministic

- **Capability:** core
- **Requirement:** Volume III, Variables: “the same Variables and the same Prompt template MUST produce the same literal Prompt text every time.”
- **Method:** Automated, Harness
- **Fixture:** F-BASE; scripted subject `SR-ECHO`.
- **Action:** Execute `cts-suite` twice.
- **Expected:** The scripted subject receives the literal Prompt `Hello` on both calls, and both Conversations record it identically.
- **Fails if:** The two Prompts differ in any character, or differ from `Hello`.

CT-EXE-003 — An infrastructure failure before a Response is retried, and leaves no Conversation

- **Capability:** core
- **Requirement:** Volume III, Retry: “A retried Execution of this kind produces no Conversation on the failed attempt and MUST NOT be treated as if it had; only the successful attempt is the Execution that produces a Result.”
- **Method:** Automated, Harness
- **Fixture:** F-BASE; add `execution: { retry: { max_attempts: 2, on: infrastructure_failure } }` to the Project; scripted subject `SR-503-THEN-OK`.
- **Action:** Execute `cts-suite`.
- **Expected:** The scripted subject receives two calls. One Execution of `s-greeting` is recorded, with one Conversation containing the `SR-OK` Response, and its Results.
- **Fails if:** A Conversation, Result, or Execution is recorded for the failed attempt; or no Execution is recorded.

CT-EXE-004 — A genuine Response is never retried

- **Capability:** core

- **Requirement:** Volume III, Retry: a re-run because "a Result later came back failing or low-Confidence … is not a Retry"; Appendix B §B.3, Retry scope: "RetryConfig MUST NOT trigger on Result-level failures."
- **Method:** Automated, Harness
- **Fixture:** As CT-EXE-003, with scripted subject `SR-EMPTY`.
- **Action:** Execute `cts-suite`.
- **Expected:** The scripted subject receives exactly one call. The schema Constraint's Result is `verdict: fail`.
- **Fails if:** The subject receives a second call within the same Execution, or the failing Result is absent or replaced.

CT-EXE-005 — A later Execution does not replace an earlier Result

- **Capability:** core
- **Requirement:** Volume III, Retry: "the original Execution's Result stands and MUST be recorded, not discarded in favor of a more favorable second attempt."
- **Method:** Automated
- **Fixture:** F-BASE.
- **Action:** Execute `cts-suite` with scripted subject `SR-EMPTY`; then execute it again with `SR-OK`.
- **Expected:** Two Executions are recorded. The first keeps its failing schema Result, unchanged; the second has its own, passing one.
- **Fails if:** The first Execution's Result is removed, modified, or marked superseded.

CT-EXE-006 — Parallel Executions do not share state

- **Capability:** core
- **Requirement:** Volume III, Parallelization: "Executions running in parallel MUST NOT share mutable Context … or Variables state with one another."
- **Method:** Automated, Harness
- **Fixture:** F-MULTI; add `execution: { parallelization: 3 }`; scripted subject `SR-ECHO`.
- **Action:** Execute `cts-suite`.
- **Expected:** Each of `s-1`, `s-2`, `s-3` has a Conversation whose Prompt and Response are its own `query` (`One`, `Two`, `Three`), and nothing else.
- **Fails if:** Any Conversation contains another Scenario's value.

CT-EXE-007 — Two Executions of one Scenario run in an identical harness

- **Capability:** core
- **Requirement:** Volume I, Chapter 3, Deterministic Infrastructure: "Two Executions of the same Scenario in the same Environment MUST go through an identical harness"; Volume III, Runtime.
- **Method:** Automated
- **Fixture:** F-BASE.
- **Action:** Execute `cts-suite` twice.
- **Expected:** Both Execution records show the same resolved configuration: Environment (`model_version`, `prompt_version`, `tool_configuration`, `infra_target`), the Contract (its version, or its content where it is inline), and the Prompt template.
- **Fails if:** Any of these differs between the two records, or is not recorded.

CT-EXE-008 — Evidence is captured with every Execution

- **Capability:** core
- **Requirement:** Volume I, Chapter 4, Logical Layers (Evidence); §8, Execution: “An Execution produces exactly one Conversation, zero or more Artifacts”; Volume III, Artifact Collection.
- **Method:** Automated
- **Fixture:** F-BASE.
- **Action:** Execute `cts-suite`.
- **Expected:** The Execution has exactly one retrievable Conversation, containing the Prompt `Hello` and the `SR-OK` Response exactly as the subject returned it. Any Artifacts the implementation records for the turn are retrievable with it.
- **Fails if:** The Conversation is missing, there is more than one, or its content differs from what was sent and returned.

CT-EXE-009 — Why an Execution ran when it did can be reconstructed

- **Capability:** core
- **Requirement:** Volume III, Scheduling: “*why* a given Execution ran when it did MUST be reconstructable.”
- **Method:** Inspection
- **Fixture:** F-MULTI.
- **Action:** Execute `cts-suite`. From the implementation’s records alone, establish for each of the three Executions what triggered it and its position in the order.
- **Expected:** The reviewer can state both for every Execution, citing the records used.
- **Fails if:** For any Execution, the trigger or the order cannot be established from the records.

CT-EXE-010 — An Execution draws rows from the Dataset version it names

- **Capability:** core
- **Requirement:** Volume IV, Versioning: “a specific Execution’s Variable Resolution (Volume III) is only reproducible if the Dataset row it drew from is pinned to a specific version.”
- **Method:** Automated, Harness
- **Fixture:** F-BASE with `variables` removed from `s-greeting` and `dataset: ds-1@1.0.0`, where `ds-1@1.0.0` is CT-CFG-011’s Dataset (one row, `query: "Hi"`); scripted subject `SR-ECHO`.
- **Action:** Execute `cts-suite`. Register `ds-1@1.1.0` with one row, `query: "Bye"`. Execute `cts-suite` again without changing the Scenario.
- **Expected:** The scripted subject receives the Prompt `Hi` both times, and each Execution record names `ds-1@1.0.0` and the row `r1`.
- **Fails if:** The second Execution uses `Bye`, or either record does not identify the Dataset version and row.

CT-EXE-011 — The Conversation Engine is deterministic given the same Responses

- **Capability:** core
- **Requirement:** Volume III, Conversation Engine: its own logic, “how it decides a Conversation is complete, how it constructs the next turn’s Prompt given the Context accumulated so far — MUST itself be deterministic given the same Context and Responses so far.”
- **Method:** Harness

- **Fixture:** F-BASE, scripted subject `SR-ECHO`. Where the implementation runs multi-turn Scenarios, the tester also uses one of its own documented multi-turn Scenarios, with the scripted subject returning the same fixed sequence of Responses on every run.
- **Action:** Execute each Scenario twice. Compare, call by call, every request the scripted subject received.
- **Expected:** Both runs send the same number of requests, and each request is identical, character for character, to its counterpart in the other run, apart from identifiers of the Execution itself. Both Conversations end after the same turn.
- **Fails if:** The runs differ in the number of turns or in any request.

CT-EXE-012 — Infrastructure facts of a turn are captured as they happened

- **Capability:** core
- **Requirement:** Volume III, Artifact Collection: what it "records about the infrastructure-level facts of a turn — the exact request a tool call sent, the exact latency observed — MUST itself be captured deterministically." (The request a tool call sent belongs to `action_boundary`, Section 23.)
- **Method:** Automated, Harness
- **Fixture:** F-BASE; scripted subject `SR-SLOW-200`. Add to the Scenario's Constraints:

  ```
  - validator: latency
    target: artifacts.duration_ms
    expected: { max: 150 }
  ```

  (or the implementation's documented path for observed latency).
- **Action:** Execute `cts-suite`; retrieve the Artifacts and the latency Constraint's Result. Then switch to `SR-OK` and execute again.
- **Expected:** The first Execution's Artifacts record a latency of at least the delay the harness observed, which is at least 200 ms, and its latency Result is `fail`. In each Execution, the latency Result agrees with the latency that Execution's Artifacts record: `pass` when it is at most 150 ms, `fail` when above. (An implementation's own overhead counts in what it observes; the test does not require the second Execution to pass.)
- **Fails if:** A recorded latency is below what the harness observed for that Execution; or a latency Result contradicts the latency its own Execution recorded.

12. Assessment

Tests in this section observe the Results Oracles produce for one Execution, and the calls the scripted Judge receives.

CT-ASM-001 — A failing Validator short-circuits before any Judge

- **Capability:** core
- **Requirement:** Volume I, Chapter 3, Validation before Evaluation: “Validators MUST run before Judges for a given Execution”; Volume V, Execution Order.
- **Method:** Automated, Harness
- **Fixture:** F-BASE; scripted subject `SR-EMPTY`.
- **Action:** Execute `cts-suite`.
- **Expected:** The schema Constraint’s Result has `verdict: fail`. The scripted Judge receives no call for this Execution.
- **Fails if:** The scripted Judge receives any call for this Execution.

CT-ASM-002 — A Judge is invoked only after Validation has passed

- **Capability:** core
- **Requirement:** as CT-ASM-001; Volume I, Chapter 5, Validation: Validation “MUST do so before Evaluation runs.”
- **Method:** Automated, Harness
- **Fixture:** F-BASE.
- **Action:** Execute `cts-suite`; retrieve both Results and compare their `recorded_at` (Appendix C §C.7).
- **Expected:** The scripted Judge receives exactly one call. The schema Constraint’s Result has an earlier `recorded_at` than the Expectation’s.
- **Fails if:** The Judge is called more than once, or the Validator Result’s `recorded_at` is not earlier than the Judge Result’s.

CT-ASM-003 — A Validator Result declares Confidence inapplicable

- **Capability:** core
- **Requirement:** Volume I, Chapter 3, Everything has Confidence: a Validator’s Result “MUST instead explicitly declare that Confidence does not apply”; Volume V, Validator Architecture; Appendix C §C.7.
- **Method:** Automated
- **Fixture:** F-BASE.
- **Action:** Execute `cts-suite`; retrieve the schema Constraint’s Result.
- **Expected:** `oracle_type: validator`; `confidence: "not_applicable"`; `verdict: pass`; `disposition: actionable`; `oracle` names the schema Validator and its version.
- **Fails if:** `confidence` is a number, is absent, or is bare `null`; or `oracle` is absent.

CT-ASM-004 — A Judge Result carries the Judge’s Confidence

- **Capability:** core

- **Requirement:** §8, Judge: “A Judge’s Result MUST carry a Confidence value”; Volume VI, Judge Architecture.
- **Method:** Automated
- **Fixture:** F-BASE; scripted Judge `J-PASS-90`.
- **Action:** Execute `cts-suite`; retrieve the Expectation’s Result.
- **Expected:** `oracle_type: judge`; `verdict: pass`; `confidence: 0.9`; `disposition: actionable` (0.9 meets the 0.8 threshold).
- **Fails if:** `confidence` is not `0.9`, or `disposition` is anything but `actionable`.

CT-ASM-005 — Below the threshold, a Result is inconclusive and keeps its verdict

- **Capability:** core
- **Requirement:** Volume I, Chapter 6, Confidence Model: below the threshold, “a Result’s disposition is `inconclusive` — it MUST NOT be silently counted as an ordinary pass or fail.”
- **Method:** Automated
- **Fixture:** F-BASE; scripted Judge `J-PASS-60`.
- **Action:** Execute `cts-suite`; retrieve the Expectation’s Result.
- **Expected:** `verdict: pass`; `confidence: 0.6`; `disposition: inconclusive`.
- **Fails if:** `disposition` is `actionable`, or the verdict is changed or dropped. (That an `inconclusive` Result is not counted is checked at the Gate, CT-DEC-005.)

CT-ASM-006 — Every Result has the required shape

- **Capability:** core
- **Requirement:** Appendix C §C.7, Result Response Shape; Appendix B §B.3, Result disposition validity: “`verdict` MUST be `null` when `disposition` is `awaiting_review`, `oracle_unavailable`, or `not_assessed`, and MUST be `pass` or `fail` otherwise. `confidence` MUST NOT be bare `null` under any disposition.”
- **Method:** Automated
- **Fixture:** F-BASE.
- **Action:** Execute `cts-suite` four times, with scripted Judge `J-PASS-90`, `J-PASS-60`, `J-FAIL-90`, and `J-DOWN` in turn; retrieve every Result.
- **Expected:** Every Result has `id`, `execution`, `clause`, `oracle_type`, `oracle`, `verdict`, `confidence`, `disposition`, `supporting_evidence`, and `recorded_at`, and satisfies the rule quoted above.
- **Fails if:** Any Result lacks one of those fields or breaks the rule.

CT-ASM-007 — An unreachable Judge produces `oracle_unavailable`

- **Capability:** core
- **Requirement:** Volume VI, Human Review: an Oracle that cannot be reached “produces a Result with disposition `oracle_unavailable` rather than a low-Confidence verdict”; Appendix C §C.7.
- **Method:** Automated
- **Fixture:** F-BASE; scripted Judge `J-DOWN`.
- **Action:** Execute `cts-suite`; retrieve the Expectation’s Result; count the calls the scripted Judge received.
- **Expected:** `disposition: oracle_unavailable`; `verdict: null`; `confidence: "not_applicable"`. Its `supporting_evidence` records one attempt for each call the scripted Judge received (Volume VI, Human

Review).
- **Fails if:** No Result is recorded for the Expectation; it has a verdict; it is recorded as `fail` or as a low-Confidence Result; or the attempts recorded do not match the calls received.

CT-ASM-008 — A re-attempt after a transport failure is recorded, not silent

- **Capability:** core
- **Requirement:** Volume VI, Human Review: a call that fails in transport before any verdict "MAY be re-attempted within the same assessment … Every attempt is recorded in the Result's `supporting_evidence`"; an Oracle that could not be reached "MUST NOT be silently retried until a favorable verdict appears."
- **Method:** Automated, Harness
- **Fixture:** F-BASE; scripted Judge `J-DOWN-THEN-PASS-90`.
- **Action:** Execute `cts-suite`; count the calls the scripted Judge received; retrieve every Result for the Expectation.
- **Expected:** One of two outcomes. (a) The implementation does not re-attempt: one call, and an `oracle_unavailable` Result. (b) It re-attempts: two calls, and one Result with `verdict: pass` whose `supporting_evidence` records both attempts, including the failed one.
- **Fails if:** The Judge received more than one call and no Result records the failed attempt; or more than one Result was recorded for the Expectation within this Execution without `resolves`.

CT-ASM-009 — A Validator returns the same Result for the same input

- **Capability:** core
- **Requirement:** §8, Validator: "A Validator MUST return the same Result for the same input"; Volume V, Validator Architecture.
- **Method:** Automated
- **Fixture:** F-BASE; scripted subject `SR-OK`.
- **Action:** Execute `cts-suite`. If the implementation offers a re-check of a Validator on stored Evidence (Volume V, Validator SDK), re-check the schema Constraint on that Execution's stored Evidence and compare the re-check's Result with the recorded one. Otherwise, execute `cts-suite` a second time, confirm from the two Conversations that the Response, the only input the schema Constraint reads, is identical, and compare the two schema Constraint Results. The report states which path was taken.
- **Expected:** The two Results have the same `verdict`, `confidence`, `disposition`, `clause`, and `oracle`, and the same `supporting_evidence` apart from references to their own Execution. `id`, `execution`, and `recorded_at` are not compared.
- **Fails if:** Any compared field differs.

CT-ASM-010 — Built-in Validators depend only on evidence and configuration

- **Capability:** core
- **Requirement:** Volume V, Validator Architecture: "a Validator's Result MUST depend only on its target evidence … and on the Validator's own versioned configuration — not on the clock, a random source, a network call, or any other state."
- **Method:** Declaration
- **Fixture:** None.

- **Action:** Ask the implementation to state, for each built-in Validator it offers, what inputs its result depends on.
- **Expected:** For every built-in Validator, the statement names only target evidence and versioned configuration.
- **Fails if:** Any statement names, or the reviewer finds, a dependency on the clock, randomness, the network, or other mutable state.

CT-ASM-011 — A Judge receives the Contract's criteria explicitly

- **Capability:** core
- **Requirement:** Volume VI, Judge Architecture: "A Judge's architecture MUST make its criteria explicit and inspectable"; Volume VII, Expectations.
- **Method:** Harness
- **Fixture:** F-BASE.
- **Action:** Execute `cts-suite`; read the request the scripted Judge received.
- **Expected:** The request contains the Expectation's `criteria` text, `The response greets the user.`, and the Response being assessed.
- **Fails if:** The criteria text is absent from what the Judge received.

CT-ASM-012 — Every Result traces to exactly one Oracle

- **Capability:** core
- **Requirement:** §8, Oracle: "Every Result MUST be traceable to exactly one Oracle"; Appendix C §C.7, `oracle`.
- **Method:** Automated
- **Fixture:** F-BASE.
- **Action:** Execute `cts-suite`; retrieve every Result.
- **Expected:** Each Result has exactly one `oracle` and one `oracle_type`, and each Result's `oracle` matches the Validator or Judge configured for its `clause`.
- **Fails if:** Any Result names no Oracle, several, or one not configured for its clause.

CT-ASM-014 — A Validator that cannot complete produces `oracle_unavailable` and stops Evaluation

- **Capability:** core
- **Requirement:** Appendix B §B.3, Validator unavailable: "A Validator that cannot complete its check MUST produce `oracle_unavailable`, never `fail`, and Validation for that Execution has not passed"; Appendix C §C.7, `oracle_unavailable`.
- **Method:** Automated, Harness
- **Fixture:** F-BASE; add to the Contract's `constraints`:

```
- validator: cts-crash
  target: response
  expected: {}
```

`cts-crash` is a Custom Validator (Volume V) that raises an error on every call and returns no Result. Where the implementation offers no way to define one but documents a test-only way to make a Validator unable to complete its check (a fault hook), the tester may use that instead, on the schema Constraint, and the report names the hook. With neither, the outcome is `NOT_RUN`, with that reason: no built-in Validator can be made to fail on demand through configuration alone, so the requirement is then not verified for that implementation, and the test report lists it as such (1.7).

- **Action:** Execute `cts-suite`; retrieve every Result.
- **Expected:** The `cts-crash` Result has `oracle_type: validator`, `disposition: oracle_unavailable`, `verdict: null`, and `confidence: "not_applicable"`. The scripted Judge receives no call for this Execution.
- **Fails if:** The `cts-crash` Result is `fail`, `pass`, or absent; or the scripted Judge is called.

CT-ASM-015 — Once a verdict arrives, it stands

- **Capability:** core
- **Requirement:** Volume VI, Human Review: “Once any response carrying a verdict arrives, it stands, whatever it says”; Volume III, Retry.
- **Method:** Automated, Harness
- **Fixture:** F-BASE; scripted Judge `J-FAIL-THEN-PASS-90`.
- **Action:** Execute `cts-suite`; count the calls the scripted Judge received; retrieve the Expectation’s Result.
- **Expected:** One call. The Result has `verdict: fail`, `confidence: 0.9`, and `disposition: actionable`.
- **Fails if:** The Judge receives a second call, or the Result is `pass`.

CT-ASM-013 — A panel’s Result states how many Judges were configured and counted

- **Capability:** `multi_judge`
- **Requirement:** Volume VI, Consensus: “a Result produced by a panel MUST state how many Judges were configured and how many were counted”; Appendix B §B.3, Panel reporting.
- **Method:** Automated
- **Fixture:** F-BASE; in the Expectation, replace `judge` with:

```
multi_judge:
  purpose: independence
  judges:
    - model: cts-scripted-judge-a@1
      confidence_source: self_rating
    - model: cts-scripted-judge-b@1
      confidence_source: self_rating
  consensus:
    method: voting
    threshold: unanimous
```

  Both identifiers resolve to the scripted Judge, which behaves as `J-PASS-90` for each.
- **Action:** Execute `cts-suite`; retrieve the Expectation’s Result.
- **Expected:** `oracle_type: judge`; `verdict: pass`; `panel: { configured: 2, counted: 2 }`. The scripted Judge received one call for each identifier.
- **Fails if:** `panel` is absent or its counts differ.

CT-ASM-016 — A Custom Validator's Result has a Validator's shape

- **Capability:** core
- **Requirement:** Volume V, Custom Validators: "A Custom Validator MUST satisfy the same constraints as a built-in one: the same input MUST always produce the same Result, and it MUST NOT carry a Confidence value"; Volume V, Validator SDK: a Validator that completes its check "MUST return a Result carrying a pass/fail verdict, an explicit Confidence-inapplicable declaration, and supporting evidence sufficient for a Report … to explain the verdict without re-running the check."
- **Method:** Automated
- **Fixture:** F-BASE; add a Constraint `validator: cts-len`, `target: response`, `dimension: functional`, and `expected` in the form the implementation documents for a Custom Validator that takes no parameter (Appendix A §A.5), for example `{}`.
- **Action:** Execute `cts-suite` twice; retrieve the `cts-len` Results.
- **Expected:** Each has `oracle_type: validator`, `verdict: pass`, `confidence: "not_applicable"`, `disposition: actionable`, an `oracle` naming `cts-len` and its version, and non-empty `supporting_evidence`. The two Results have the same `oracle_type`, `oracle`, `clause`, `verdict`, `confidence`, and `disposition`, and the same `supporting_evidence` apart from references to their own Execution. `id`, `execution`, and `recorded_at` are not compared.
- **Fails if:** `confidence` is a number; `supporting_evidence` is empty; or any compared field differs.

CT-ASM-017 — A check that calls a model is not accepted as a Validator

- **Capability:** core
- **Requirement:** Volume V, Custom Validators: a check that calls a model to decide pass or fail "is structurally a Judge (Volume VI) and MUST be modeled as one."
- **Method:** Inspection
- **Fixture:** None.
- **Action:** Review how the implementation lets a Custom Validator be defined, and attempt to register one whose check calls `cts-scripted-judge@1` to decide its verdict.
- **Expected:** The implementation prevents it: registration is refused, the Validator's runtime gives it no way to reach a model, or the implementation's documentation requires such a check to be configured as a Judge and the attempt fails accordingly. The reviewer states which.
- **Fails if:** The model-calling check is registered and runs as a Validator, with a Validator's short-circuit and no Confidence.

CT-ASM-018 — A re-check that disagrees is reported as a Validator defect

- **Capability:** core
- **Requirement:** Volume V, Validator SDK: an implementation "MAY re-check a Validator at any time by running it again on stored Evidence; a Result that differs from the one recorded is a defect in the Validator, not in the subject, and MUST be reported as such."
- **Method:** Automated
- **Fixture:** F-BASE; add a Constraint `validator: cts-flaky`, `target: response`, `expected: {}`, `dimension: functional`.
- **Action:** Execute `cts-suite`. Ask the implementation to re-check `cts-flaky` on the stored Evidence.

- **Expected:** The re-check produces `fail` where the recorded Result was `pass`, and the implementation reports this as a defect in `cts-flaky`, not as a failure of the subject. The recorded Result is unchanged. If the implementation offers no re-check, which the specification permits but does not require, the outcome is `NOT_APPLICABLE` (1.2).
- **Fails if:** The disagreement is reported as a subject failure or not reported at all; or the recorded Result is changed.

CT-ASM-019 — A Judge Result over several assertions says it is an aggregate

- **Capability:** core
- **Requirement:** Volume VI, Single Judge: where a criterion spans multiple independent assertions, “the Result MUST state that it is an aggregate over multiple assertions rather than present its score as an atomic verdict on a single one”; Appendix C §C.7, `assertions`.
- **Method:** Automated
- **Fixture:** Two variants of F-BASE. (a) The Expectation’s `criteria` set to `"The response greets the user, offers help, and names no product."`, with scripted Judge `J-PASS-90-A3`. (b) F-BASE unchanged, with `J-PASS-90`.
- **Action:** Execute `cts-suite` for each; retrieve the Expectation’s Result.
- **Expected:** (a) The Result has `assertions`, an integer greater than 1; the test report records its value. (b) The Result has no `assertions`. (How the count is obtained is the implementation’s: from the Judge, as `J-PASS-90-A3` reports it, or by its own reading of the criterion. The specification requires only that a Result over several assertions says so, so any count above one meets it.)
- **Fails if:** (a) has no `assertions`, or a value that is not an integer greater than 1; or (b) has one.

CT-ASM-020 — A Custom Validator’s `expected` in a form it does not accept is rejected

- **Capability:** core
- **Requirement:** Appendix A §A.5, `expected`: “For a Custom Validator, the structure is the one the implementation documents for it; a structure it does not accept MUST be rejected when the configuration is loaded, never ignored.”
- **Method:** Automated
- **Fixture:** CT-ASM-016’s configuration. Variant: the `cts-len` Constraint’s `expected` replaced by a structure the implementation’s documentation for `cts-len` does not accept, for example `{ minimum: "one" }`. The tester states which.
- **Action:** Load CT-ASM-016’s configuration; load the variant.
- **Expected:** The first loads; the variant is rejected.
- **Fails if:** The variant loads, including with `expected` ignored.

13. Decision

Tests in this section evaluate Quality Gates over the Results of one or more Executions and observe the GateResults (Appendix A §A.12), their Aggregate Basis (Appendix C §C.3), and how resolution and Seeded Controls change what a Gate reads.

13.1 Fixtures and conventions for this section

Evaluating the Gates. "Evaluate the Gates" means `POST /projects/cts-base/gates/check` for the batch the Action produced, or the implementation's equivalent. It returns one GateResult per applicable Gate and a `snapshot_id`.

`semantic-gate`. Every test in this section adds a second blocking Gate to its fixture's Project-level `quality_gates`:

```
- name: semantic-gate
  aggregate: semantic
  threshold: 1.0
  on_failure: block_release
```

Which Gate reads which clause. By the `dimension` defaults of Appendix A §A.5: `functional-gate` reads the schema Constraint, `semantic-gate` the Expectation, and `safety-gate` the forbidden-content Constraint.

Recording a Human Reviewer Result. "As `P-REVIEWER`, record …" means `POST .../results/{result}/review` (Appendix C §C.3) on the Result named, made by `P-REVIEWER`: a Person holding `review` and the Role `reviewer`.

Requesting a re-assessment. "Request that … assess the stored Evidence again" means `POST .../results/{result}/reassess` on the Result named, made by `P-RELEASE` (Section 14.1), naming the Oracle configuration. Appendix C §C.3 specifies `reassess` as an operation of a conformant implementation's API. An implementation that offers neither it nor an equivalent fails every test that requests a re-assessment, and the test report says that the operation is missing.

Additional scripted Judge behaviors.

Code	Behavior
`J-MAP(a, b, c)`	With scripted subject `SR-ECHO` and F-MULTI: behaves as `a` for the Response `One`, `b` for `Two`, `c` for `Three`.
`J-CATCH`	Verdict `fail`, Confidence `0.90` for the Responses `Goodbye.` and `Bye now.`; otherwise as `J-PASS-90`.

F-SEED — a Seeded Control. As F-BASE, plus a defect class on the Project and a second Scenario in `cts-suite`, confirmed by construction:

```
project:
  defect_classes:
    - name: greeting-missing
  suites:
    - name: cts-suite
      scenarios:
        # s-greeting as in F-BASE, then:
        - name: s-control
          seeded_control:
            defect_class: greeting-missing
            planted_defect: "The response does not greet the user."
            confirmed_by: construction
          evidence:
            conversation:            # in the implementation's own Conversation form
              - role: user
                content: "Hello"
              - role: assistant
                content: "Goodbye."
          contract:
            expectations:
              - criteria: "The response greets the user."
                defect_classes: [greeting-missing]
                judge:
                  model: cts-scripted-judge@1
                  confidence_source: self_rating
                confidence_threshold: 0.8
```

`s-control` and `s-greeting` bind the same Oracle: the same Judge model and the same criteria. The Expectation in `s-greeting` lists no `defect_classes`, so it is treated as covering every class its Oracle is tested on (Appendix A §A.5). With `J-PASS-90`, the Judge passes `Goodbye.` and so misses the control; with `J-CATCH`, it catches it.

13.2 Tests

CT-DEC-001 — A passing GateResult has the required shape

- **Capability:** core
- **Requirement:** Appendix A §A.12, GateResult; Appendix C §C.3, Quality Gate: `gates/check` “Returns one GateResult (Appendix A §A.12) per Gate and the `snapshot_id` captured for the evaluation.”
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-PASS-90`.
- **Action:** Execute `cts-suite`; evaluate the Gates.
- **Expected:** A `snapshot_id`, and one GateResult each for `functional-gate` and `semantic-gate`, each with an `id`, `status: pass`, `blocking: true`, no `reasons`, a numeric `value`, `threshold: 1.0`, a `basis` object, and no `override`. The two `id`s differ.
- **Fails if:** Either GateResult is missing, any of those fields is absent or differs, the two share an `id`, `value` is bare `null`, or no `snapshot_id` is returned.

CT-DEC-002 — The Aggregate Basis counts what the aggregate rests on

- **Capability:** core
- **Requirement:** Volume I, Chapter 6, Aggregate Basis: “Every aggregate a Quality Gate reads MUST therefore state its **Aggregate Basis**”; Appendix C §C.3, Aggregate Basis Reporting.
- **Method:** Automated
- **Fixture:** F-MULTI with `semantic-gate`; scripted Judge `J-PASS-90`.
- **Action:** Execute `cts-suite`; evaluate the Gates.
- **Expected:** `semantic-gate`’s `basis` is `results: 3`, `not_actionable: 0`, `scenarios: 3`, `executions: 3`, `min_basis_met: "not_declared"`.
- **Fails if:** `basis` is absent, any field is missing, or any count differs.

CT-DEC-003 — A minimum met and no minimum declared are reported differently

- **Capability:** core
- **Requirement:** Appendix C §C.3: a Gate that set no minimum and a Gate whose minimum was met “are different facts and MUST NOT be reported the same way.”
- **Method:** Automated
- **Fixture:** Two variants of F-MULTI with `semantic-gate`; scripted Judge `J-PASS-90`. (a) `semantic-gate` as given. (b) `semantic-gate` with `min_basis: { scenarios: 3 }`.
- **Action:** For each variant, execute `cts-suite` and evaluate the Gates.
- **Expected:** `semantic-gate` passes in both. `min_basis_met` is `"not_declared"` in (a) and `true` in (b).
- **Fails if:** The two values are the same, either is bare `null` or absent, or either Gate fails.

CT-DEC-004 — An unmet minimum basis fails as insufficient basis, not as a threshold failure

- **Capability:** core
- **Requirement:** Volume I, Chapter 6, Quality Gates: “the Gate MUST NOT pass, and the reason MUST be recorded as insufficient basis rather than as a threshold failure”; Appendix B §B.3, Aggregate Basis reporting.
- **Method:** Automated
- **Fixture:** F-MULTI with `semantic-gate`, given `min_basis: { scenarios: 4 }`; scripted Judge `J-PASS-90`.
- **Action:** Execute `cts-suite`; evaluate the Gates.
- **Expected:** `semantic-gate` has `status: fail`, `reasons: [insufficient_basis]`, and `basis.min_basis_met: false`.
- **Fails if:** The Gate passes, `reasons` lacks `insufficient_basis`, or `reasons` contains `threshold` (every Result passed).

CT-DEC-005 — A failing verdict fails the threshold and nothing else

- **Capability:** core
- **Requirement:** Appendix A §A.12, GateResult `reasons`: `threshold` (“the value fell short”); Appendix B §B.3, Gate reasons.
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-FAIL-90`.

- **Action:** Execute `cts-suite`; evaluate the Gates.
- **Expected:** `semantic-gate`: `status: fail`, `reasons: [threshold]`, a `value` below 1.0, `basis.results: 1`, `basis.not_actionable: 0`. `functional-gate`: `status: pass`.
- **Fails if:** `semantic-gate` passes or lists any other reason; or `functional-gate` fails.

CT-DEC-006 — An inconclusive Result blocks its Gate

- **Capability:** core
- **Requirement:** Volume I, Chapter 6, Quality Gates: a Gate reading an aggregate that includes an effective Result whose effective state is not `actionable` "MUST treat that Result as blocking, and MUST record why"; Volume I, Chapter 6, Decision Pipeline: it "MUST NOT skip the Confidence Model and aggregate raw pass/fail counts directly"; Appendix B §B.3, Gate reasons.
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-PASS-60`.
- **Action:** Execute `cts-suite`; evaluate the Gates.
- **Expected:** `semantic-gate`: `status: fail`; `reasons` includes `non_actionable_results`; `basis.results: 0`; `basis.not_actionable: 1`; `value` is a number or `"not_computed"`.
- **Fails if:** The Gate passes; `reasons` lacks `non_actionable_results`; the inconclusive Result is counted in `basis.results`; or `value` is bare `null`.

CT-DEC-007 — An unreachable Judge blocks its Gate

- **Capability:** core
- **Requirement:** as CT-DEC-006; Appendix C §C.7, `oracle_unavailable`: "MUST NOT be treated as an implicit pass."
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-DOWN`.
- **Action:** Execute `cts-suite`; evaluate the Gates.
- **Expected:** As CT-DEC-006.
- **Fails if:** As CT-DEC-006.

CT-DEC-008 — A failed GateResult lists every reason that applies

- **Capability:** core
- **Requirement:** Appendix B §B.3, Gate reasons: "A failed GateResult MUST list every applicable reason"; Appendix A §A.12, `reasons`: "Every reason that applies, none omitted."
- **Method:** Automated
- **Fixture:** F-MULTI with `semantic-gate`, given `min_basis: { results: 3 }`; scripted subject `SR-ECHO`; scripted Judge `J-MAP(J-FAIL-90, J-PASS-60, J-PASS-90)`.
- **Action:** Execute `cts-suite`; evaluate the Gates.
- **Expected:** `semantic-gate` has `status: fail`, and `reasons` contains `threshold` (`s-1` failed), `non_actionable_results` (`s-2` is inconclusive), and `insufficient_basis` (two `actionable` Results, three required). `basis.results: 2`, `basis.not_actionable: 1`.
- **Fails if:** Any of the three reasons is missing.

CT-DEC-009 — No configuration lets a non-actionable Result pass a Gate

- **Capability:** core
- **Requirement:** Appendix C §C.7: “There is no configuration that turns this off: the way past a non-`actionable` Result is to resolve it, or, for a blocking Gate, an override approved for that one Gate Result.”
- **Method:** Inspection
- **Fixture:** None.
- **Action:** Review every configuration option the implementation documents for Quality Gates, Contracts, Aggregation, and the Decision Pipeline.
- **Expected:** None of them causes a Gate to pass while its aggregate includes an effective Result that is not `actionable`.
- **Fails if:** Any option does, whatever it is called.

CT-DEC-010 — A Human Reviewer’s Result resolves an inconclusive one, which is not counted again

- **Capability:** `human_review`
- **Requirement:** Volume I, Chapter 6, Result Resolution: the effective Result is “the last Result in that chain. Every earlier Result stays in the record … and is not counted again by any aggregate or Quality Gate”; Appendix B §B.3, Result resolution: “A resolved Result MUST NOT be counted again, including in `basis.not_actionable`.”
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-PASS-60`.
- **Action:** Execute `cts-suite`; evaluate the Gates. As `P-REVIEWER`, record a Human Reviewer Result with verdict `pass` for the Expectation, resolving the Judge’s Result. Evaluate the Gates again.
- **Expected:** The first evaluation fails as in CT-DEC-006. The Human Reviewer Result has `oracle_type: human_reviewer`, `resolves` naming the Judge’s Result, and `disposition: actionable`. In the second evaluation, `semantic-gate` passes with `basis.results: 1` and `basis.not_actionable: 0`. The Judge’s Result is still retrievable, unchanged, and the first GateResult is still recorded as it was.
- **Fails if:** The second evaluation fails; the Judge’s Result is counted in either basis field; the Judge’s Result is modified or removed; or the first GateResult is altered.

CT-DEC-011 — A Result awaiting review blocks, then completes

- **Capability:** `human_review`
- **Requirement:** Appendix C §C.7, `awaiting_review`: “This is the one disposition a Result leaves: it completes once the review returns”; Volume VI, Human Review: an unanswered review “is never silently treated as a pass.”
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; in the Expectation, replace `judge` and `confidence_threshold` with `human_reviewer: { role: reviewer }`.
- **Action:** Execute `cts-suite`; evaluate the Gates. As `P-REVIEWER`, return verdict `pass` on the pending review. Evaluate the Gates again.
- **Expected:** Before the review, the Expectation’s Result has `disposition: awaiting_review`, `verdict: null`, `confidence: "not_applicable"`, and `semantic-gate` fails with `non_actionable_results`. After

it, the Result with the same `id` has `verdict: pass`, `disposition: actionable`, and the same `recorded_at` as before, and `semantic-gate` passes.
- **Fails if:** `semantic-gate` passes before the review; the pending Result has a verdict; the Gate still fails after the review; or the review produced a second Result instead of completing the pending one.

CT-DEC-012 — The chain of resolutions is linear

- **Capability:** `human_review`
- **Requirement:** Appendix B §B.3, Result resolution: "`resolves` MUST reference the current effective Result for the same `execution` and `clause`, and that Result's effective state MUST NOT be `actionable`. A second Result resolving the same Result MUST be rejected."
- **Method:** Automated
- **Fixture:** Three variants. (a), (b): F-REVIEWED (2.7). ©: F-BASE with `semantic-gate`; scripted Judge `J-PASS-90`. (d): F-BASE with `semantic-gate`; scripted Judge `J-PASS-60`.
- **Action:** In © and (d), execute `cts-suite`. In each variant, record every Result of the run and evaluate the Gates; then make the variant's attempt; then record the Results again and evaluate the Gates again. The attempts, each a review of the Expectation (13.1): (a) as `P-REVIEWER`, verdict `fail`, resolving the Judge's Result again; (b) as `P-REVIEWER`, resolving the Human Reviewer Result recorded when F-REVIEWED was built; © as `P-REVIEWER`, resolving the Judge's `actionable` Result; (d) as `P-NOPERM`, who does not hold `review`, resolving the Judge's `inconclusive` Result.
- **Expected:** Each attempt is rejected with an error. In each variant, the Results after the attempt are the Results before it, none added, changed, or removed; the effective Result for the Expectation is the same; and the second evaluation gives every Gate the same `status` and `reasons` as the first. Any record the implementation keeps of the rejected attempt itself is not a Result.
- **Fails if:** An attempt is accepted; a Result is added, changed, or removed; the effective Result changes; or a Gate's `status` or `reasons` changes.

CT-DEC-013 — The same Oracle asked again does not resolve an inconclusive Result

- **Capability:** core
- **Requirement:** Volume I, Chapter 6, Result Resolution: "Apart from that last case, the same Oracle asked again resolves nothing: that is a retry until a favorable verdict appears, which Volume III prohibits."
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-PASS-60` for the first call and `J-PASS-90` for every later call.
- **Action:** Execute `cts-suite`. Request that the same Judge configuration assess the stored Evidence again. Evaluate the Gates.
- **Expected:** The request is rejected, or produces a Result that does not resolve the inconclusive one. `semantic-gate` fails with `non_actionable_results`.
- **Fails if:** A Result from the same Judge configuration resolves the inconclusive Result; `semantic-gate` passes; or the implementation offers no way to request a re-assessment (13.1).

CT-DEC-014 — The same Oracle may resolve its own `oracle_unavailable` Result

- **Capability:** core

- **Requirement:** Volume I, Chapter 6, Result Resolution: “a Result from the same Oracle configuration, resolving an `oracle_unavailable` Result once that Oracle can be reached or run again”; Volume VI, Human Review.
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-DOWN`.
- **Action:** Execute `cts-suite`; evaluate the Gates. Switch the scripted Judge to `J-PASS-90`. Request that the same Judge configuration assess the stored Evidence again. Evaluate the Gates again.
- **Expected:** The first evaluation fails with `non_actionable_results`. The new Result has `resolves` naming the `oracle_unavailable` Result, `verdict: pass`, and `disposition: actionable`. The second evaluation passes with `basis.not_actionable: 0`. The scripted subject received one call in total.
- **Fails if:** The new Result does not resolve the old one; the second evaluation fails; the subject is called again; or the implementation offers no way to request a re-assessment (13.1).

CT-DEC-015 — A missed Seeded Control invalidates its defect class without changing any Result

- **Capability:** `seeded_controls`
- **Requirement:** Volume I, Chapter 6, Quality Gates: “A Quality Gate MUST NOT pass on any aggregate to which an Oracle contributed Results sharing a missed Seeded Control’s defect class (Volume VIII) in that run”; Appendix B §B.3, Invalidation is derived: “A missed Seeded Control MUST NOT change any recorded Result”; Appendix C §C.3, Seeded Control Reporting.
- **Method:** Automated
- **Fixture:** F-SEED with `semantic-gate`; scripted Judge `J-PASS-90`.
- **Action:** Execute `cts-suite`; retrieve the Execution records and every Result; evaluate the Gates.
- **Expected:**
  - `seeded_control_results` has one entry for `s-control`: `defect_class: greeting-missing`, `outcome: missed`, `confirmed_by: construction`, `invalidation_scope: defect_class`, `reason: null`.
  - The scripted subject received exactly one call (for `s-greeting`; `s-control` uses its fixed Evidence).
  - `s-greeting`’s Expectation either has a recorded Result with `verdict: pass` and `disposition: actionable` whose `invalidated_by` lists `{ scenario: s-control, defect_class: greeting-missing }`, or, if the Judge was no longer invoked for it, a Result with `disposition: not_assessed` and `verdict: null`.
  - `semantic-gate` fails, with `seeded_control_miss` in `reasons`. Its `basis` counts no Result of `s-control`.
  - `functional-gate` passes: the schema Constraint’s Oracle missed no control.
- **Fails if:** `semantic-gate` passes; `reasons` lacks `seeded_control_miss`; the invalidated Result’s recorded `verdict` or `disposition` was changed; an `s-control` Result is counted in any `basis` field; or `functional-gate` fails.

CT-DEC-016 — A caught Seeded Control is excluded from Aggregation and invalidates nothing

- **Capability:** `seeded_controls`
- **Requirement:** Volume VIII, Seeded Controls: “Seeded Control Results themselves MUST be excluded from Aggregation.”

- **Method:** Automated
- **Fixture:** F-SEED with `semantic-gate`; scripted Judge `J-CATCH`.
- **Action:** Execute `cts-suite`; evaluate the Gates.
- **Expected:** `s-control`'s entry has `outcome: caught` and `invalidation_scope: none`. `semantic-gate` passes with `basis.results: 1` and `basis.scenarios: 1`.
- **Fails if:** The failing `s-control` Result is counted, so that `semantic-gate` fails or `basis` counts two Results or two Scenarios.

CT-DEC-017 — Full-run invalidation reaches every Gate of the run

- **Capability:** `seeded_controls`
- **Requirement:** Volume I, Chapter 6, Quality Gates: full-run invalidation "widens the rule, it never lifts it"; Appendix C §C.3: "Under `full_run`, no Result of that run counts as `actionable` for any Quality Gate, whatever its defect class"; Appendix A §A.12, GateResult `value`: "Never bare `null`."
- **Method:** Automated
- **Fixture:** F-SEED with `semantic-gate`; add `execution: { on_seeded_control_miss: halt_run }` to the Project; scripted Judge `J-PASS-90`.
- **Action:** Execute `cts-suite`; evaluate the Gates.
- **Expected:** `s-control`'s entry has `outcome: missed`, `invalidation_scope: full_run`, and a non-empty `reason`. Both `functional-gate` and `semantic-gate` fail, each with `seeded_control_miss` in `reasons`, and each `value` is a number or `"not_computed"`.
- **Fails if:** Either Gate passes; either lacks `seeded_control_miss`; `reason` is empty or `null`; or a `value` is bare `null`.

CT-DEC-018 — An Oracle that has not been requalified cannot resolve an invalidated Result

- **Capability:** `seeded_controls`
- **Requirement:** Volume I, Chapter 6, Result Resolution: a requalified Oracle may resolve "only once the Oracle has been requalified for every defect class whose miss reached that Result"; Volume VIII, Requalification.
- **Method:** Automated
- **Fixture:** F-SEED-MISSED (2.7). The Judge configuration is unchanged and has caught no fresh control of `greeting-missing`.
- **Action:** Request that the same Judge configuration assess `s-greeting`'s stored Evidence again. Evaluate the Gates.
- **Expected:** The request is rejected, or produces a Result that does not resolve the invalidated or `not_assessed` one. `semantic-gate` still fails with `seeded_control_miss`.
- **Fails if:** A Result from that Judge configuration resolves the earlier Result; `semantic-gate` passes; or the implementation offers no way to request a re-assessment (13.1).

CT-DEC-019 — A Validator-enforced Safety failure vetoes its aggregate

- **Capability:** core
- **Requirement:** Volume I, Chapter 6, Aggregation Model: "a Validator-enforced Safety failure acts as a hard veto on the aggregate regardless of how well everything else scored"; Appendix A §A.5, Constraint `veto`.

- **Method:** Automated
- **Fixture:** F-MULTI with `semantic-gate`; scripted subject `SR-ECHO`. Add to the Project:

```
contracts:
  - constraints:
      - validator: forbidden-content
        target: response
        expected:
          patterns:
            - regex: "Three"
        veto: true
```

  and add to its `quality_gates`:

```
- name: safety-gate
  aggregate: safety
  threshold: 0.5
  on_failure: block_release
```

- **Action:** Execute `cts-suite`; evaluate the Gates.
- **Expected:** The forbidden-content Constraint passes for `s-1` and `s-2` and fails for `s-3`. `safety-gate` fails, although two of three Results passed and its threshold is 0.5.
- **Fails if:** `safety-gate` passes.

CT-DEC-020 — A weighted failure counts for more than an unweighted one

- **Capability:** core
- **Requirement:** Volume I, Chapter 6, Aggregation Model: “A failure on a Scenario marked high-stakes MUST count for more than an equally-sized failure on a low-stakes one”; Appendix A §A.5, Policies `weight`.
- **Method:** Automated
- **Fixture:** F-MULTI with `semantic-gate`; scripted subject `SR-ECHO`. In `s-1`’s `contract`, add `policies: { weight: 3.0 }`; in `s-2`’s and `s-3`’s, add `policies: { weight: 1.0 }`.
- **Action:** Run A: execute `cts-suite` with scripted Judge `J-MAP(J-FAIL-90, J-PASS-90, J-PASS-90)`; evaluate the Gates. Run B: execute it with `J-MAP(J-PASS-90, J-FAIL-90, J-PASS-90)`; evaluate the Gates.
- **Expected:** `semantic-gate` fails in both runs, and its `value` in Run A is lower than in Run B.
- **Fails if:** The two values are equal, or Run A’s is higher.

CT-DEC-021 — A statistical interval is not labelled Confidence

- **Capability:** core
- **Requirement:** Volume I, Chapter 6, Aggregate Basis: “An implementation MAY report such an interval alongside an aggregate; it MUST NOT label it Confidence.”
- **Method:** Inspection
- **Fixture:** Two variants: CT-DEC-002’s configuration and CT-DEC-008’s.
- **Action:** In each, execute `cts-suite` and evaluate the Gates. Review how every aggregate is presented in the GateResults and in any Report or Dashboard the implementation produces for those runs, and any interval,

margin of error, or similar figure reported with it.
- **Expected:** Where an interval appears, it is not called Confidence. The word Confidence is used only for an Oracle's certainty in one verdict.
- **Fails if:** Any interval or aggregate-level figure is labelled Confidence.

CT-DEC-022 — Results from a changed Environment are stale

- **Capability:** core
- **Requirement:** Volume I, Chapter 6, Quality Gates: "A Gate MUST NOT pass on an aggregate that includes a stale Result, and records the reason as `stale_results`"; Appendix B §B.3, Stale Results.
- **Method:** Automated, Harness
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-PASS-90`.
- **Action:** Execute `cts-suite` (batch 1); evaluate the Gates. As a governed change (Section 14.1), set `cts-env`'s `prompt_version` to `cts-prompt@2`. Evaluate the Gates again for batch 1. Then execute `cts-suite` again (batch 2) and evaluate the Gates for it.
- **Expected:** The first evaluation passes. The second captures a new `snapshot_id`, and both `functional-gate` and `semantic-gate` fail with `stale_results` in `reasons`; the stale Results are counted in `basis.not_actionable`, not in `basis.results`. The evaluation for batch 2 passes.
- **Fails if:** Either Gate passes in the second evaluation; `reasons` lacks `stale_results`; or the evaluation for batch 2 fails.

CT-DEC-023 — Only Results for a changed clause are stale

- **Capability:** core
- **Requirement:** Volume I, Chapter 6, Quality Gates: a Result is stale when "its clause differs from that clause in the Snapshot. A clause is matched by its `id` … A clause without an `id` cannot be matched across versions, so any change in its Contract makes its Results stale"; Appendix B §B.3, Stale Results.
- **Method:** Automated
- **Fixture:** Two variants of F-BASE with `semantic-gate`; scripted Judge `J-PASS-90`. (a) The schema Constraint has `id: schema-1` and the Expectation `id: greets`. (b) No `id`s.
- **Action:** For each: execute `cts-suite`; evaluate the Gates. As a governed change, set the Expectation's `confidence_threshold` in `s-greeting`'s inline Contract to `0.85`. Evaluate the Gates again for the same batch.
- **Expected:** (a) In the second evaluation, `semantic-gate` fails with `stale_results` and `functional-gate` passes, its clause unchanged. (b) Both fail with `stale_results`.
- **Fails if:** (a) `semantic-gate` passes, or `functional-gate` fails; (b) either Gate passes, or either lacks `stale_results`.

CT-DEC-024 — A Gate with nothing to aggregate fails with insufficient basis

- **Capability:** core
- **Requirement:** Volume I, Chapter 6, Quality Gates: "Where the Results a Gate reads include no `actionable` Result, no aggregate can be computed. The Gate MUST NOT pass, its value is recorded as not computed, and insufficient basis is among its recorded reasons, alongside every other reason that applies"; Appendix A §A.12, `reasons` (`insufficient_basis`); Appendix B §B.3, Aggregate Basis reporting.
- **Method:** Automated

- **Fixture:** F-BASE with `semantic-gate`, declaring no `min_basis`; scripted Judge `J-PASS-60`.
- **Action:** Execute `cts-suite`; evaluate the Gates.
- **Expected:** `semantic-gate`: `status: fail`, `value: "not_computed"`, and `reasons` containing both `insufficient_basis` and `non_actionable_results`.
- **Fails if:** The Gate passes; `value` is a number or bare `null`; or `reasons` lacks either.

14. Release and Governance

Tests in this section record Release Decisions, overrides, Baselines, and configuration changes, and observe what is accepted, what is rejected, and what is recorded.

14.1 Fixtures and conventions for this section

Section 13.1 applies here as well: “evaluate the Gates”, `semantic-gate`, `P-REVIEWER`, and the clause-to-Gate mapping.

Principals. The tester sets up the following in the implementation, with Team membership giving each access to `cts-base`. Role names are illustrative; what matters is the Permissions each Role holds.

Principal	Kind	Permissions
`P-RELEASE`	Person	`read`, `execute`, `release_decide`
`P-APPROVER`	Person	`read`, `execute`, `contract_define`, `gate_override`, `baseline_approve`, `environment_edit`, `scenario_edit`, `environment_delete`; for loading fixtures, `project_create`, `project_edit`, `dataset_create` (Appendix C §C.3)
`P-NOPERM`	Person	`read`, `execute`
`P-EDITOR`	Person	`read`, `project_edit`, `scenario_edit`
`P-REVIEWER`	Person	`read`, `review`; holds the Role `reviewer`
`P-ADMIN`	Person	`read`, `project_delete`, `audit_read`
`SA-CI`	Service Account, owned by `P-RELEASE`	`read`, `execute`, `release_decide`, `gate_override`, `baseline_approve`, `contract_define`

`P-APPROVER` loads every fixture, unless a test says otherwise. Where an implementation needs a further Permission to load or replace a configuration, the tester grants it to `P-APPROVER` and states it in the test report.

`SA-CI` deliberately holds Permissions it may not use for every action: what a Service Account may do is limited by what it is, not only by what it holds (Volume XII, Principals).

`RP-1`. A Release Policy (Appendix A §A.14) approved by `P-RELEASE` through `POST /projects/cts-base/release-policies`:

```
id: RP-1
project: cts-base
suites: [cts-suite]
environments: [cts-env]
service_accounts: [SA-CI]
```

Recording a Release Decision. “Record a Release Decision” means `POST /projects/cts-base/releases`, or the implementation’s equivalent, with this body unless the test says otherwise:

```
project: cts-base
environment: cts-env
suite: cts-suite
execution_batch_id: "<the batch the Action produced>"
gates: "<every GateResult the latest evaluation returned, keyed by Gate name>"
decision: approved
approved_by: "<the acting Principal>"
approved_at: "<now>"
approval_role: "<a Role of the acting Principal that holds release_decide>"
report_id: "<the Report the implementation produced for the batch>"
snapshot_id: "<the snapshot_id the latest evaluation returned>"
```

Where the implementation sets a field itself (`gates`, `approved_by`, `approved_at`, or `approval_role`, from the acting Principal and the evaluation named), the tester omits it, and the test checks the stored value instead of the one sent.

Overriding. “Override `<gate>`” means `POST /projects/cts-base/gates/<gate>/override` naming that Gate’s GateResult from the latest evaluation by its `id`, with the batch’s `execution_batch_id` and the justification `"CTS override"`, made by `P-APPROVER` unless the test says otherwise.

A governed change. A change to a Gate, an inline Contract, or an Environment made by `P-APPROVER` through the operation Appendix C §C.3 lists for it, approved where Approval Workflow governs it.

14.2 Tests

CT-REL-001 — A Release Decision is recorded when every blocking Gate passes

- **Capability:** core
- **Requirement:** Volume I, Chapter 6, Quality Gates: “A blocking Gate MUST pass for a Release Decision to authorize shipping”; Appendix A §A.12, Release Decision; Appendix B §B.3, Approval Permissions: “a Release Decision’s [`approval_role`] MUST hold `release_decide`.”
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-PASS-90`.
- **Action:** Execute `cts-suite`; evaluate the Gates. As `P-RELEASE`, record a Release Decision. Retrieve it.
- **Expected:** Accepted. The stored decision has an `id` and every field of the body above, with `decision: approved`, `approved_by` naming `P-RELEASE`, and a `gates` map holding the GateResults for `functional-gate` and `semantic-gate`, unchanged.
- **Fails if:** Rejected, or any field is missing or altered.

CT-REL-002 — A failed blocking Gate prevents an approval

- **Capability:** core
- **Requirement:** Appendix A §A.12: “`decision: approved` requires every blocking GateResult to have `status: pass`”; Volume I, Chapter 6, Quality Gates: “the Decision Pipeline does not silently continue.”

- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-FAIL-90`.
- **Action:** Execute `cts-suite`; evaluate the Gates. As `P-RELEASE`, record a Release Decision (a) with `decision: approved`, then (b) with `decision: rejected`.
- **Expected:** (a) rejected; (b) accepted.
- **Fails if:** (a) is accepted, or (b) is rejected.

CT-REL-003 — A failed advisory Gate is recorded and does not prevent an approval

- **Capability:** core
- **Requirement:** Volume I, Chapter 6, Quality Gates: an advisory Gate's "result is recorded in every Report and Release Decision whose scope it covers, but its failure does not by itself prevent shipping"; Appendix A §A.10, `on_failure: warn`.
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`, given `on_failure: warn`; scripted Judge `J-FAIL-90`.
- **Action:** Execute `cts-suite`; evaluate the Gates. As `P-RELEASE`, record a Release Decision. Retrieve it and the Report it names.
- **Expected:** Accepted with `decision: approved`. Its `semantic-gate` GateResult has `status: fail`, `blocking: false`, `reasons: [threshold]`, and no `override`. The Report shows `semantic-gate` as failed.
- **Fails if:** The decision is rejected; the advisory GateResult is missing, shown as passed, or carries `blocking: true`; or the Report omits it.

CT-REL-004 — A Release Decision must carry every applicable Gate

- **Capability:** core
- **Requirement:** Appendix B §B.3, Release Decision coverage: "`gates` MUST contain a GateResult for every Quality Gate, blocking or advisory, applicable to the decision's scope"; Appendix A §A.10: "A Release Decision is subject to the Project's Gates and to those of the Suite it covers."
- **Method:** Automated
- **Fixture:** F-BASE; add to `suites[0]`:

  ```
  quality_gates:
    - name: suite-gate
      aggregate: semantic
      threshold: 0.5
      on_failure: warn
  ```

  Scripted Judge `J-PASS-90`.
- **Action:** Execute `cts-suite`; evaluate the Gates. As `P-RELEASE`, record a Release Decision whose `gates` holds only the `functional-gate` GateResult.
- **Expected:** The evaluation returns GateResults for `functional-gate` and `suite-gate`. The Release Decision is rejected; the error names `suite-gate`.
- **Fails if:** The evaluation omits `suite-gate`, or the Release Decision is accepted.

CT-REL-005 — No Release Decision without an applicable Gate

- **Capability:** core
- **Requirement:** Appendix B §B.3, Release Decision requires a Gate: “A Release Decision MUST NOT be recorded for a scope to which no Quality Gate applies”; Appendix C §C.3, `POST /projects/{project}/releases`.
- **Method:** Automated
- **Fixture:** F-BASE; remove `quality_gates`.
- **Action:** Execute `cts-suite`; evaluate the Gates. As `P-RELEASE`, record a Release Decision with `gates: {}` and whatever `snapshot_id` the evaluation returned, if any.
- **Expected:** The evaluation returns no GateResult. The Release Decision is rejected.
- **Fails if:** The Release Decision is accepted.

CT-REL-006 — An approved override lets a failed blocking Gate result in a release

- **Capability:** core
- **Requirement:** Volume I, Chapter 6, Quality Gates: an exception “MUST be recorded as an explicit, auditable override rather than a quiet pass”; Appendix A §A.12: “`decision: overridden` requires every blocking GateResult with `status: fail` to carry an `override`”; Appendix B §B.3, Gate Result override.
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-FAIL-90`.
- **Action:** Execute `cts-suite`; evaluate the Gates. As `P-RELEASE`, record a Release Decision with `decision: overridden`. Override `semantic-gate`. Without evaluating the Gates again, retrieve the `semantic-gate` GateResult and record the Release Decision with `decision: overridden` once more.
- **Expected:** The first attempt is rejected (no override yet). The override is approved and recorded against that GateResult, which now carries `override` naming the Approval Workflow record; the GateResult keeps `status: fail`. The second attempt is accepted. The override record names `P-APPROVER` and the justification.
- **Fails if:** The first attempt is accepted; the GateResult’s `status` becomes `pass`; the override record omits the approving Person or the justification; or the second attempt is rejected.

CT-REL-007 — An override applies to one Gate Result only

- **Capability:** core
- **Requirement:** Volume XII, Approval Workflow: an override “MUST NOT be applied to any other evaluation, including a later evaluation of the same Gate for a different batch”; Appendix B §B.3: “One override record MUST NOT be referenced by more than one GateResult.”
- **Method:** Automated
- **Fixture:** F-OVERRIDDEN (2.7).
- **Action:** Execute `cts-suite` again (a new batch) with `J-FAIL-90`; evaluate the Gates. As `P-RELEASE`, record a Release Decision for the new batch with `decision: overridden`, its `semantic-gate` GateResult carrying the `override` reference recorded when F-OVERRIDDEN was built.
- **Expected:** The new `semantic-gate` GateResult carries no `override`. The Release Decision is rejected.
- **Fails if:** The new GateResult shows the earlier override, or the Release Decision is accepted.

CT-REL-008 — An override needs a Permission, a justification, and a failed blocking Gate

- **Capability:** core
- **Requirement:** Appendix C §C.3, override: “Body MUST name the GateResult by its `id` …, give its `execution_batch_id`, and include a non-empty justification”; Appendix B §B.3, Gate Result override; Appendix A §A.12, `override`: “Only on a blocking GateResult with `status: fail`”; Appendix C §C.4: “If the Principal lacks the required Permission, the API MUST reject the request.”
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-FAIL-90`.
- **Action:** Execute `cts-suite`; evaluate the Gates. Attempt to override: (a) `semantic-gate` as `P-NOPERM`; (b) `semantic-gate` as `P-APPROVER` with an empty justification; © `semantic-gate` as `P-APPROVER` with no GateResult `id`; (d) `functional-gate`, which passed, as `P-APPROVER`; (e) `semantic-gate` as `P-APPROVER`, naming the `id` of the `functional-gate` GateResult.
- **Expected:** All five are rejected, and no GateResult carries an `override`.
- **Fails if:** Any of the five is accepted.

CT-REL-009 — A Gate-specific override Permission is enforced

- **Capability:** core
- **Requirement:** Appendix A §A.10, `override_requires`: a Project “MAY require a stricter, Gate-specific Permission in addition to `gate_override`”; Appendix C §C.4.
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`, given `override_requires: safety_override`; scripted Judge `J-FAIL-90`. `P-APPROVER` does not hold `safety_override`.
- **Action:** Execute `cts-suite`; evaluate the Gates. Override `semantic-gate`.
- **Expected:** Rejected.
- **Fails if:** Accepted.

CT-REL-010 — A Gate changed after the Snapshot blocks the Release Decision until re-evaluated

- **Capability:** core
- **Requirement:** Volume I, Chapter 6, Quality Gates: “If any of that configuration changed after the Snapshot was captured, the Release Decision MUST NOT be recorded until its Gates are evaluated again … Where only a Gate definition changed, no new Execution is needed”; Appendix B §B.3, Release Decision currency.
- **Method:** Automated, Harness
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-PASS-90`.
- **Action:** Execute `cts-suite`; evaluate the Gates. As a governed change, set `semantic-gate`’s `threshold` to `0.9`. As `P-RELEASE`, record a Release Decision with the first evaluation’s `snapshot_id`. Then evaluate the Gates again for the same batch and record the Release Decision with the new `snapshot_id`.
- **Expected:** The first attempt is rejected (1.5). The second evaluation returns a new `snapshot_id` and a `semantic-gate` GateResult with `threshold: 0.9`; the second attempt is accepted. The scripted subject and Judge received no further calls after the first Execution.
- **Fails if:** The first attempt is accepted; the second is rejected; or the re-evaluation started new Executions.

CT-REL-011 — An inline Contract changed after the Snapshot blocks the Release Decision

- **Capability:** core
- **Requirement:** Appendix B §B.3, Release Decision currency: “any Contract (Reusable or inline)”; Appendix A §A.13, `inline_contracts`: “any difference from the current content is a change.”
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-PASS-90`.
- **Action:** Execute `cts-suite`; evaluate the Gates. As a governed change, set the Expectation’s `confidence_threshold` in `s-greeting`’s inline Contract to `0.85`. As `P-RELEASE`, record a Release Decision with the evaluation’s `snapshot_id`.
- **Expected:** Rejected (1.5).
- **Fails if:** Accepted.

CT-REL-012 — An Environment changed after the Snapshot blocks the Release Decision

- **Capability:** core
- **Requirement:** Appendix B §B.3, Release Decision currency: “any Contract (Reusable or inline), Quality Gate, or Environment in its scope”; Appendix B §B.3, Stale Results.
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-PASS-90`.
- **Action:** Execute `cts-suite`; evaluate the Gates. As a governed change, set `cts-env`’s `prompt_version` to `cts-prompt@2`. As `P-RELEASE`, record a Release Decision with the evaluation’s `snapshot_id`. Then evaluate the Gates again for the same batch, without new Executions, and record the Release Decision with the new `snapshot_id`.
- **Expected:** Both attempts are rejected: the first because `cts-env` changed since its Snapshot; the second because the new evaluation’s blocking GateResults fail with `stale_results` (CT-DEC-022).
- **Fails if:** Either attempt is accepted.

CT-REL-013 — A Snapshot captures the configuration in scope and is never modified

- **Capability:** core
- **Requirement:** Appendix A §A.13, Snapshot: “Never modified once captured”; Appendix B §B.3, Snapshot immutability: “A Snapshot MUST NOT be modified after `captured_at`.”
- **Method:** Automated
- **Fixture:** F-CONTRACT-CHANGED (2.7).
- **Action:** Retrieve the Snapshot named by the evaluation’s `snapshot_id`.
- **Expected:** It has `id`, `captured_at`, `project: cts-base`, `suites: [cts-suite]`, `environments` holding `cts-env` as defined in F-BASE, `contracts: {}`, `inline_contracts` holding `s-greeting`’s Contract in full under `scenario:cts-suite/s-greeting` with `confidence_threshold: 0.8`, and `quality_gates` holding both Gate definitions. It is unchanged by the later governed change.
- **Fails if:** Any of these is missing or differs, or the Snapshot reflects the change.

CT-REL-014 — A GateResult keeps the threshold it was evaluated against

- **Capability:** core

- **Requirement:** Appendix A §A.12, GateResult `threshold`: “copied at evaluation time, so the record stays checkable if the Gate definition changes later.”
- **Method:** Automated
- **Fixture:** F-GATE-CHANGED (2.7).
- **Action:** Retrieve the `semantic-gate` GateResult from evaluation 1.
- **Expected:** `threshold: 1.0`.
- **Fails if:** It shows `0.9`, or the earlier GateResult can no longer be retrieved.

CT-REL-015 — Defining or changing a Gate is governed under `contract_define`

- **Capability:** core
- **Requirement:** Appendix A §A.10: “Defining or changing a Quality Gate is governed by Approval Workflow under `contract_define`”; Volume I, Chapter 6: changing a Gate from blocking to advisory “is governed like any other change to a Gate”; Appendix C §C.5.
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`.
- **Action:** As `P-NOPERM`, attempt (a) to define a new Gate and (b) to set `semantic-gate`’s `on_failure` to `warn`. Then make change (b) as `P-APPROVER`. Query the audit log for action type `contract_define`.
- **Expected:** (a) and (b) as `P-NOPERM` are rejected. The change as `P-APPROVER` goes through Approval Workflow and appears in the audit log as `contract_define`, naming `P-APPROVER`.
- **Fails if:** Either attempt by `P-NOPERM` takes effect, or the change by `P-APPROVER` is not logged as `contract_define`.

CT-REL-016 — Recording a Release Decision needs `release_decide`

- **Capability:** core
- **Requirement:** Appendix C §C.4: “Every mutating operation MUST check the resolved Principal’s Permissions before executing”; Appendix B §B.3, Approval Permissions.
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-PASS-90`.
- **Action:** Execute `cts-suite`; evaluate the Gates. Record a Release Decision as `P-NOPERM`, giving as `approval_role` a Role it holds.
- **Expected:** Rejected.
- **Fails if:** Accepted.

CT-REL-017 — A Service Account cannot approve an override or a Baseline

- **Capability:** core
- **Requirement:** Volume XII, Principals: “`gate_override` and `baseline_approve` MUST be approved by a Person”; Appendix A §A.11, `approved_by`: “MUST be a Person, not a Service Account.”
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-FAIL-90`.
- **Action:** Execute `cts-suite`; evaluate the Gates. As `SA-CI`, attempt (a) to override `semantic-gate` and (b) to designate the `s-greeting` Execution as a Baseline.
- **Expected:** Both rejected, although `SA-CI` holds `gate_override` and `baseline_approve`. No override and no Baseline is recorded.

- **Fails if:** Either is accepted.

CT-REL-018 — A Service Account records a Release Decision only under a standing policy

- **Capability:** core
- **Requirement:** Appendix B §B.3, Automatic release: “A Release Decision whose `approved_by` is a Service Account MUST carry `release_policy`, and its `decision` MUST be `approved` (every blocking GateResult `pass`, no `override`) or `rejected`”; Volume XII, Principals: “The record MUST reference the policy.”
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`, plus a second Environment `cts-env-2` identical to `cts-env` except for its name; `RP-1` in place.
- **Action:** Batch 1, with `J-PASS-90`: execute, evaluate the Gates, and as `SA-CI` record a Release Decision (a) without `release_policy`, then (b) with `release_policy: RP-1`. Batch 2, with `J-FAIL-90`: execute, evaluate, override `semantic-gate` as `P-APPROVER`, and as `SA-CI` with `release_policy: RP-1` record © `decision: overridden` and (d) `decision: rejected`. Batch 3, against `cts-env-2` with `J-PASS-90`: execute, evaluate, and as `SA-CI` record (e) `approved` with `release_policy: RP-1`. Then, as `P-RELEASE`, revoke `RP-1`; batch 4, against `cts-env` with `J-PASS-90`: execute, evaluate, and as `SA-CI` record (f) `approved` with `release_policy: RP-1`.
- **Expected:** (a) rejected; (b) accepted, and the stored decision references `RP-1`; © rejected; (d) accepted; (e) rejected, since `RP-1` does not cover `cts-env-2`; (f) rejected, since `RP-1` is revoked. The decisions (b) and (d) are still recorded after the revocation. `RP-1`, retrieved after it, has `revoked_at` and `revoked_by` naming `P-RELEASE`, and its other fields unchanged.
- **Fails if:** (a), ©, (e), or (f) is accepted; (b) or (d) is rejected; (b)'s record does not reference `RP-1`; the revocation removed (b) or (d); or the revoked `RP-1` lacks `revoked_at` or `revoked_by`.

CT-REL-019 — Audit records which kind of Principal acted

- **Capability:** core
- **Requirement:** Volume XII, Principals: a Service Account “MUST have an owning Person accountable for what it does. Audit records which kind of Principal acted.”
- **Method:** Automated, Inspection
- **Fixture:** F-SA-DECISIONS (2.7).
- **Action:** Query the audit log for action type `release_decide`. Retrieve `SA-CI`’s details. If the implementation lets a Service Account be created, attempt to create one with no owning Person.
- **Expected:** The entries for (b) and (d) name `SA-CI` as a Service Account; any entry for a Person’s action names a Person. `SA-CI`’s details name `P-RELEASE` as owner; where the implementation offers no operation to retrieve a Principal, a reviewer establishes the owner by Inspection (1.8). The Service Account with no owner is rejected.
- **Fails if:** An entry does not state the kind of Principal; no owner is recorded for `SA-CI`; or the owner-less Service Account is created.

CT-REL-020 — A Service Account applies `contract_define` only with a named approver and review

- **Capability:** core

- **Requirement:** Appendix B §B.3, Human approval: “A `contract_define` applied by a Service Account MUST name the approving Person and reference the external review”; Volume XII, Principals.
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`.
- **Action:** As `SA-CI`, set `semantic-gate`’s `threshold` to `0.9` (a) with no approver or review reference; (b) naming `P-APPROVER` as approver and a review reference `review-cts-1`. Retrieve the Approval Workflow record for (b).
- **Expected:** (a) rejected; (b) accepted, and its record names `P-APPROVER` and `review-cts-1`.
- **Fails if:** (a) takes effect, or (b)'s record lacks either reference.

CT-REL-021 — A Baseline is designated explicitly, by a Person holding `baseline_approve`

- **Capability:** core
- **Requirement:** Volume X, Baselines: “an Execution’s Result MUST be explicitly designated as the reference standard, not inferred from simply being the first or the most recent Execution of a Scenario”; Appendix A §A.11; Appendix B §B.3, Approval Permissions.
- **Method:** Automated
- **Fixture:** F-BASE; scripted Judge `J-PASS-90`.
- **Action:** Execute `cts-suite` twice. List Baselines. Then designate the first Execution of `s-greeting` as a Baseline (a) as `P-NOPERM`, (b) as `P-APPROVER`. List Baselines again.
- **Expected:** The first listing is empty. (a) is rejected. (b) is accepted, and the Baseline has `scenario: s-greeting`, `environment: cts-env`, `execution_id` of the first Execution, `approved_by` naming `P-APPROVER`, `approved_at`, and an `approval_role` holding `baseline_approve`.
- **Fails if:** A Baseline exists before (b); (a) is accepted; or any field of (b) is missing or wrong.

CT-REL-022 — Deleting a referenced Environment does not break history

- **Capability:** core
- **Requirement:** Appendix C §C.8: “`DELETE` MUST NOT silently invalidate a Baseline, a Snapshot, a Report, or an audit record that references the deleted entity”; Volume XII, Traceability.
- **Method:** Automated
- **Fixture:** F-BASELINED (2.7).
- **Action:** As `P-APPROVER`, `DELETE` `cts-env`. Retrieve the Baseline and the Snapshot.
- **Expected:** Either the deletion is rejected, or both the Baseline and the Snapshot remain retrievable with `cts-env` preserved as a frozen record.
- **Fails if:** The deletion succeeds and either record is lost, or its reference to `cts-env` no longer resolves.

CT-REL-023 — A Release Decision can be traced back to its evidence

- **Capability:** core
- **Requirement:** Volume XII, Traceability: reconstruct “which Report a Release Decision relied on, which Snapshot pinned the configuration it was evaluated under, which Quality Gates it read … who approved”; “A Release Decision that cannot be traced this far back is not meaningfully audited.”
- **Method:** Inspection
- **Fixture:** F-OVERRIDDEN (2.7); its Release Decision.

- **Action:** From the implementation's records alone, follow the chain from the Release Decision to its Report, Snapshot, GateResults, the override and its approver, the Results each Gate read, their Executions, and each Execution's Conversation.
- **Expected:** The reviewer reaches every link, citing the record used at each step.
- **Fails if:** Any link cannot be followed.

CT-REL-024 — An overridden Gate is reflected as overridden in the pipeline

- **Capability:** `ci_cd:<platform>`
- **Requirement:** Volume XIII, Quality Gates: "the Pipeline MUST reflect the override's outcome, not the underlying Gate's raw failure."
- **Method:** Automated
- **Fixture:** CT-REL-006's configuration, run from a pipeline on the claimed platform.
- **Action:** Run the pipeline to the point where it checks the Gates; override `semantic-gate`; let the pipeline continue.
- **Expected:** Before the override, the pipeline's Gate check fails. After it, the pipeline shows the overridden outcome, distinguishable from a plain pass.
- **Fails if:** The pipeline stays failed after the override, or shows it as a plain pass.

CT-REL-025 — An unanswered review fails the pipeline instead of hanging it

- **Capability:** `ci_cd:<platform>` together with `human_review`
- **Requirement:** Volume XIII, Quality Gates: "a Pipeline that times out waiting on an unset `review_timeout` MUST fail the same way it would for any other blocking Gate, not hang."
- **Method:** Automated
- **Fixture:** As CT-DEC-011's fixture, with no `review_timeout`, run from a pipeline on the claimed platform with a pipeline timeout the tester sets.
- **Action:** Run the pipeline; do not answer the review.
- **Expected:** When the pipeline's timeout is reached, the pipeline fails as for a failed blocking Gate.
- **Fails if:** The pipeline passes, or is still waiting when its timeout plus the grace period of the waiting budget (1.6) has passed.

CT-REL-026 — The basic loop runs end to end through all five layers

- **Capability:** core
- **Requirement:** §10, The Non-Negotiable Core: "The five logical layers … MUST each be met somewhere. An implementation that cannot define a Scenario and Contract, execute it, capture evidence, assess it with at least one Oracle, and reach some Decision has not implemented … AQEF's basic loop at all."
- **Method:** Automated
- **Fixture:** F-BASE; scripted Judge `J-PASS-90`.
- **Action:** In one sequence, with no step outside the implementation: load F-BASE; execute `cts-suite`; retrieve the Conversation; retrieve the Results; evaluate the Gates; as `P-RELEASE`, record a Release Decision.
- **Expected:** Each step succeeds: Definition (CT-CFG-001), Execution (CT-EXE-001), Evidence (CT-EXE-008), Assessment (CT-ASM-003, CT-ASM-004), Decision (CT-DEC-001, CT-REL-001), each as its own test describes.
- **Fails if:** Any layer cannot be reached within the implementation.

CT-REL-027 — A Conformance Statement is published

- **Capability:** core
- **Requirement:** §10, Conformance Statement: “An implementation that claims AQEF Conformance MUST publish a **Conformance Statement** stating: the version of this specification it conforms to; the capabilities it claims … using the identifiers below.”
- **Method:** Inspection
- **Fixture:** None.
- **Action:** Obtain the implementation’s published Conformance Statement.
- **Expected:** It states a specification version, lists its claimed capabilities using §10’s identifiers, and states how it binds Projects and Suites to Teams, or that it binds none (Volume XII, Teams). It is the statement this test run uses to decide which tests are `NOT_APPLICABLE`.
- **Fails if:** No statement is published; it states no version; a claimed capability is not given by a §10 identifier; or it says nothing about Team binding.

CT-REL-028 — Every operation is authenticated and resolves to a Principal

- **Capability:** core
- **Requirement:** Appendix C §C.4: “All API operations MUST be authenticated” and the mechanism “MUST resolve to a Principal — a Person or a Service Account with an owning Person.”
- **Method:** Automated
- **Fixture:** F-BASE.
- **Action:** With no credentials, request `GET /projects/cts-base` and `POST /projects/cts-base/executions`. Then, as `P-NOPERM`, request `POST /projects/cts-base/executions` and query the audit log for that action.
- **Expected:** Both unauthenticated requests are rejected and start no Execution. The authenticated request is accepted, and the record of it names `P-NOPERM` as a Person.
- **Fails if:** Either unauthenticated request succeeds, or the authenticated one cannot be attributed to `P-NOPERM`.

CT-REL-029 — A Report shows each Gate’s status, basis, and why it did not pass

- **Capability:** core
- **Requirement:** Volume XI, Quality Gates: every applicable Gate “MUST show its current status — pass, fail, or a Governance-logged override … and, where it failed, which aggregate and threshold it failed against”; “A Gate’s displayed status MUST include the Aggregate Basis”; Appendix C §C.10, `gates`.
- **Method:** Automated
- **Fixture:** Two variants. (a) F-OVERRIDDEN (2.7). (b) CT-DEC-004’s configuration.
- **Action:** In (b), execute `cts-suite` and evaluate the Gates. In each, retrieve the Report the implementation produced for the batch after its Gates were evaluated, in (a) including any Report produced after the override.
- **Expected:** Each Report has the fields of Appendix C §C.10. In (a), `gates` holds `semantic-gate`’s GateResult with `status: fail`, its `threshold`, `basis`, and `override`. In (b), `semantic-gate`’s GateResult has `reasons: [insufficient_basis]` and its `basis`.
- **Fails if:** A §C.10 field is missing; `semantic-gate` is absent from `gates`; (a) lacks the `override`; or (b) reports `threshold` as a reason.

CT-REL-030 — Changing a Contract or Gate through any operation needs `contract_define`

- **Capability:** core
- **Requirement:** Appendix B §B.3, Governed configuration change: “Creating or changing a `contract`, `contracts`, or `quality_gates` value, through whatever operation, is `contract_define`”; Appendix C §C.5.
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`.
- **Action:** As `P-EDITOR`, who holds `scenario_edit` and `project_edit` but not `contract_define`: (a) replace `s-greeting` with `PUT .../scenarios/s-greeting`, changing only `variables.query` to `"Hi"`; (b) replace it again, changing its Expectation’s `confidence_threshold` to `0.5`; © replace the Project with `PUT /projects/cts-base`, changing `semantic-gate`’s `threshold` to `0.5`. Then query the audit log for action type `contract_define`.
- **Expected:** (a) accepted. (b) and © rejected, and the configuration keeps `confidence_threshold: 0.8` and `threshold: 1.0`. Made by `P-APPROVER` instead, (b) goes through Approval Workflow and is logged as `contract_define`.
- **Fails if:** (a) is rejected; (b) or © takes effect for `P-EDITOR`; or the change by `P-APPROVER` is not logged as `contract_define`.

CT-REL-031 — A Release Policy is approved by a Person and never modified

- **Capability:** core
- **Requirement:** Appendix C §C.3, Release Policy: approving and revoking a policy require that “The Principal MUST be a Person”; “No operation modifies a Release Policy”; Appendix A §A.14.
- **Method:** Automated
- **Fixture:** F-BASE; `RP-1` in place.
- **Action:** Retrieve `RP-1`. As `SA-CI`, which holds `release_decide`, attempt to approve a policy `RP-2` with the same content, and to revoke `RP-1`. As `P-RELEASE`, attempt to change `RP-1`’s `environments` through any operation the implementation offers.
- **Expected:** `RP-1` has `id`, `project`, `suites`, `environments`, `service_accounts`, `approved_by` naming `P-RELEASE`, an `approval_role` holding `release_decide`, and `approved_at`. Both attempts by `SA-CI` are rejected. No operation changes `RP-1`’s content; it is unchanged when retrieved again.
- **Fails if:** A field is missing; an attempt by `SA-CI` is accepted; or `RP-1`’s content changes.

CT-REL-032 — A Permission is checked, not a Role name

- **Capability:** core
- **Requirement:** Volume XII, Permissions: “A Permission MUST be checkable independently of any Role that happens to include it”; Appendix C §C.4: “Permission checks are against the atomic Permission, not the Role.”
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-PASS-90`. Two further Persons: `P-ROLE-A`, holding only a Role named `release_manager` that grants `read` and `execute`; and `P-ROLE-B`, holding only a Role named `observer` that grants `read`, `execute`, and `release_decide`.
- **Action:** Execute `cts-suite`; evaluate the Gates. Record a Release Decision with the body of 14.1, made (a) by `P-ROLE-A` with `approved_by: P-ROLE-A` and `approval_role: release_manager`, then (b) by `P-`

`ROLE-B` with `approved_by: P-ROLE-B` and `approval_role: observer`. Every other field is the same in both. Query the audit log, or the error returned, for the reason of (a).

- **Expected:** (a) is rejected because `P-ROLE-A` lacks the Permission `release_decide`: the error or the audit record says so, and does not report the body as malformed. (b) is accepted, and the stored decision names `observer` as `approval_role`.
- **Fails if:** (a) is accepted; (a) is rejected for a reason other than the missing Permission; or (b) is rejected. Either of the first two means that the Role's name, not its Permissions, decided.

CT-REL-033 — A Service Account is never presented as a Person

- **Capability:** core
- **Requirement:** Volume XII, Principals: "A Service Account is not a stand-in for a person and MUST NOT be set up to look like one."
- **Method:** Automated, Inspection
- **Fixture:** F-SA-DECISIONS (2.7).
- **Action:** Retrieve decision (b), its audit entry, and `SA-CI`'s Principal record. Review whether the implementation lets a Service Account be created or presented with a Person's attributes (a personal name, a personal e-mail login, or a flag marking it as a Person).
- **Expected:** Wherever the implementation records a Principal's kind, `SA-CI`'s is Service Account: in its audit entries, in its Principal record, and in any field of the decision that states a kind. A Release Decision has no such field of its own (Appendix A §A.12); decision (b) names `SA-CI` in `approved_by` and carries `release_policy`, which only a Service Account's decision does (Appendix B §B.3, Automatic release). The implementation offers no way to mark a Service Account as a Person.
- **Fails if:** `SA-CI` appears anywhere as, or indistinguishable from, a Person; or a Service Account can be set up as one.

CT-REL-034 — A Baseline's Dataset and Contract versions stay recoverable

- **Capability:** core
- **Requirement:** Appendix B §B.3, Baseline reproducibility: "A Baseline MUST reference a Dataset version and Contract version that are still recoverable"; Appendix C §C.8.
- **Method:** Automated
- **Fixture:** CT-EXE-010's fixture, with `s-greeting`'s Contract moved into a Reusable Contract `cts-contract` (version 1) that the Scenario references by name.
- **Action:** Execute `cts-suite`; as `P-APPROVER`, designate the Execution as a Baseline. As a governed change, publish version 2 of `cts-contract` with `confidence_threshold: 0.85`, and register `ds-1@1.1.0`. Attempt to delete `ds-1@1.0.0` and version 1 of `cts-contract`, by whatever operations the implementation offers. Retrieve the Baseline, the Dataset version, and the Contract version it references.
- **Expected:** The Baseline names `ds-1@1.0.0` and version 1 of `cts-contract`, and both can still be retrieved with their original content; any deletion was rejected or left them preserved as frozen records.
- **Fails if:** The Baseline does not identify both versions, or either can no longer be retrieved as it was.

CT-REL-035 — A Report surfaces what earlier Volumes flagged

- **Capability:** core

- **Requirement:** Volume XI, Reports: “a Report MUST surface the specific signals earlier Volumes have already flagged as worth including”; Appendix C §C.10, `signals`; `unadmitted_judges`: “Every Judge configuration whose Results are in scope and that is not admitted”; Volume VI, Admission: “Whether or not any Gate requires it, every Report shows the Judges in its scope that have not been admitted.”
- **Method:** Automated
- **Fixture:** F-MULTI with `semantic-gate`; add `execution: { scheduling: sequential, retry: { max_attempts: 2, on: infrastructure_failure } }`; scripted subject `SR-ECHO`, except that its first call for `s-1` fails as in `SR-503-THEN-OK`; scripted Judge `J-MAP(J-PASS-60, J-PASS-60, J-PASS-90)`.
- **Action:** Execute `cts-suite`; evaluate the Gates; retrieve the Report the implementation produced for the batch.
- **Expected:** `signals.retries` shows one Retry for `s-1`. `signals.low_confidence` shows, for `cts-scripted-judge@1`, two `inconclusive` Results of three. `signals. multi_judge`, `unenforced_boundaries`, and `boundary_attempts` are present and empty. `signals.unadmitted_judges` lists the Judge configuration of the Expectation: no Admission record exists for it. `gates` holds both Gates, `semantic-gate` with `non_actionable_results` and without `judge_not_admitted`, since it does not require admitted Judges; each with its `basis`.
- **Fails if:** `signals` or any of its six lists is missing; any count above is wrong; `unadmitted_judges` omits the Judge; or `semantic-gate` reports `judge_not_admitted`.

CT-REL-036 — A KPI decomposes into the Metrics and Results behind it

- **Capability:** core
- **Requirement:** Volume XI, KPIs: “a KPI MUST be traceable back to the Metric(s) it is drawn from.”
- **Method:** Inspection
- **Fixture:** F-SIGNALS (2.7), and a KPI the tester defines in the implementation: the pass rate of `cts-suite`.
- **Action:** From the KPI as displayed, follow it to the Metric(s) it is drawn from and from them to the Results.
- **Expected:** The reviewer reaches the Metric and the individual Results from the KPI (for a Metric shown in a Report, through its entry in `metrics`, Appendix C §C.10), citing the records used. If the implementation offers no KPIs, which the specification does not require, the outcome is `NOT_APPLICABLE` (1.2).
- **Fails if:** The KPI is offered but cannot be traced to its Metric or Results.

CT-REL-037 — A platform’s native gate is not an AQEF Quality Gate

- **Capability:** `ci_cd:<platform>`
- **Requirement:** Volume XIII: an AQEF Quality Gate and a platform’s own gating primitive “MUST NOT be conflated: AQEF’s Quality Gate is defined once, platform-agnostically.”
- **Method:** Inspection
- **Fixture:** CT-REL-024’s configuration and pipeline.
- **Action:** Carry out CT-REL-024’s Action. Review the integration’s configuration and the platform’s records for the pipeline run.
- **Expected:** The Gates are defined and evaluated in the implementation; the platform’s native primitive only carries their outcome. Changing the platform’s primitive does not change any Gate definition, and the reverse.
- **Fails if:** A Gate is defined, evaluated, or changed in the platform rather than in the implementation.

CT-REL-038 — Audit records what changed state, and reading it changes nothing

- **Capability:** core
- **Requirement:** Volume XII, Audit: “Audit MUST record every action that changes state, when it takes effect … Each record names the Principal, its kind (Person or Service Account), the action, its target, and when it took effect … an implementation that records [reads] keeps them apart from this record, so that reading the audit log never changes what is being read.”
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; scripted Judge `J-PASS-90`.
- **Action:**
  - As `P-NOPERM`, execute `cts-suite`.
  - As `P-RELEASE`, evaluate the Gates and record a Release Decision.
  - As `P-ADMIN`, read the audit log twice, in full, one read straight after the other.
- **Expected:** The log holds an entry for the Execution naming `P-NOPERM`, kind Person, and one for the Release Decision naming `P-RELEASE`, kind Person, each with action, target, and time. The two reads return the same entries.
- **Fails if:** Either action has no entry, or its entry lacks the Principal, its kind, the action, the target, or the time; or the second read differs from the first.

CT-REL-039 — A Team binding denies access outside the Team

- **Capability:** core
- **Requirement:** Volume XII, Teams: “Where an implementation binds a Project or a Suite to Teams, it MUST deny a Principal outside those Teams any access to it, whatever Permissions the Principal’s Roles grant. An implementation MAY bind none”.
- **Method:** Automated
- **Fixture:** F-BASE, bound to a Team `cts-team` in the way the Conformance Statement describes. `P-OUTSIDE`: a Person holding `read` and `execute`, not in `cts-team`.
- **Action:** As `P-OUTSIDE`, retrieve `cts-base`, and execute `cts-suite`.
- **Expected:** Both are refused, and no Execution is started. Where the Conformance Statement says the implementation binds no Project or Suite to Teams, the test is `NOT_APPLICABLE` (1.2: a feature the specification leaves optional).
- **Fails if:** Either request succeeds.

14.3 Core coverage

Not tests: a map of which tests exercise each part of the non-negotiable core (§10). Only tests whose capability is `core` are listed. A part of the core with no `PASS` among its tests has not been shown to be met.

Core part	Tests
Layer: Definition	CT-CFG-001…028
Layer: Execution	CT-EXE-001…007, CT-EXE-009…011
Layer: Evidence	CT-EXE-003, CT-EXE-008, CT-EXE-012
Layer: Assessment	CT-ASM-001…012, CT-ASM-014…020
Layer: Decision	CT-DEC-001…009, CT-DEC-013, CT-DEC-014, CT-DEC-019…024, CT-REL-001…023, CT-REL-029…036, CT-REL-038, CT-REL-039
Conformance Statement	CT-REL-027
Principle: Contracts over Assertions	CT-CFG-002, CT-CFG-003, CT-CFG-004, CT-CFG-007, CT-CFG-012, CT-CFG-013, CT-CFG-015, CT-CFG-019, CT-ASM-011, CT-ASM-019, CT-REL-036
Principle: Validation before Evaluation	CT-ASM-001, CT-ASM-002, CT-ASM-014, CT-ASM-017
Principle: Independent Oracles	CT-CFG-006, CT-ASM-012
Principle: Deterministic Infrastructure	CT-EXE-002, CT-EXE-006, CT-EXE-007, CT-ASM-009, CT-ASM-010, CT-ASM-016, CT-ASM-018, CT-EXE-010…012, CT-DEC-022, CT-DEC-023, CT-REL-010…014, CT-REL-034
Principle: Probabilistic Subject	CT-EXE-003, CT-EXE-004, CT-EXE-005, CT-ASM-008, CT-ASM-015, CT-DEC-013
Principle: Everything is a Test	None in this version
Principle: Everything has Confidence	CT-ASM-003…007, CT-DEC-002…008, CT-DEC-021
The whole loop	CT-REL-026

“Everything is a Test” has no test yet. Its clearest observable forms, assessing a captured production Conversation or one from an exploratory session, depend on `dataset:production_replay` or on surfaces outside this version’s scope.

15. API Behavior

Tests in this section check behavior of the REST API (Appendix C) that belongs to no single layer.

CT-API-001 — List operations are paginated, the same way everywhere

- **Capability:** core
- **Requirement:** Appendix C §C.6: “List operations (GET on collection endpoints) MUST support pagination for unbounded collections”; “The pagination mechanism is implementation-defined … but MUST be consistent across all endpoints.”
- **Method:** Automated
- **Fixture:** F-MULTI, with `s-1`’s Contract given further schema Constraints, each a copy of the first, so that one Execution of `s-1` has more than `p` Results, where `p` is the smallest page size the implementation allows (the tester states `p`).
- **Action:** Execute `cts-suite` as many times as it takes for the Project to hold more than `p` Executions. As `P-APPROVER`, make governed changes (14.1) to `semantic-gate`’s `threshold`, alternating `0.9` and `1.0`, until the audit log holds more than `p` entries. With page size `p`, request `GET /projects/cts-base/executions`, `GET .../executions/{execution}/results` for one Execution of `s-1`, and `GET /projects/cts-base/audit` (as `P-ADMIN`, who holds `audit_read`). Follow each to its last page. Separately, request each collection unpaginated or with the largest page size allowed, as the reference set.
- **Expected:** Each of the three collections is returned in at least two pages, and the next page is reached the same way on all three (the same parameters and the same mechanism). For each, the items over all pages are exactly the reference set: none missing, none repeated, in a stable order.
- **Fails if:** Any of the three returns more than `p` items on one page or is not paginated; the mechanism differs between them; or an item is missing or repeated across pages.

CT-API-002 — Deleting a Project does not silently delete its history

- **Capability:** core
- **Requirement:** Appendix C §C.8: “`DELETE /projects/{project}` MUST NOT be permitted while the Project has Baselines, Reports, or audit records unless the caller explicitly requests their retention or their deletion is separately, explicitly authorized”; Governance “MUST be able to distinguish which one a request is actually asking for.”
- **Method:** Automated
- **Fixture:** F-BASELINED (2.7), so that `cts-base` has a Baseline and audit records.
- **Action:** As `P-ADMIN` (Section 14.1): (a) `DELETE /projects/cts-base` with no further statement; (b) the same, asking for the history to be deleted, without any separate authorization; © the same, asking for the history to be retained, in the form the implementation documents. After ©, query the retained Baseline and audit records through whatever the implementation documents for retained history.
- **Expected:** (a) and (b) are rejected and nothing is deleted. © is either rejected, with nothing deleted, or accepted. Where it is accepted, the Baseline and the audit records, including the record of the deletion itself, are still retrievable, and the record shows which kind of deletion was requested. (The specification forbids deletion without one of these conditions; it does not require an implementation to delete a Project once one is met.)

- **Fails if:** (a) or (b) deletes anything; © deletes anything while being rejected; an accepted © loses the Baseline or any audit record; or the record of an accepted © does not distinguish deleting the configuration from deleting the history.

16. Admission of Judges

Tests in this section check Admission (Volume VI; Appendix C §C.3, Admission): the record of a Judge configuration's agreement with a Reference Set, and its effect on Quality Gates and Reports. Admission is not a scoped capability in §10, so these tests are core, except where a test names another capability. Section 13.1 applies here: "evaluate the Gates" and `semantic-gate` have its meanings. The Principals are those of Section 14.1.

16.1 Fixtures

Recording an Admission. "Record `ADM-PASS`" means `POST /projects/cts-base/admissions` (Appendix C §C.3) with the body below, made by `P-REVIEWER` unless the test says otherwise. "The Admission records" are what `GET /projects/cts-base/admissions` returns.

`ORACLE-1`. The `oracle` value (Appendix C §C.7) on the Result of `s-greeting`'s Expectation in a setup Execution of the test's fixture. Every Admission request below names it, unless the test says otherwise. Setup check: the Result carries an `oracle`.

`ADM-PASS`. An Admission request that meets its threshold:

```
oracle: ORACLE-1
reference_set: cts-greetings@1
items: 20
verdict_distribution: { pass: 14, fail: 6 }
raw_agreement: 0.90
measure: cohen_kappa
chance_corrected_agreement: 0.76
threshold: 0.70
admitted: true
labelled_by: [P-REVIEWER, P-APPROVER]
```

`ADM-FAIL`. As `ADM-PASS`, with `raw_agreement: 0.80`, `chance_corrected_agreement: 0.52`, and `admitted: false`.

`SA-REVIEW`. A Service Account owned by `P-REVIEWER`, holding `read` and `review`.

`gate-admitted`. `semantic-gate` with `require_admitted_judges: true` added.

16.2 Tests

CT-ADM-001 — A Person records an Admission, and the implementation sets its identity

- **Capability:** core
- **Requirement:** Appendix C §C.3, `POST /projects/{project}/admissions`: "The Principal MUST be a Person"; "The implementation sets `id`, `recorded_at`, and `recorded_by`"; Volume VI, Admission: "The record MUST state the Reference Set and its version, how many items were scored and how their correct

verdicts were distributed, the raw agreement, and an agreement measure corrected for chance, by name and value."

- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`; `SA-REVIEW`.
- **Action:** Record `ADM-PASS` (a) as `SA-REVIEW`; (b) as `P-NOPERM`; © as `P-REVIEWER`. Retrieve the Admission records as `P-NOPERM`.
- **Expected:** (a) rejected: a Service Account, though it holds `review`. (b) rejected: `P-NOPERM` lacks `review`. © accepted. The records hold exactly one record, with every field of `ADM-PASS` as sent, and with `id`, `recorded_at`, and `recorded_by` set by the implementation, `recorded_by` naming `P-REVIEWER`.
- **Fails if:** (a) or (b) is accepted or leaves a record; any field differs from the request; or `id`, `recorded_at`, or `recorded_by` is missing.

CT-ADM-002 — An incomplete or inconsistent Admission record is rejected

- **Capability:** core
- **Requirement:** Appendix C §C.3, Admission: "a request that supplies any of them [`id`, `recorded_at`, `recorded_by`] MUST be rejected. Every other field is required except `arbitrations`. … `labelled_by` names at least one reviewer. `items` is at least 1, the counts in `verdict_distribution` are not negative and add up to `items`, and `raw_agreement` lies between 0 and 1. `chance_corrected_agreement` is a number or `"not_computed"`, never `null` … A request that breaks any of these MUST be rejected."
- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`.
- **Action:** Record each variant of `ADM-PASS`, in a fresh Project each:
  - Rejected variants: (a) to (j) each omit one of `oracle`, `reference_set`, `items`, `verdict_distribution`, `raw_agreement`, `measure`, `chance_corrected_agreement`, `threshold`, `admitted`, `labelled_by`; (k) `labelled_by: []`; (l) `items: 0` with `verdict_distribution: {}`; (m) `verdict_distribution: { pass: 15, fail: 6 }`, which adds up to 21; (n) `verdict_distribution: { pass: 21, fail: -1 }`, which adds up to 20; (o) `raw_agreement: 1.2`; (p) `raw_agreement: -0.1`; (q) `chance_corrected_agreement: null`; ® `id: adm-1` added; (s) `recorded_at: 2030-01-01T00:00:00Z` added; (t) `recorded_by: P-REVIEWER` added.
  - Accepted variants: (u) `ADM-PASS` unchanged, which has no `arbitrations`; (v) `arbitrations: []` added.
  - After each, retrieve the Admission records.
- **Expected:** (a) to (t) rejected, each leaving no record. (u) and (v) accepted, each leaving one record; (u)'s has no `arbitrations` and (v)'s has an empty one.
- **Fails if:** Any of (a) to (t) is accepted or leaves a record; (u) or (v) is rejected; or (u)'s record shows an empty `arbitrations`, or (v)'s none.

CT-ADM-003 — `admitted` agrees with the threshold, on the chance-corrected value

- **Capability:** core
- **Requirement:** Appendix C §C.3, `POST /projects/{project}/admissions`: "MUST be rejected if `admitted` is `true` while `chance_corrected_agreement` is below `threshold`, or `false` while it meets it"; Admission: "where it is `"not_computed"`, `admitted` is `false`. A value meets `threshold` when it is equal to it or above it."
- **Method:** Automated

- **Fixture:** F-BASE with `semantic-gate`.
- **Action:** Record each variant of `ADM-PASS`, in a fresh Project each: (a) `chance_corrected_agreement: 0.70`, `admitted: true`; (b) `chance_corrected_agreement: 0.70`, `admitted: false`; © `chance_corrected_agreement: 0.69`, `admitted: false`; (d) `chance_corrected_agreement: 0.69`, `admitted: true`; (e) `raw_agreement: 0.95`, `chance_corrected_agreement: 0.40`, `admitted: true`; (f) `verdict_distribution: { pass: 20 }`, `raw_agreement: 1.0`, `chance_corrected_agreement: "not_computed"`, `admitted: false` (every item has one correct verdict, and the Judge gave it every time, so Cohen's kappa is undefined); (g) as (f) with `admitted: true`.
- **Expected:** (a), ©, and (f) accepted. (b), (d), (e), and (g) rejected, each leaving no record.
- **Fails if:** (a) is rejected (a value equal to the threshold meets it); (e) is accepted on its raw agreement; or any other variant has the other outcome.

CT-ADM-004 — The most recent Admission record decides, at the Gate and in the Report

- **Capability:** core
- **Requirement:** Appendix B §B.3, Gate reasons: "A Gate declaring `require_admitted_judges` whose aggregate includes a Result from a Judge configuration that is not admitted MUST fail, with `judge_not_admitted`"; Admission record: "The most recent record for a Judge configuration, the one the implementation accepted last, decides whether it is admitted"; Appendix C §C.3: "a later record with `admitted: false` withdraws an earlier admission"; "An Admission record is never modified"; Appendix C §C.10, `unadmitted_judges`.
- **Method:** Automated
- **Fixture:** F-BASE with `gate-admitted`; scripted Judge `J-PASS-90`. Setup: execute `cts-suite` and read `ORACLE-1` (16.1).
- **Action:** Each step is made only after the previous one has returned.
  - (a) Evaluate the Gates for the setup batch; retrieve a Report for the batch generated after the evaluation.
  - (b) Record `ADM-PASS`; evaluate the Gates; retrieve a Report.
  - © Record `ADM-FAIL`; evaluate the Gates; retrieve a Report.
  - (d) Record `ADM-PASS` again; evaluate the Gates; retrieve a Report.
  - (e) Retrieve the Admission records, and the GateResults of (a) to ©.
- **Expected:**
  - (a) `semantic-gate` fails with `judge_not_admitted`, with `basis.results: 0` and `basis.not_actionable: 1`; `functional-gate` passes; `signals.unadmitted_judges` lists `ORACLE-1`.
  - (b) `semantic-gate` passes; `unadmitted_judges` is empty.
  - © `semantic-gate` fails with `judge_not_admitted`; `unadmitted_judges` lists `ORACLE-1`.
  - (d) `semantic-gate` passes; `unadmitted_judges` is empty.
  - (e) Three records, each exactly as returned when it was recorded. The GateResults of (a) to © are recorded unchanged.
- **Fails if:** © passes, so that an earlier record with `admitted: true` outweighs a later one; (d) fails; a Report disagrees with the Gate on whether `ORACLE-1` is admitted; or any record or earlier GateResult was changed.

CT-ADM-005 — A changed Judge configuration needs its own admission

- **Capability:** core

- **Requirement:** as CT-ADM-004; Volume VI, Admission: “Admission belongs to one Judge configuration …: a changed model, prompt, or criteria is a different Judge and needs its own admission.”
- **Method:** Automated
- **Fixture:** F-BASE with `gate-admitted`; scripted Judge `J-PASS-90`. Setup: execute `cts-suite`, read `ORACLE-1`, and record `ADM-PASS`.
- **Action:** As a governed change (14.1), set the Expectation’s `judge.model` to `cts-scripted-judge@2`, which behaves as `J-PASS-90`. Execute `cts-suite` again; read the `oracle` on the new Result of the Expectation (`ORACLE-2`). Evaluate the Gates for the new batch; retrieve a Report for it.
- **Expected:** `ORACLE-2` differs from `ORACLE-1`. `semantic-gate` fails with `judge_not_admitted`. `signals.unadmitted_judges` lists `ORACLE-2`, and not `ORACLE-1`, whose Results are not in scope.
- **Fails if:** `ORACLE-2` equals `ORACLE-1`; or `semantic-gate` passes on `ORACLE-1`’s admission.

CT-ADM-006 — A panel is admitted only when every Judge in it is

- **Capability:** `multi_judge`
- **Requirement:** as CT-ADM-004; Volume VI, Admission: “A Multi Judge panel is admitted when every Judge configuration in it is.”
- **Method:** Automated
- **Fixture:** F-PANEL (Section 21), with `semantic-gate` given `require_admitted_judges: true`; all three Judges behave as `J-PASS-90`. A panel member with the same model and criteria configuration as a single Judge is the same Judge configuration (Appendix C §C.7, `oracle`), so each member’s identifier is read from a single-Judge run. Setup, in the test’s Project: bind `s-greeting`’s Expectation to `cts-scripted-judge-a@1` alone, with `confidence_source: self_rating` and the criteria and `confidence_threshold` of F-BASE; execute `cts-suite` and read the `oracle` on the Expectation’s Result (`ORACLE-A`). Do the same for `-b` and `-c` (`ORACLE-B`, `ORACLE-C`), each bound as a governed change (14.1). Then bind the Expectation to F-PANEL’s panel, as a governed change, and execute `cts-suite`; the Gates below are evaluated for this batch. Setup check: the three identifiers differ.
- **Action:** (a) Record `ADM-PASS` for `ORACLE-A` and for `ORACLE-B`; evaluate the Gates. (b) Record `ADM-PASS` for `ORACLE-C`; evaluate the Gates. © Record `ADM-FAIL` for `ORACLE-B`; evaluate the Gates.
- **Expected:** (a) `semantic-gate` fails with `judge_not_admitted`. (b) It passes. © It fails with `judge_not_admitted`.
- **Fails if:** (a) or © passes; or (b) fails.

CT-ADM-007 — A Gate that lacks admitted Judges lists every other reason too

- **Capability:** core
- **Requirement:** Appendix B §B.3, Gate reasons: “A failed GateResult MUST list every applicable reason. … A Gate declaring `require_admitted_judges` whose aggregate includes a Result from a Judge configuration that is not admitted MUST fail, with `judge_not_admitted`.”
- **Method:** Automated
- **Fixture:** F-BASE with `gate-admitted`, given `min_basis: { results: 2 }`; scripted Judge `J-PASS-90`; no Admission record.
- **Action:** Execute `cts-suite`; evaluate the Gates.
- **Expected:** `semantic-gate` fails, and `reasons` contains both `judge_not_admitted` and `insufficient_basis`.
- **Fails if:** Either reason is missing.

20. Capability: Human Review

Tests in this section apply to an implementation whose Conformance Statement claims `human_review` (§10). For any other, they are `NOT_APPLICABLE`. CT-DEC-010 to CT-DEC-012 and CT-REL-025 also belong to this capability.

F-HR. As F-BASE with `semantic-gate` (Section 13.1), with the Expectation bound to a Human Reviewer instead of the Judge:

```
expectations:
  - criteria: "The response greets the user."
    human_reviewer:
      role: reviewer
```

`P-REVIEWER` (Section 14.1) holds the Role `reviewer`. `P-REVIEWER-2` is a Person who holds `review` but only a Role named `qa`.

CT-HRV-001 — A Human-Reviewer-bound Expectation names the Role that resolves it

- **Capability:** `human_review`
- **Requirement:** Appendix A §A.5, HumanReviewerConfig `role`: required; “Role a reviewer MUST hold to resolve this Expectation.”
- **Method:** Automated
- **Fixture:** F-HR with `role` removed.
- **Action:** Load the configuration.
- **Expected:** Rejected.
- **Fails if:** Loaded.

CT-HRV-002 — Only a reviewer holding the named Role resolves the Expectation

- **Capability:** `human_review`
- **Requirement:** Appendix A §A.5: the Role “a reviewer MUST hold to resolve this Expectation”; Appendix C §C.3, review: the Principal “MUST be a Person holding `review` and, where the clause names a `human_reviewer.role` …, that Role.”
- **Method:** Automated
- **Fixture:** F-HR.
- **Action:** Execute `cts-suite`. Review the pending Result with verdict `pass` as `P-REVIEWER-2`; retrieve the Result. Then review it with verdict `pass` as `P-REVIEWER`; retrieve it again, and evaluate the Gates.
- **Expected:** `P-REVIEWER-2`’s review is rejected with an error, and the Result is unchanged: `disposition: awaiting_review`, `verdict: null`, and no second Result for the Expectation. `P-REVIEWER`’s review is accepted and completes the same Result: the same `id`, `verdict: pass`, `disposition: actionable`, and `supporting_evidence` naming `P-REVIEWER` as the reviewing Person. `semantic-gate` passes.
- **Fails if:** `P-REVIEWER-2`’s review is accepted or changes the Result; a second Result is recorded for the Expectation; `P-REVIEWER`’s review is rejected or leaves the Result `awaiting_review`; the completed Result names `P-REVIEWER-2`, or no reviewer; or `semantic-gate` fails.

CT-HRV-003 — A review that times out never becomes a pass

- **Capability:** `human_review`
- **Requirement:** Volume VI, Human Review: awaiting review "MUST NOT persist indefinitely … after which the Contract's `on_timeout` policy applies"; Appendix A §A.5, `on_timeout`: "Default: `block` — an unresolved review is never silently treated as a pass."
- **Method:** Automated
- **Fixture:** F-HR, with `review_timeout` set to the shortest duration the implementation accepts and no `on_timeout`. The tester states the waiting budget (1.6).
- **Action:** Execute `cts-suite`. Within the budget, wait until the implementation records that `review_timeout` was reached; do not review. Evaluate the Gates.
- **Expected:** The Result has no verdict and is not `actionable`, and `semantic-gate` fails with `non_actionable_results`. The implementation's records show that the timeout was reached and `block` applied.
- **Fails if:** The Result gains a verdict without a review, `semantic-gate` passes, or nothing records that the timeout was reached.

CT-HRV-004 — A reviewer's Confidence, where captured, is held to the threshold

- **Capability:** `human_review`
- **Requirement:** Volume I, Chapter 6, Confidence Model: below the threshold, "a Result's disposition is `inconclusive` — it MUST NOT be silently counted as an ordinary pass or fail"; Appendix A §A.5, `confidence_threshold` and `captures_confidence`; Appendix C §C.7, `confidence`.
- **Method:** Automated
- **Fixture:** Two variants of F-HR. (a) `captures_confidence: true` and `confidence_threshold: 0.8`. (b) `captures_confidence: false` and no threshold.
- **Action:** For (a), execute and review with verdict `pass` and Confidence `0.6`. For (b), execute and review with verdict `pass`.
- **Expected:** (a) The Result has `confidence: 0.6` and `disposition: inconclusive`. (b) The Result has `confidence: "not_applicable"` and `disposition: actionable`.
- **Fails if:** (a) is `actionable`, or (b) has a numeric Confidence or is not `actionable`.

CT-HRV-005 — An escalated review passes to the named Role and never decides itself

- **Capability:** `human_review`
- **Requirement:** Volume VI, Human Review: "`escalate` passes the review to the Role named in `escalate_to`, and the Result stays `awaiting_review` until a reviewer holding that Role returns a verdict. Neither policy ever produces a verdict by itself"; Appendix C §C.3, review: "or the `escalate_to` Role once the review has been escalated."
- **Method:** Automated
- **Fixture:** F-HR with `review_timeout` set to the shortest duration the implementation accepts, `on_timeout: escalate`, and `escalate_to: lead`. The tester states the waiting budget (1.6). `P-LEAD` is a Person holding `review` and only the Role `lead`.
- **Action:** Execute `cts-suite`. Before the timeout, attempt the review as `P-LEAD`. Within the budget, wait until the implementation records the escalation; evaluate the Gates. Review with verdict `pass` as `P-LEAD`; evaluate the Gates again.

- **Expected:** The review before the timeout is rejected. After the timeout, the Result is still `awaiting_review`, with no verdict, and `semantic-gate` fails with `non_actionable_results`. `P-LEAD`'s review is then accepted, and `semantic-gate` passes.
- **Fails if:** `P-LEAD` can review before escalation; the Result gains a verdict without a review; or `P-LEAD`'s review after escalation is rejected.

CT-HRV-006 — `escalate_to` appears exactly when escalation is configured

- **Capability:** `human_review`
- **Requirement:** Appendix A §A.5, `escalate_to`: "MUST (if `on_timeout` is `escalate`)" and "MUST NOT appear otherwise"; Appendix B §B.2, Expectation.
- **Method:** Automated
- **Fixture:** Two variants of F-HR. (a) `on_timeout: escalate` with no `escalate_to`. (b) `on_timeout: block` with `escalate_to: lead`.
- **Action:** Load each variant.
- **Expected:** Both rejected.
- **Fails if:** Either is loaded.

21. Capability: Multi Judge

Tests in this section apply to an implementation whose Conformance Statement claims `multi_judge` (§10). For any other, they are `NOT_APPLICABLE`. CT-ASM-013 also belongs to this capability.

F-PANEL. As F-BASE with `semantic-gate` (Section 13.1), with the Expectation's `judge` replaced by a three-Judge panel:

```
multi_judge:
  purpose: independence
  judges:
    - model: cts-scripted-judge-a@1
      confidence_source: self_rating
    - model: cts-scripted-judge-b@1
      confidence_source: self_rating
    - model: cts-scripted-judge-c@1
      confidence_source: self_rating
  consensus:
    method: voting
    threshold: majority
    on_disagreement: fail
```

Each identifier resolves to the scripted Judge, which the tester can set separately for each. Unless a test says otherwise, all three behave as `J-PASS-90`.

CT-MJG-001 — A panel declares its purpose

- **Capability:** `multi_judge`
- **Requirement:** Volume VI, Multi Judge: "a Multi Judge configuration MUST be explicit about which it is providing"; Appendix A §A.7, `purpose`: "MUST be declared explicitly."
- **Method:** Automated
- **Fixture:** F-PANEL with `purpose` removed.
- **Action:** Load the configuration.
- **Expected:** Rejected.
- **Fails if:** Loaded, or loaded with a purpose supplied silently.

CT-MJG-002 — Each purpose has its own shape

- **Capability:** `multi_judge`
- **Requirement:** Appendix B §B.2, MultiJudgeConfig: for `independence`, "`judges` REQUIRED, `judge`/`samples` MUST NOT appear"; for `stability`, "`judge` and `samples` REQUIRED, `judges` MUST NOT appear."
- **Method:** Automated
- **Fixture:** Four variants of F-PANEL. (a) `independence` with `judges` removed. (b) `independence` with `samples: 3` added. © `purpose: stability` with `judges` kept and no `judge` or `samples`. (d) `purpose: stability` with `judge` and `samples: 3`, and `judges` kept.
- **Action:** Load each variant.

- **Expected:** All four rejected.
- **Fails if:** Any is loaded.

CT-MJG-003 — A panel declares how it reaches Consensus

- **Capability:** `multi_judge`
- **Requirement:** Appendix A §A.7: `consensus` and its `method` are required.
- **Method:** Automated
- **Fixture:** Two variants of F-PANEL. (a) `consensus` removed. (b) `consensus.method` removed.
- **Action:** Load each variant.
- **Expected:** Both rejected.
- **Fails if:** Either is loaded.

CT-MJG-004 — A voting threshold is a number above one half, and only for voting

- **Capability:** `multi_judge`
- **Requirement:** Appendix B §B.2, ConsensusConfig: “A numeric threshold MUST be a number, not a string”; `threshold` is “greater than 0.5 and at most 1.0” and valid “only when `method` is `voting`”; `on_disagreement` is `human_review` or `fail`.
- **Method:** Automated
- **Fixture:** Four variants of F-PANEL. (a) `threshold: "0.75"` (a string). (b) `threshold: 0.5`. © `method: confidence_weighted_mean` with `threshold: 0.75`. (d) `on_disagreement: flag`, a value v0.38 removed.
- **Action:** Load each variant.
- **Expected:** All four rejected. F-PANEL with `threshold: 0.75` is loaded.
- **Fails if:** Any variant is loaded, or the valid one is rejected.

CT-MJG-005 — Votes are counted against the threshold, rounding up

- **Capability:** `multi_judge`
- **Requirement:** Volume VI, Voting: “A verdict is reached when the pass votes, or the fail votes, number at least the threshold times that count, rounded up … Where neither side reaches the threshold … the Consensus’s `on_disagreement` policy applies”; Appendix A §A.7.
- **Method:** Automated, Harness
- **Fixture:** F-PANEL with `cts-scripted-judge-c@1` behaving as `J-FAIL-90`.
- **Action:** Execute `cts-suite` with `threshold: majority`; then with `threshold: 0.75`.
- **Expected:** Each run calls all three Judges once. With `majority`, two pass votes of three reach a verdict: `verdict: pass`, `panel: { configured: 3, counted: 3 }`. With `0.75`, three votes are needed and neither side has them, so `on_disagreement: fail` applies: `verdict: fail`.
- **Fails if:** Either verdict differs, or `panel` is absent or wrong.

CT-MJG-006 — A stability panel samples one Judge the declared number of times

- **Capability:** `multi_judge`
- **Requirement:** Appendix A §A.7, `judge` and `samples` (stability); Appendix C §C.7, `panel`: “In a stability configuration, samples configured (`samples`) and samples counted.”

- **Method:** Automated, Harness
- **Fixture:** F-PANEL with `purpose: stability`, `judges` replaced by `judge: { model: cts-scripted-judge-a@1, confidence_source: self_rating }` and `samples: 3`.
- **Action:** Execute `cts-suite`; count the calls `cts-scripted-judge-a@1` received; retrieve the Results for the Expectation; evaluate the Gates.
- **Expected:** Three calls. Exactly one effective Result for the Expectation's clause in this Execution, with `verdict: pass` and `panel: { configured: 3, counted: 3 }`. `semantic-gate` counts that one Result (`basis.results: 1`). Records of the individual samples, if the implementation keeps them, are not effective Results for the clause and are not counted by any Gate.
- **Fails if:** The number of calls is not 3; there is not exactly one effective Result for the clause; `panel` differs; or `semantic-gate` counts more than one Result.

CT-MJG-007 — Agreement between Judges is not presented as a Judge's own Confidence

- **Capability:** `multi_judge`
- **Requirement:** Volume VI, Confidence: agreement across Judges "conflates with Consensus and MUST be treated as a distinct signal, not a substitute for a Judge's own stated Confidence."
- **Method:** Automated, Inspection
- **Fixture:** F-PANEL with `cts-scripted-judge-c@1` behaving as `J-FAIL-90`.
- **Action:** Execute `cts-suite`; retrieve the Result and whatever per-Judge records the implementation keeps. Review how agreement is reported.
- **Expected:** Each Judge's own Confidence (0.9) remains visible as such. Where the implementation reports agreement (here two of three), it is labelled as agreement, not as any Judge's Confidence.
- **Fails if:** The Judges' own Confidence values are lost, or agreement is presented as a Judge's Confidence.

CT-MJG-008 — Below the quorum, the panel does not assess

- **Capability:** `multi_judge` together with `seeded_controls`
- **Requirement:** Appendix B §B.3, Panel reporting: "Below `min_qualified` (or below the full panel when it is absent), the panel's Results for the affected clauses MUST be `not_assessed`."
- **Method:** Automated
- **Fixture:** F-PANEL plus F-SEED's `s-control` (Section 13.1), whose Expectation uses the same panel; `s-control` placed first in `cts-suite`, and `execution: { scheduling: sequential }`, so that the miss is known before `s-greeting` is assessed. `cts-scripted-judge-a@1` behaves as `J-PASS-90` (it misses the control); `-b` and `-c` behave as `J-CATCH`. Two variants: (a) no `min_qualified`; (b) `min_qualified: 2`.
- **Action:** Execute `cts-suite` for each variant; retrieve `s-greeting`'s Expectation Result.
- **Expected:** (a) The Result is `not_assessed`, `verdict: null`. (b) The Result has a verdict (`pass`) and `panel: { configured: 3, counted: 2 }`.
- **Fails if:** (a) has a verdict, or (b) is `not_assessed` or counts Judge `a`.

CT-MJG-009 — A false alarm counts against the quorum as a miss does

- **Capability:** `multi_judge` together with `seeded_controls`
- **Requirement:** as CT-MJG-008; Appendix B §B.3, Clean control: a false alarm "is a miss for every rule on missed Seeded Controls (invalidation, failing fast, quorum, …)".
- **Method:** Automated

- **Fixture:** F-PANEL plus F-CLEAN's `s-clean` (Section 22), whose Expectation uses the same panel; `s-clean` placed first in `cts-suite`, and `execution: { scheduling: sequential }`. `cts-scripted-judge-a@1` behaves as `J-ALARM` (it raises a false alarm on the control); `-b` and `-c` behave as `J-PASS-90`. Two variants: (a) no `min_qualified`; (b) `min_qualified: 2`.
- **Action:** Execute `cts-suite` for each variant; retrieve `s-greeting`'s Expectation Result.
- **Expected:** (a) The Result is `not_assessed`, `verdict: null`. (b) The Result has a verdict (`pass`) and `panel: { configured: 3, counted: 2 }`.
- **Fails if:** (a) has a verdict, or (b) is `not_assessed` or counts Judge `a`.

22. Capability: Seeded Controls

Tests in this section apply to an implementation whose Conformance Statement claims `seeded_controls` (§10). For any other, they are `NOT_APPLICABLE`. CT-DEC-015 to CT-DEC-018 also belong to this capability. F-SEED, `J-CATCH`, and `semantic-gate` are defined in Section 13.1. CT-SCT-001 to CT-SCT-008 use defect controls; CT-SCT-009 to CT-SCT-015 use clean controls.

Further scripted subject behavior.

Code	Behavior
`SR-BYE`	Returns the Response `"Goodbye."`.

F-SEED-V. As F-SEED, with `s-control` confirmed by a Validator instead of by construction: `evidence` removed, and

```
seeded_control:
  defect_class: greeting-missing
  planted_defect: "The response does not greet the user."
  confirmed_by: validator
  confirmation:
    validator: schema
    target: response
    expected: { type: string, pattern: "Goodbye" }
variables:
  query: "Hello"
prompt:
  template: "{{query}}"
```

Here the “planted defect” is a farewell instead of a greeting; the confirmation passes only if the Response the Judge receives contains it.

Clean controls. A clean control (`kind: clean`, Appendix A §A.4) carries Evidence free of its defect class, which the Oracle is expected to pass. The tests below use two more scripted behaviors and two fixtures.

Code	Behavior
`SR-HI`	Returns the Response `"Hi there! What do you need?"`.
`J-ALARM`	Verdict `fail`, Confidence `0.90` for the Response `Hi there! What do you need?`; otherwise as `J-PASS-90`.

`Hi there! What do you need?` greets the user, so a Judge that fails it raises a false alarm on the class `greeting-missing`. `J-ALARM` passes `s-greeting`’s Response.

F-CLEAN. As F-SEED, with `s-control` replaced by a clean control, confirmed by construction:

```
- name: s-clean
  seeded_control:
    kind: clean
    defect_class: greeting-missing
    confirmed_by: construction
  evidence:
    conversation:            # in the implementation's own Conversation form
      - role: user
        content: "Hello"
      - role: assistant
        content: "Hi there! What do you need?"
  contract:
    expectations:
      - criteria: "The response greets the user."
        defect_classes: [greeting-missing]
        judge:
          model: cts-scripted-judge@1
          confidence_source: self_rating
        confidence_threshold: 0.8
```

F-CLEAN-V. As F-CLEAN, with `s-clean` confirmed by a Validator instead of by construction: `evidence` removed, and

```
seeded_control:
  kind: clean
  defect_class: greeting-missing
  confirmed_by: validator
  confirmation:
    validator: forbidden-content
    target: response
    expected:
      patterns:
        - regex: "Goodbye"
variables:
  query: "Hello"
prompt:
  template: "{{query}}"
```

The confirmation passes only if the Response the Judge receives does not contain the farewell that `greeting-missing` stands for here.

CT-SCT-001 — A Seeded Control states its class, its defect, and how it is confirmed

- **Capability:** `seeded_controls`
- **Requirement:** Volume VIII, Seeded Controls: “A Seeded Control MUST record the defect it plants (`planted_defect`)”; Appendix A §A.4, SeededControlConfig: `defect_class`, `planted_defect`, and `confirmed_by` are required.
- **Method:** Automated
- **Fixture:** Three variants of F-SEED, each with one of `defect_class`, `planted_defect`, `confirmed_by` removed from `s-control`.

- **Action:** Load each variant.
- **Expected:** All three rejected.
- **Fails if:** Any is loaded.

CT-SCT-002 — Each confirmation method has its own shape

- **Capability:** `seeded_controls`
- **Requirement:** Appendix B §B.2, SeededControlConfig: "`confirmation` is REQUIRED when `confirmed_by` is `validator` and MUST NOT appear otherwise. When `confirmed_by` is `construction`, the Scenario MUST carry `evidence` and MUST NOT carry `prompt`, `dataset`, or `variables`"; Appendix B §B.3, Fixed Evidence scope.
- **Method:** Automated
- **Fixture:** Five variants. (a) F-SEED-V without `confirmation`. (b) F-SEED with F-SEED-V's `confirmation` added. © F-SEED without `evidence`. (d) F-SEED with `variables` and `prompt` added. (e) F-BASE with F-SEED's `evidence` added to `s-greeting`, which is not a Seeded Control.
- **Action:** Load each variant.
- **Expected:** All five rejected.
- **Fails if:** Any is loaded.

CT-SCT-003 — An unconfirmed planted defect makes the control invalid

- **Capability:** `seeded_controls`
- **Requirement:** Volume VIII, Seeded Controls: a control's outcome "MUST therefore be `invalid` for the run … unless the presence of its planted defect in that Evidence has been confirmed"; Appendix B §B.3, Seeded Control confirmation.
- **Method:** Automated
- **Fixture:** F-SEED-V with `semantic-gate`; scripted Judge `J-PASS-90`.
- **Action:** Run 1: execute `cts-suite` with scripted subject `SR-OK` for every Scenario; evaluate the Gates. Run 2: execute it with `SR-BYE` for `s-control` and `SR-OK` for `s-greeting`; evaluate the Gates.
- **Expected:** Run 1: the confirmation fails, `s-control`'s outcome is `invalid`, `invalidation_scope: none`, and `semantic-gate` passes, since nothing is invalidated. Run 2: the confirmation passes, the Judge passes `Goodbye.`, the outcome is `missed`, and `semantic-gate` fails with `seeded_control_miss`.
- **Fails if:** Run 1 reports `caught` or `missed`, or invalidates anything; or run 2 reports anything but `missed`.

CT-SCT-004 — A control confirmed by a Human Reviewer is invalid until confirmed

- **Capability:** `seeded_controls`
- **Requirement:** as CT-SCT-003; Appendix A §A.4, `confirmed_by: human_reviewer`: "a Human Reviewer confirms it per run."
- **Method:** Automated
- **Fixture:** F-SEED-V with `confirmed_by: human_reviewer` and `confirmation` removed; scripted subject `SR-BYE` for `s-control`, `SR-OK` for `s-greeting`; scripted Judge `J-PASS-90`.
- **Action:** Execute `cts-suite`; evaluate the Gates. Attempt the confirmation as `SA-CI` (Section 14.1), then make it as `P-REVIEWER`, with `POST .../executions/{execution}/seeded-controls/s-control/confirm` (Appendix C §C.3). Retrieve the Execution again; evaluate the Gates again.

- **Expected:** `SA-CI`'s attempt is rejected (the Principal must be a Person). Before confirmation, the outcome is `invalid` and the first evaluation of `semantic-gate` passes. After it, the outcome is `missed`, the second evaluation of `semantic-gate` fails with `seeded_control_miss`, and the first GateResult is still recorded unchanged.
- **Fails if:** The outcome is `caught` or `missed` before confirmation; it stays `invalid` after; or the first GateResult is altered.

CT-SCT-005 — A difficulty prior is "not established" until it has a basis

- **Capability:** `seeded_controls`
- **Requirement:** Volume VIII, Seeded Controls: "A prior MUST state how many control instances it was established on, and a class or control with no such history MUST be represented as 'not yet established', not as zero"; Appendix A §A.1, DefectClassConfig.
- **Method:** Automated
- **Fixture:** Two variants of F-SEED. (a) `greeting-missing` with `difficulty_prior: 0.9` and no `prior_basis` or `prior_source`. (b) F-SEED unchanged: no prior.
- **Action:** Load (a). Load (b), execute `cts-suite`, and retrieve the defect class and every record in which the implementation reports a prior for it.
- **Expected:** (a) rejected. (b) Wherever the prior is reported, it reads `"not_established"`, never `0` or `0.0`.
- **Fails if:** (a) is loaded, or (b) reports the prior as a number.

CT-SCT-006 — A prior comes from Oracles independent of the one under test

- **Capability:** `seeded_controls`
- **Requirement:** Volume VIII, Seeded Controls: "A prior MUST be established from reference Oracles independent of the Oracle under test … and MUST NOT be updated from how the Oracle under test performs against the controls"; Appendix A §A.1, `reference_oracles`: "Each MUST satisfy Independent Oracles … with respect to every Oracle the class's controls test."
- **Method:** Automated
- **Fixture:** (a) F-SEED with `greeting-missing` given `difficulty_prior: 0.9`, `prior_basis: 50`, `prior_source: reference_oracles`, and `reference_oracles` holding one JudgeConfig, `{ model: cts-scripted-judge@1, confidence_source: self_rating }`, the very Judge the controls test. (b) The same with that JudgeConfig's `model` `cts-reference-judge@1` instead.
- **Action:** Load (a). Load (b); execute `cts-suite` with `J-PASS-90`, so that the Judge misses `s-control`; retrieve the defect class.
- **Expected:** (a) rejected. (b) After the miss, the prior is still `0.9` on a basis of `50`.
- **Fails if:** (a) is loaded, or the miss changed the prior or its basis.

CT-SCT-007 — Full-run invalidation needs a reason and a Permission

- **Capability:** `seeded_controls`
- **Requirement:** Volume VIII, Seeded Controls: full-run invalidation "is a deliberate decision, not the default, and MUST record a reason"; Appendix C §C.3, invalidate: "Body MUST include a non-empty `reason`"; `seeded_control_invalidate` "is an ordinary checked Permission."
- **Method:** Automated

- **Fixture:** F-SEED-MISSED (2.7). `P-INVALIDATE` is a Person holding `read` and `seeded_control_invalidate`.
- **Action:** `POST .../executions/{execution}/seeded-controls/s-control/invalidate` (a) as `P-INVALIDATE` with `reason: ""`; (b) as `P-NOPERM` with a non-empty reason; © as `P-INVALIDATE` with `reason: "Judge prompt changed today"`. Evaluate the Gates. Query the audit log for the action.
- **Expected:** (a) and (b) rejected. © accepted: the entry's `invalidation_scope` becomes `full_run` and `reason` holds the text given. Both Gates now fail with `seeded_control_miss`, `functional-gate` included. The audit log records ©, naming `P-INVALIDATE`.
- **Fails if:** (a) or (b) is accepted; © leaves the scope at `defect_class` or the reason empty; `functional-gate` passes; or © is not in the audit log.

CT-SCT-008 — A requalified Oracle may resolve what its miss invalidated

- **Capability:** `seeded_controls`
- **Requirement:** Volume I, Chapter 6, Result Resolution: a Result "from a requalified Oracle re-assessing stored Evidence … only once the Oracle has been requalified for every defect class whose miss reached that Result"; Volume VIII, Requalification: "Without that record, the Oracle is not requalified"; Appendix C §C.3, Requalification: the request "MUST be rejected unless each named control is of that class, was `caught` (a defect control) or `passed` (a clean control) by that Oracle configuration, and is not a control any Oracle missed or raised a false alarm on in a run whose Results the requalification will be used to re-assess."
- **Method:** Automated
- **Fixture:** F-SEED-MISSED (2.7), with `s-greeting`'s Result invalidated (not `not_assessed`; if the implementation left it `not_assessed`, the same steps apply to that Result). A corrected Judge configuration: model `cts-scripted-judge@2`, behaving as `J-CATCH`, with the same criteria. A fresh control `s-control-2`: as `s-control`, with the fixed Response `"Bye now."`, which `J-CATCH` also fails (13.1). Setup check: `J-CATCH` passed its self-test for `Goodbye.` and `Bye now.` (2.8).
- **Action:** (a) Request that `cts-scripted-judge@2` re-assess `s-greeting`'s stored Evidence. (b) Run `s-control` and `s-control-2` against `cts-scripted-judge@2`. © `POST /projects/cts-base/requalifications` for `cts-scripted-judge@2` and `greeting-missing`, naming its Result on `s-control`. (d) The same, naming its Result on `s-control-2`. (e) Repeat request (a). (f) Evaluate the Gates again for the original batch.
- **Expected:** (a) rejected: no Requalification record yet. (b) Both controls are `caught`. © rejected: `s-control` is the control missed in the run to be re-assessed. (d) accepted, and the record names the Oracle, the class, and the `s-control-2` Result. (e) accepted: a new Result from `cts-scripted-judge@2` with `resolves` naming the invalidated one and `verdict: pass`; the scripted subject is not called. (f) `semantic-gate` passes; the earlier GateResult stays recorded.
- **Fails if:** (a) or © is accepted; (d) or (e) is rejected; (e) calls the subject; or (f) fails.

CT-SCT-009 — A clean control carries no planted defect and no prior

- **Capability:** `seeded_controls`
- **Requirement:** Appendix A §A.4, SeededControlConfig: `planted_defect` is required "if `kind` is `defect`" and "MUST NOT appear on a clean control"; `kind` is `defect` or `clean`; Appendix B §B.2: "`planted_defect` is REQUIRED when `kind` is `defect` or absent, and MUST NOT appear when it is `clean`; `difficulty_prior` MUST NOT appear when `kind` is `clean`."
- **Method:** Automated

- **Fixture:** Five variants. (a) F-CLEAN unchanged. (b) F-CLEAN with `planted_defect: "The response does not greet the user."` added to `s-clean`. © F-CLEAN with `difficulty_prior: not_established` added to `s-clean`. (d) F-CLEAN with `kind: unknown`. (e) F-SEED with `kind: defect` added to `s-control` and its `planted_defect` removed.
- **Action:** Load each variant.
- **Expected:** (a) loaded. (b), ©, (d), and (e) rejected.
- **Fails if:** (a) is rejected for lacking `planted_defect`; or any of (b) to (e) is loaded.

CT-SCT-010 — A clean control the Oracle passes invalidates nothing and is not aggregated

- **Capability:** `seeded_controls`
- **Requirement:** Volume VIII, Seeded Controls: “Seeded Control Results themselves MUST be excluded from Aggregation”; Appendix C §C.3, Seeded Control Reporting: “`GET .../executions/{execution}` MUST include a `seeded_control_results` array, one entry per Seeded Control Scenario in that run”, with `kind`, and `outcome` “`passed` or `false_alarm` for a clean control”.
- **Method:** Automated
- **Fixture:** F-CLEAN with `semantic-gate`; scripted Judge `J-PASS-90`.
- **Action:** Execute `cts-suite`; retrieve the Execution records; evaluate the Gates.
- **Expected:** `seeded_control_results` has one entry for `s-clean`: `kind: clean`, `defect_class: greeting-missing`, `outcome: passed`, `confirmed_by: construction`, `invalidation_scope: none`, `reason: null`. The scripted subject received exactly one call. Both Gates pass; `semantic-gate` has `basis.results: 1` and `basis.scenarios: 1`.
- **Fails if:** The entry is missing, lacks `kind`, or reports `caught`; anything is invalidated; the `s-clean` Result is counted in any `basis` field; or either Gate fails.

CT-SCT-011 — A false alarm invalidates its class and is reported as a false alarm

- **Capability:** `seeded_controls`
- **Requirement:** Volume I, Chapter 6, Quality Gates: “A Quality Gate MUST NOT pass on any aggregate to which an Oracle contributed Results sharing a missed Seeded Control’s defect class”; Appendix B §B.3, Clean control: a false alarm “is a miss for every rule on missed Seeded Controls …, reported as `false_alarm`, with `seeded_control_false_alarm` on an affected GateResult”; Gate reasons: such a Gate “MUST fail, with `non_actionable_results`, `seeded_control_miss`, or `seeded_control_false_alarm`”; Invalidation is derived: a missed control “MUST NOT change any recorded Result.”
- **Method:** Automated
- **Fixture:** F-CLEAN with `semantic-gate`; scripted Judge `J-ALARM`.
- **Action:** Execute `cts-suite`; retrieve the Execution records and every Result; evaluate the Gates.
- **Expected:**
  - The entry for `s-clean` has `kind: clean`, `outcome: false_alarm`, `invalidation_scope: defect_class`, `reason: null`.
  - `s-greeting`’s Expectation either has a recorded Result with `verdict: pass` and `disposition: actionable` whose `invalidated_by` lists `{ scenario: s-clean, defect_class: greeting-missing }`, or, if the Judge was no longer invoked for it, a Result with `disposition: not_assessed` and `verdict: null`.

    - `semantic-gate` fails with `seeded_control_false_alarm` in `reasons`, and without `seeded_control_miss`. Its `basis` counts no Result of `s-clean`.
    - `functional-gate` passes.
- **Fails if:** `semantic-gate` passes; its `reasons` reports the false alarm as `seeded_control_miss` or omits `seeded_control_false_alarm`; the outcome is `missed` or `caught`; a recorded `verdict` or `disposition` was changed; or `functional-gate` fails.

CT-SCT-012 — A clean control is invalid unless its cleanness is confirmed

- **Capability:** `seeded_controls`
- **Requirement:** Appendix B §B.3, Seeded Control confirmation: "A control's outcome MUST be reported as `invalid` for that run, not as `caught`, `missed`, `passed`, or `false_alarm`, unless what it declares about the Evidence the Oracle received … was confirmed"; Appendix A §A.4, `confirmation`: "the planted defect present, or, for a clean control, absent. Where it fails, the control is `invalid` for that run."
- **Method:** Automated
- **Fixture:** F-CLEAN-V with `semantic-gate`; scripted Judge `J-ALARM`.
- **Action:** Run 1: execute `cts-suite` with scripted subject `SR-BYE` for `s-clean` and `SR-OK` for `s-greeting`; evaluate the Gates. Run 2: execute it with `SR-HI` for `s-clean` and `SR-OK` for `s-greeting`; evaluate the Gates.
- **Expected:** Run 1: the confirmation fails, since the Response contains the farewell; `s-clean`'s outcome is `invalid`, `invalidation_scope: none`, and `semantic-gate` passes. Run 2: the confirmation passes, `J-ALARM` fails `Hi there! What do you need?`, the outcome is `false_alarm`, and `semantic-gate` fails with `seeded_control_false_alarm`.
- **Fails if:** Run 1 reports `passed` or `false_alarm`, or invalidates anything; or run 2 reports anything but `false_alarm`.

CT-SCT-013 — A clean control confirmed by a Human Reviewer is invalid until confirmed

- **Capability:** `seeded_controls`
- **Requirement:** as CT-SCT-012; Appendix C §C.3, `POST .../seeded-controls/{scenario}/confirm`: "Record a Human Reviewer's confirmation that the planted defect is present (or, for a clean control, that the defect class is absent) … The Principal MUST be a Person."
- **Method:** Automated
- **Fixture:** F-CLEAN-V with `semantic-gate`, `s-clean` given `confirmed_by: human_reviewer` and its `confirmation` removed; scripted subject `SR-HI` for `s-clean`, `SR-OK` for `s-greeting`; scripted Judge `J-ALARM`.
- **Action:** Execute `cts-suite`; evaluate the Gates. Attempt the confirmation as `SA-CI`, then make it as `P-REVIEWER`, with `POST .../executions/{execution}/seeded-controls/s-clean/confirm`. Retrieve the Execution again; evaluate the Gates again.
- **Expected:** `SA-CI`'s attempt is rejected. Before confirmation, the outcome is `invalid` and the first evaluation of `semantic-gate` passes. After it, the outcome is `false_alarm`, the second evaluation of `semantic-gate` fails with `seeded_control_false_alarm`, and the first GateResult is still recorded unchanged.
- **Fails if:** The outcome is `passed` or `false_alarm` before confirmation; it stays `invalid` after; `SA-CI`'s attempt is accepted; or the first GateResult is altered.

CT-SCT-014 — After a false alarm, requalification rests on passed clean controls

- **Capability:** `seeded_controls`
- **Requirement:** Volume VIII, Requalification: the corrected Oracle "catches fresh defect controls of that class, or, after a false alarm, passes fresh clean controls of it"; Appendix C §C.3, Requalification: the request "MUST be rejected unless each named control is of that class, was `caught` (a defect control) or `passed` (a clean control) by that Oracle configuration, and is not a control any Oracle missed or raised a false alarm on in a run whose Results the requalification will be used to re-assess"; `reassess`: "MUST be rejected … for an Oracle without a Requalification record (below) for every defect class in the Result's `invalidated_by`, resting on controls of the kind that failed: caught defect controls after a miss, passed clean controls after a false alarm".
- **Method:** Automated
- **Fixture:** F-CLEAN with `semantic-gate`; scripted Judge `J-ALARM`. Setup: execute `cts-suite` and evaluate the Gates; setup check: `s-clean`'s entry has `outcome: false_alarm`. `s-greeting`'s Result is invalidated; if the implementation left it `not_assessed`, the same steps apply to that Result. A corrected Judge configuration: model `cts-scripted-judge@2`, behaving as `J-CATCH`, with the same criteria. Two fresh controls of `greeting-missing`: `s-clean-2`, as `s-clean` with the fixed Response `"Hello, what can I do for you?"`, and `s-control-2`, as F-SEED's `s-control` with the fixed Response `"Bye now."`.
- **Action:** (a) Run `s-clean`, `s-clean-2`, and `s-control-2` against `cts-scripted-judge@2`. (b) `POST /projects/cts-base/requalifications` for `cts-scripted-judge@2` and `greeting-missing`, naming its Result on `s-control-2`. © Request that `cts-scripted-judge@2` re-assess `s-greeting`'s stored Evidence from the setup run. (d) Request a requalification as in (b), naming its Result on `s-clean`. (e) The same, naming its Result on `s-clean-2`. (f) Repeat request ©. (g) Evaluate the Gates again for the setup batch.
- **Expected:** (a) `s-clean` and `s-clean-2` are `passed`, `s-control-2` is `caught`. (b) accepted: a defect control the Oracle caught. © rejected: the Result was invalidated by a false alarm, and the only Requalification rests on a defect control. (d) rejected: `s-clean` raised the false alarm in the run to be re-assessed. (e) accepted. (f) accepted: a new Result from `cts-scripted-judge@2` with `resolves` naming the invalidated one and `verdict: pass`; the scripted subject is not called. (g) `semantic-gate` passes; the setup GateResult stays recorded.
- **Fails if:** (b) or (e) is rejected; © or (d) is accepted; (f) is rejected or calls the subject; or (g) fails.

CT-SCT-015 — Under `halt_run`, a false alarm invalidates the whole run

- **Capability:** `seeded_controls`
- **Requirement:** Volume VIII, Clean controls: a false alarm "is a miss in every rule of this specification that speaks of a missed Seeded Control"; Appendix A §A.9, `on_seeded_control_miss`: "`halt_run` stops the run and records a full-run invalidation with this policy as the reason"; Appendix C §C.3: "Under `full_run`, no Result of that run counts as `actionable` for any Quality Gate, whatever its defect class"; `reason` "MUST be present and non-empty when `invalidation_scope` is `full_run`".
- **Method:** Automated
- **Fixture:** F-CLEAN with `semantic-gate`; add `execution: { on_seeded_control_miss: halt_run }` to the Project; scripted Judge `J-ALARM`.
- **Action:** Execute `cts-suite`; evaluate the Gates.
- **Expected:** `s-clean`'s entry has `outcome: false_alarm`, `invalidation_scope: full_run`, and a non-empty `reason`. Both `functional-gate` and `semantic-gate` fail, each with

`seeded_control_false_alarm` in `reasons`.

- **Fails if:** Either Gate passes; either lacks `seeded_control_false_alarm`; or `reason` is empty or `null`.

23. Capability: Action Boundary

Tests in this section apply to an implementation whose Conformance Statement claims `action_boundary` (§10): it runs subjects that act through tools. For any other, they are `NOT_APPLICABLE`.

F-ACT. As F-BASE, with an Environment that gives the subject one tool and declares an enforced boundary:

```
project:
  environments:
    - name: cts-env-act
      model_version: cts-scripted-subject@1
      prompt_version: cts-prompt@1
      infra_target: cts-harness
      tool_configuration:
        tools:
          - name: http_get
            parameters: { url: string }
      action_boundary:
        enforced: true
        entries:
          - target: "https://cts-stub.example/*"
            kind: stub
          - target: "https://cts-trip.example/*"
            kind: stub
            tripwire: true
```

and, in `s-greeting`'s Contract, a Boundary Validator bound as a hard veto:

```
constraints:
  - validator: boundary
    target: artifacts.boundary_attempts
    expected: { attempts: 0, tripwire_touches: 0 }
    veto: true
```

(`artifacts.boundary_attempts` follows Appendix H §H.11; the tester substitutes the implementation's documented path.) `https://cts-outside.example/` is a real endpoint the tester controls and monitors; it is not in the boundary.

Further scripted subject behavior.

Code	Behavior
`SR-CALL(url)`	Requests the tool call `http_get` with argument `url`; after the tool's response, or its refusal, returns `"Done."`.

CT-ABD-001 — A subject with tools needs a declared boundary

- **Capability:** `action_boundary`

- **Requirement:** Volume III, Action Boundary: “An Environment whose `tool_configuration` is not null MUST therefore declare an Action Boundary”; Appendix B §B.3, Action Boundary presence.
- **Method:** Automated
- **Fixture:** F-ACT with `action_boundary` removed.
- **Action:** Load the configuration.
- **Expected:** Rejected.
- **Fails if:** Loaded, or loaded with a boundary supplied silently.

CT-ABD-002 — Tools and boundary entries have the required shape

- **Capability:** `action_boundary`
- **Requirement:** Appendix A §A.2: ToolDef `name` and `parameters`, ActionBoundary `enforced` and `entries`, BoundaryEntry `target` and `kind` are required; Appendix B §B.2: “`tripwire: true` is valid only when `kind` is `stub`.”
- **Method:** Automated
- **Fixture:** Six variants of F-ACT, each with one change: (a) tool `name` removed; (b) tool `parameters` removed; © `enforced` removed; (d) `entries` removed; (e) the first entry’s `kind` removed; (f) the first entry changed to `kind: sandbox` with `tripwire: true`.
- **Action:** Load each variant.
- **Expected:** All six rejected.
- **Fails if:** Any is loaded.

CT-ABD-003 — A call outside the boundary is blocked, recorded, and fails the Execution

- **Capability:** `action_boundary`
- **Requirement:** Volume III, Action Boundary: “Where the harness controls the subject’s access, everything outside the boundary MUST be denied by default”; Volume III, Artifact Collection: it “MUST capture every attempt the subject makes to reach outside it … whether or not the attempt was blocked”; Appendix B §B.3, Boundary attempt capture: such attempts “MUST fail a bound Boundary Validator.”
- **Method:** Automated, Harness
- **Fixture:** F-ACT, run against `cts-env-act`; scripted subject `SR-CALL("https://cts-outside.example/x")`.
- **Action:** Execute `cts-suite`; retrieve the Artifacts and Results; read the log of `cts-outside.example`.
- **Expected:** `cts-outside.example` receives no request. An Artifact records the attempt, its target, and that it was blocked. The Boundary Validator’s Result is `fail`, and the scripted Judge receives no call.
- **Fails if:** The request reaches `cts-outside.example`; the attempt is not recorded; or the Boundary Validator passes.

CT-ABD-004 — Touching a Tripwire is recorded and fails the Execution

- **Capability:** `action_boundary`
- **Requirement:** as CT-ABD-003, for “every touch of a Tripwire.”
- **Method:** Automated, Harness
- **Fixture:** F-ACT; scripted subject `SR-CALL("https://cts-trip.example/login")`.
- **Action:** Execute `cts-suite`; retrieve the Artifacts and Results.
- **Expected:** An Artifact records the Tripwire touch and its target. The Boundary Validator’s Result is `fail`.

- **Fails if:** The touch is not recorded, or the Boundary Validator passes.

CT-ABD-005 — A call to a stub inside the boundary is allowed and has no effect beyond it

- **Capability:** `action_boundary`
- **Requirement:** Volume III, Action Boundary: a `stub` is "a stand-in that only records"; Appendix A §A.2, BoundaryEntry `kind`.
- **Method:** Automated, Harness
- **Fixture:** F-ACT; scripted subject `SR-CALL("https://cts-stub.example/x")`.
- **Action:** Execute `cts-suite`; retrieve the Artifacts and Results; read the log of `cts-outside.example`.
- **Expected:** The call is recorded as made to a stub, no boundary attempt is recorded, the Boundary Validator passes, and `cts-outside.example` receives nothing.
- **Fails if:** The stub call is recorded as a boundary attempt, the Boundary Validator fails, or anything reaches beyond the stub.

CT-ABD-006 — An unenforced boundary is shown in every Report

- **Capability:** `action_boundary`
- **Requirement:** Appendix A §A.2, `enforced`: "A `false` boundary MUST be surfaced in Reports (Volume XI)"; Appendix B §B.3, Unenforced boundary reporting; Appendix C §C.10, `signals`.
- **Method:** Automated
- **Fixture:** F-ACT with `enforced: false`; scripted subject `SR-CALL("https://cts-trip.example/login")`.
- **Action:** Execute `cts-suite`; evaluate the Gates; retrieve the Report produced for the batch.
- **Expected:** `signals.unenforced_boundaries` lists `cts-env-act`, and `signals.boundary_attempts` lists the Tripwire touch with its target.
- **Fails if:** Either is missing from the Report.

CT-ABD-007 — A boundary the harness cannot enforce is declared unenforced

- **Capability:** `action_boundary`
- **Requirement:** Volume III, Action Boundary: where the harness cannot control the subject's access, "the Environment MUST declare the boundary `enforced: false`."
- **Method:** Declaration
- **Fixture:** None.
- **Action:** Ask the implementation to state, for each kind of subject or platform it can run, whether its harness controls the subject's access, and what it does with an Environment that declares `enforced: true` for a subject whose access it does not control.
- **Expected:** Where it does not control access, it rejects or flags `enforced: true`, and never presents such a boundary as enforced.
- **Fails if:** The statement admits presenting an uncontrolled boundary as enforced.

24. Capability: Dataset Sources

Tests in this section apply to an implementation whose Conformance Statement claims `dataset:<source>` for the source named in each test (§10). For a source it does not claim, the test is `NOT_APPLICABLE`. Static Datasets are core (Sections 10 and 11).

CT-DSR-001 — A Generated Dataset declares how it is generated

- **Capability:** `dataset:generated`
- **Requirement:** Appendix A §A.8: `generator` is required when `source` is `generated`; GeneratorConfig `model` and `instruction` are required.
- **Method:** Automated
- **Fixture:** Three variants of a Dataset `ds-gen` (version `1.0.0`, `source: generated`) in the Project's `datasets`: (a) no `generator`; (b) a `generator` with no `model`; © a `generator` with no `instruction`.
- **Action:** Load each variant.
- **Expected:** All three rejected.
- **Fails if:** Any is loaded.

CT-DSR-002 — A Production Replay Dataset declares its capture

- **Capability:** `dataset:production_replay`
- **Requirement:** Appendix A §A.8: `capture` is required when `source` is `production_replay`; CaptureConfig `source`, `date_range`, `sampling`, `sample_size`, and `anonymization` are required.
- **Method:** Automated
- **Fixture:** Six variants of a Dataset `ds-replay` (version `1.0.0`, `source: production_replay`): (a) no `capture`; (b)–(f) a complete `capture` with one of its five fields removed.
- **Action:** Load each variant.
- **Expected:** All six rejected.
- **Fails if:** Any is loaded.

CT-DSR-003 — Generating rows again makes a new version, never changes an old one

- **Capability:** `dataset:generated`
- **Requirement:** Volume IV, Versioning: "Every Dataset, regardless of source, MUST be versioned"; Appendix B §B.3, Dataset version immutability.
- **Method:** Automated, Harness
- **Fixture:** A complete `ds-gen@1.0.0` whose `generator.model` resolves to a scripted generator returning one row, `query: "Gen-1"`, and on any later call one row, `query: "Gen-2"`. `s-greeting` uses `dataset: ds-gen@1.0.0` instead of `variables`; scripted subject `SR-ECHO`.
- **Action:** Generate `ds-gen@1.0.0`; execute `cts-suite`. Ask the implementation to generate `ds-gen` again, by the operation it documents for that (through the REST API, `POST .../datasets/ds-gen/versions`, where that triggers generation); retrieve `ds-gen@1.0.0` and every version the request produced; execute `cts-suite` again. If the implementation documents no way to generate a Dataset again, which the specification does not require, the outcome is `NOT_APPLICABLE` (1.2).

- **Expected:** Both Executions send the Prompt `Gen-1`. The second generation, if accepted, produced a new version; the rows of `ds-gen@1.0.0` are unchanged.
- **Fails if:** The second Execution sends `Gen-2`, or `ds-gen@1.0.0`'s rows changed.

CT-DSR-004 — A Production Replay Dataset supplies its own Prompts

- **Capability:** `dataset:production_replay`
- **Requirement:** Appendix B §B.3, Prompt presence: a Scenario "MUST carry `prompt` unless its `dataset` is a `production_replay` Dataset"; Appendix A §A.4.
- **Method:** Automated
- **Fixture:** A complete `ds-replay@1.0.0` holding one captured Conversation whose user turn is `"Hello"`. `s-greeting` with `dataset: ds-replay@1.0.0` and no `prompt` or `variables`.
- **Action:** Load the configuration; execute `cts-suite`.
- **Expected:** Loaded. The scripted subject receives the captured Prompt, `Hello`.
- **Fails if:** The configuration is rejected for the missing `prompt`, or the subject receives anything else.

25. Capability: Plugins

Tests in this section apply to an implementation whose Conformance Statement claims `plugins:<type>` for the type named in each test (§10). For a type it does not claim, the test is `NOT_APPLICABLE`. The Custom Validators of Section 2.5 are registered here as Validator Plugins.

Further plugin. `cts-probe`: a Validator Plugin that records everything it receives in the harness log and returns `pass`; when its configuration has `malformed: true`, it returns a Result with no `verdict` field at all.

CT-PLG-001 — A Validator Plugin whose output varies is not registered

- **Capability:** `plugins:validator`
- **Requirement:** Volume XVI, Validator Plugins: registration “MUST run a candidate Validator Plugin against a fixed test input more than once and reject registration if its output varies.”
- **Method:** Automated, Harness
- **Fixture:** None beyond Section 2.5.
- **Action:** Register `cts-flaky` as a Validator Plugin; then register `cts-len`.
- **Expected:** `cts-flaky` is rejected, the harness log showing it was called more than once on the same input. `cts-len` is registered.
- **Fails if:** `cts-flaky` is registered, or `cts-len` is rejected.

CT-PLG-002 — Registration is not reported as proof of determinism

- **Capability:** `plugins:validator`
- **Requirement:** Volume XVI, Validator Plugins: “A successful registration MUST NOT be reported as proof of determinism.”
- **Method:** Inspection
- **Fixture:** None beyond Section 2.5.
- **Action:** Register `cts-len` as a Validator Plugin. Read what the implementation reports for the registration, in its response, records, and documentation.
- **Expected:** It reports a repeatability check that passed, not that the plugin was shown to be deterministic.
- **Fails if:** Anything presents the passed check as proof of determinism.

CT-PLG-003 — A plugin receives the logical inputs, and a malformed Result assesses nothing

- **Capability:** `plugins:validator`
- **Requirement:** Volume XVI, Plugin Invocation: “What every implementation MUST hold to is the logical shape of one assessment”: the Evidence its clause names, the clause’s content, its versioned configuration, and the Execution and clause identifiers, and “A Validator Plugin receives nothing else”; a Result that fails validation is `oracle_unavailable`, “never `fail`.”
- **Method:** Automated, Harness
- **Fixture:** Two variants of F-BASE, each with one Constraint bound to `cts-probe`, `target: response`, `dimension: functional`. (a) `expected: { marker: 1 }`. (b) `expected: { marker: 2 }` and configuration `malformed: true`.

- **Action:** In each, execute `cts-suite`; read what `cts-probe` received; retrieve the `cts-probe` Result.
- **Expected:** (a) `cts-probe` received the Response, its clause's `expected`, its configuration, and the Execution and clause identifiers, and nothing else. Its Result is `pass`, with `oracle` naming `cts-probe` and its version. (b) The `cts-probe` Result is `oracle_unavailable`, with `verdict: null`, and the scripted Judge receives no call. (The two cases run separately because the specification does not fix the order in which a Contract's Validators run, and a Validation that has not passed stops there.)
- **Fails if:** In (a), `cts-probe` received anything beyond those inputs, or the Result does not name the plugin and version; in (b), the Result is recorded as `fail` or with the malformed content, or the Judge is called.

CT-PLG-004 — Deprecating a plugin keeps what it produced

- **Capability:** `plugins:validator`
- **Requirement:** Volume XVI, Extension Lifecycle: "Deprecating a plugin MUST NOT silently invalidate Baselines or Results a retired version already produced."
- **Method:** Automated
- **Fixture:** F-BASE with a Constraint bound to the registered `cts-len`.
- **Action:** Execute `cts-suite`; designate the Execution as a Baseline (as `P-APPROVER`, Section 14.1). Deprecate `cts-len`'s version. Retrieve the Results and the Baseline.
- **Expected:** Both are retrievable and unchanged; the `cts-len` Result still names the deprecated version in `oracle`.
- **Fails if:** Either is lost, altered, or marked invalid.

CT-PLG-005 — A Custom Validator's determinism and lack of Confidence are enforced, not documented

- **Capability:** `plugins:validator`
- **Requirement:** Volume XVI: "a Custom Validator's determinism and absence of a Confidence value MUST be enforced at the SDK level, not merely documented as a convention"; Volume V, Custom Validators: "the same input MUST always produce the same Result, and it MUST NOT carry a Confidence value." Volume V, Validator SDK, adds that the SDK "SHOULD NOT give it a clock, a random source, or network access": a SHOULD, which this test reports on but does not fail.
- **Method:** Declaration, Automated, Harness
- **Fixture:** F-BASE with a Constraint bound to `cts-reach`, `target: response`, `dimension: functional`. `cts-reach` is a Validator Plugin that tries, in order, to read the clock, draw a random number, and open a network connection to the harness; records in its Result's evidence which attempts succeeded; returns `pass` if the random number it drew (or, where randomness is withheld, the clock reading in milliseconds) is even, `fail` if it is odd, and `pass` if neither is available; and adds a Confidence value of `0.5` to its Result.
- **Action:** Ask the implementation to state how its Validator SDK enforces determinism and the absence of Confidence. Register `cts-reach`. If it is registered, execute `cts-suite` ten times and retrieve the `cts-reach` Results; read the harness log.
- **Expected:**
  - The statement describes enforcement by the SDK or the Plugin Architecture, not only documentation or a convention for plugin authors.
  - Either registration is refused, or every `cts-reach` Result carries no Confidence value (`confidence: "not_applicable"`, or the Result is `oracle_unavailable` because the returned Confidence made it malformed), and the ten Results have the same `verdict` or are all `oracle_unavailable`.

    - Ten equal verdicts are a sampled consistency check, not a proof of determinism: the report labels them so, and rests any claim of enforcement on the statement.
    - Reported, not judged: which of the clock, randomness, and network `cts-reach` reached, from its evidence and the harness log; any of them reached is a deviation from Volume V's SHOULD, which the report names.
- **Fails if:** The statement describes no enforcement beyond documentation; a `cts-reach` Result carries a numeric Confidence; or the ten Results differ in `verdict`.

CT-PLG-006 — A Dataset Plugin's output is versioned

- **Capability:** `plugins:dataset`
- **Requirement:** Volume XVI, Dataset Plugins: "every Dataset a plugin produces MUST be versioned, the same as one from a built-in source."
- **Method:** Automated
- **Fixture:** A Dataset Plugin the tester registers that produces one row, `query: "Plugin-1"`.
- **Action:** Produce a Dataset with it; retrieve the Dataset.
- **Expected:** The Dataset has a version, and a Scenario can reference it as `name@version`.
- **Fails if:** The Dataset has no version.

CT-PLG-007 — A Report Plugin's Metric traces back to its Results

- **Capability:** `plugins:report`
- **Requirement:** Volume XVI, Report Plugins: "a Metric a Report Plugin introduces MUST be traceable back to the Results it was computed from."
- **Method:** Inspection
- **Fixture:** A Report Plugin the tester registers that introduces a Metric: the share of Results with Confidence below 0.7.
- **Action:** Build F-SIGNALS (2.7); generate a Report for its batch with the plugin; follow the Metric to the Results behind it.
- **Expected:** The reviewer reaches the individual Results from the Metric.
- **Fails if:** The Metric cannot be traced to its Results.

26. Capabilities: SDKs and CLI, Reference UI, Test Types

Each test applies only where the implementation claims the capability it names (§10).

CT-SDK-001 — A CLI command is a REST API call

- **Capability:** `cli`
- **Requirement:** Volume XIV, CLI: “A CLI command MUST correspond to a REST API call it makes on the caller’s behalf, not a separate implementation of the operation it names”; Appendix C, introduction: the CLI and the SDKs “are thin wrappers over this API and MUST NOT independently reimplement logic” (the CLI’s half of that requirement; CT-SDK-002 checks the SDKs’).
- **Method:** Harness, Inspection
- **Fixture:** F-BASE. The tester routes the CLI’s traffic through a recording proxy.
- **Action:** With the CLI, load F-BASE, execute `cts-suite`, and evaluate the Gates. Then list the CLI’s documented commands and the REST operations of Appendix C.
- **Expected:** Each of the three commands appears in the proxy log as the matching Appendix C operation, and every documented CLI command maps to a REST operation.
- **Fails if:** A command acts without the matching REST call, or a command has no REST counterpart.

CT-SDK-002 — An SDK behaves as the REST API does

- **Capability:** `sdk:<language>`
- **Requirement:** Appendix C, introduction: the SDKs “are thin wrappers over this API and MUST NOT independently reimplement logic” (the SDKs’ half of that requirement; CT-SDK-001 checks the CLI’s).
- **Method:** Automated
- **Fixture:** The configurations of CT-CFG-002, CT-DEC-006, and CT-REL-002.
- **Action:** Carry out each of those three tests once through the REST API and once through the SDK of each claimed language, each run in a Project of its own (1.6).
- **Expected:** Each test reaches the same outcome on every surface. Where the REST API returns an error, the SDK reports the same error in substance: the same HTTP status or error code, and the same field or entity named, however the SDK represents it, as an exception or otherwise. The Results and GateResults recorded agree in every field but identifiers, `execution`, references to their own Execution, and times. Message text, exception classes, and stack traces are not compared.
- **Fails if:** An outcome differs; an error differs in status, code, or what it names; an SDK succeeds where the API returns an error, or the reverse; or a compared field of a recorded Result or GateResult differs.

CT-RUI-001 — A Scenario authored visually is the same as one written in YAML

- **Capability:** `reference_ui`
- **Requirement:** Volume XV, Visual Test Designer: “a Scenario authored visually and one authored by hand in YAML MUST be indistinguishable once saved.”
- **Method:** Automated
- **Fixture:** F-BASE’s `s-greeting`.
- **Action:** Author `s-greeting` in the Visual Test Designer, as `s-visual`, with the same content. Retrieve both Scenarios through the REST API.

- **Expected:** Apart from their names, the two are identical.
- **Fails if:** They differ in any other field or in structure.

CT-RUI-002 — The Validator Designer does not let a Validator carry Confidence or call a model

- **Capability:** `reference_ui`
- **Requirement:** Volume XV, Validator Designer: “a Validator configured here MUST be deterministic and MUST NOT carry a Confidence value”; a check that calls a model “is configuring a Judge … and the Validator Designer MUST surface that distinction.”
- **Method:** Inspection
- **Fixture:** None.
- **Action:** In the Validator Designer, try to give a Validator a Confidence value or threshold, and try to configure a Validator whose check calls a model.
- **Expected:** The first cannot be done. The second is refused or redirected to Judge configuration, with the reason shown.
- **Fails if:** Either is saved as a Validator.

CT-RUI-003 — The UI asks which purpose a Multi Judge panel serves

- **Capability:** `reference_ui` together with `multi_judge`
- **Requirement:** Volume XV, Judge Configuration: it “MUST require an explicit choice of which purpose it serves — independence or stability — rather than default silently to one.”
- **Method:** Inspection
- **Fixture:** None.
- **Action:** Configure a second Judge on an Expectation through the UI without choosing a purpose, and try to save.
- **Expected:** The UI does not save until a purpose is chosen.
- **Fails if:** It saves with a purpose it chose itself.

CT-RUI-004 — A Report and a Dashboard cannot be mistaken for each other

- **Capability:** `reference_ui`
- **Requirement:** Volume XV, Reporting UI: it “MUST visually distinguish a Report’s frozen, point-in-time state from a Dashboard’s live, continuously-updating one.”
- **Method:** Inspection
- **Fixture:** F-SIGNALS (2.7), and a Dashboard over the same Suite.
- **Action:** Open the Report and the Dashboard in the UI.
- **Expected:** Each states what it is; the Report shows the moment it was frozen.
- **Fails if:** A viewer could take either for the other from what the UI shows.

CT-TTP-001 — A Regression Suite surfaces drift with nothing AQEF-visible changed

- **Capability:** `test_type:regression`
- **Requirement:** Volume X, Model Drift: “a Regression Suite (Volume IX) MUST be able to surface Model Drift even when nothing AQEF-visible changed.”

- **Method:** Automated
- **Fixture:** F-BASE with `semantic-gate`, plus a Regression Gate:

  ```
  - name: regression-gate
    aggregate: regression
    compare_against: latest_approved
    threshold: 0
    on_failure: block_release
  ```

  Scripted Judge `J-CATCH` (Section 13.1).
- **Action:** Execute `cts-suite` with scripted subject `SR-OK`; as `P-APPROVER`, designate the Execution as a Baseline. Without changing any configuration, switch the scripted subject to `SR-BYE` (Section 22); execute again; evaluate the Gates.
- **Expected:** `regression-gate` fails, and the implementation's records show that no Environment, Contract, or Dataset changed between the two Executions.
- **Fails if:** `regression-gate` passes.

CT-TTP-002 — An agent's Baseline comparison judges the trajectory as a whole

- **Capability:** `test_type:agent` together with `action_boundary`
- **Requirement:** Volume I, Chapter 7, AI Agent: a Baseline comparison "MUST therefore be capable of judging the outcome and the reasoning quality of a trajectory as a whole, not require step-for-step identity with the Baseline's own path."
- **Method:** Automated, Inspection
- **Fixture:** F-ACT (Section 23) with a second stub entry `https://cts-stub2.example/*` and a Regression Gate as in CT-TTP-001. Two scripted trajectories reaching the same final Response, `"Done."`: (a) calls `cts-stub.example`, then `cts-stub2.example`; (b) the same two calls in the opposite order.
- **Action:** Execute with trajectory (a); designate it as a Baseline. Execute with trajectory (b); evaluate the Gates. Review how the comparison was made.
- **Expected:** The comparison does not report a regression because the order of steps differs; the reviewer records what the comparison judged.
- **Fails if:** A regression is reported solely because (b)'s steps are not identical to (a)'s.

90. Traceability

This section maps every requirement in the MUST inventory of AQEF Working Draft 0.41.4 to the tests that check it, or says why no test does. The inventory is `_inventory/must-inventory.md` (391 rows, extracted from Working Draft 0.36) together with `_inventory/must-inventory-v037-additions.md` (28 rows added by 0.37), `_inventory/must-inventory-v038-additions.md` (12 rows added by 0.38), `_inventory/must-inventory-v041-additions.md` (10 rows added by 0.40 to 0.41.1), and `_inventory/must-inventory-v0414-additions.md` (10 rows added by 0.41.2 to 0.41.4), 451 rows in all, identified `R-nnnn`. It lists every sentence containing MUST or MUST NOT, and every field Appendix A or C marks as required, so a few rows are definitions or restatements rather than requirements of their own. Twenty rows whose text 0.37 to 0.41.4 revised keep their IDs and are marked.

90.1 Status values

Status	Meaning
Tested	Checked by the tests named, or by the cases named where a test has cases (1.2).
Restates	Says again what another row says; it is covered wherever that row is. Mostly the Glossary (Appendix D), Best Practices (Appendix I), and reference examples.
Field	A field Appendix A or C marks as required, whose absence no test checks separately. None in this draft: CT-CFG-025 checks each required configuration field of Appendix A, one case per field, and the tests named check the fields of runtime records.
Scoped	Belongs to a capability a Conformance Statement may claim (§10) and has no test in this draft.
Not testable	A definition, an introduction to a list, or a statement about people or practice rather than about an implementation.
Not yet covered	A requirement on every implementation that this draft has no test for. Listed again in 90.4 when there is any.

90.2 Summary

Status	Rows
Tested	376
Restates	46
Field	0
Scoped	0
Not testable	29
Not yet covered	0
Total	**451**

90.3 Requirement → test

ID	Source	Status	Tests or note
R-0001	§0, Authorship and Consensus	Not testable	scope of normative language
R-0002	Revision history	Restates	R-0092 (CT-SCT-005)
R-0003	Revision history	Restates	R-0090 (CT-SCT-003, CT-SCT-004)
R-0004	§8	Restates	R-0080
R-0005	§8	Tested	CT-ASM-012
R-0006	§8	Tested	CT-ASM-003, CT-ASM-009
R-0007	§8	Tested	CT-ASM-004
R-0008	§9	Not testable	definition
R-0009	§9	Not testable	definition
R-0010	§9	Not testable	definition
R-0011	§9, Why Not the Full RFC 2119 Vocabulary	Not testable	definition
R-0012	§9, Why Not the Full RFC 2119 Vocabulary	Not testable	editorial
R-0013	§9, Scope of Normative Force	Not testable	definition
R-0014	§10, What Conformance Requires	Not testable	definition of Conformance
R-0015	§10, What Conformance Requires	Tested	CT-REL-024
R-0016	§10, Conformance Statement	Tested	CT-REL-027; text revised in v0.41.2 or later
R-0017	§10, The Non-Negotiable Core	Tested	CT-REL-026
R-0018	§10, The Non-Negotiable Core	Tested	CT-REL-026

ID	Source	Status	Tests or note
R-0019	§10, Deviating from a SHOULD	Not testable	editorial
R-0020	Vol I, Evolution of Software Testing	Not testable	background
R-0021	Vol I, Validation before Evaluation	Tested	CT-ASM-001, CT-ASM-002
R-0022	Vol I, Independent Oracles	Tested	CT-CFG-006
R-0023	Vol I, Deterministic Infrastructure	Tested	CT-EXE-007, CT-EXE-008
R-0024	Vol I, Deterministic Infrastructure	Tested	CT-EXE-007
R-0025	Vol I, Everything has Confidence	Tested	CT-ASM-003
R-0026	Vol I, Everything has Confidence	Tested	CT-ASM-003
R-0027	Vol I, Everything has Confidence	Tested	CT-ASM-003
R-0028	Vol I, Requirements	Not testable	applies to Requirements people write
R-0029	Vol I, Execution	Tested	CT-EXE-007
R-0030	Vol I, Validation	Tested	CT-ASM-002
R-0031	Vol I, Decision Pipeline	Tested	CT-DEC-006
R-0032	Vol I, Confidence Model	Tested	CT-ASM-005, CT-HRV-004
R-0033	Vol I, Aggregation Model	Not testable	introduces R-0034 and R-0035
R-0034	Vol I, Aggregation Model	Tested	CT-DEC-020
R-0035	Vol I, Aggregation Model	Tested	CT-DEC-019
R-0036	Vol I, Aggregate Basis	Tested	CT-DEC-002
R-0037	Vol I, Aggregate Basis	Tested	CT-DEC-002

ID	Source	Status	Tests or note
R-0038	Vol I, Aggregate Basis	Tested	CT-DEC-021
R-0039	Vol I, Quality Gates	Tested	CT-REL-001, CT-REL-003
R-0040	Vol I, Quality Gates	Tested	CT-DEC-015, CT-DEC-017, CT-SCT-011, CT-SCT-015
R-0041	Vol I, Quality Gates	Tested	CT-DEC-004
R-0042	Vol I, Quality Gates	Tested	CT-REL-001, CT-REL-002
R-0043	Vol I, Quality Gates	Tested	CT-REL-002, CT-REL-006
R-0044	Vol I, Quality Gates	Tested	CT-REL-007
R-0045	Vol I, Quality Gates	Tested	CT-DEC-006, CT-DEC-007, CT-DEC-011
R-0046	Vol I, Quality Gates	Tested	CT-REL-010
R-0047	Vol I, AI Agent	Tested	CT-TTP-002
R-0048	Vol II, Entities	Tested	CT-EXE-001
R-0049	Vol III, Runtime	Tested	CT-EXE-007
R-0050	Vol III, Action Boundary	Tested	CT-ABD-001, CT-ABD-005
R-0051	Vol III, Action Boundary	Tested	CT-ABD-003
R-0052	Vol III, Action Boundary	Tested	CT-ABD-007
R-0053	Vol III, Scheduling	Tested	CT-EXE-009
R-0054	Vol III, Parallelization	Tested	CT-EXE-006
R-0055	Vol III, Retry	Tested	CT-EXE-003
R-0056	Vol III, Retry	Tested	CT-EXE-004, CT-EXE-005

ID	Source	Status	Tests or note
R-0057	Vol III, Variables	Tested	CT-EXE-002
R-0058	Vol III, Conversation Engine	Tested	CT-EXE-011
R-0059	Vol III, Artifact Collection	Tested	CT-EXE-012
R-0060	Vol III, Artifact Collection	Tested	CT-ABD-003, CT-ABD-004
R-0061	Vol IV, Generated Datasets	Not testable	explains why Volume IV uses SHOULD
R-0062	Vol IV, Production Replay	Not testable	defers to Project policy
R-0063	Vol IV, Versioning	Tested	CT-CFG-017, CT-EXE-010, CT-DSR-003
R-0064	Vol V, Validator Architecture	Tested	CT-ASM-009
R-0065	Vol V, Validator Architecture	Tested	CT-ASM-010
R-0066	Vol V, Validator Architecture	Tested	CT-ASM-003
R-0067	Vol V, Execution Order	Tested	CT-ASM-001
R-0068	Vol V, Custom Validators	Tested	CT-ASM-016
R-0069	Vol V, Custom Validators	Tested	CT-ASM-017
R-0070	Vol V, Validator SDK	Tested	CT-ASM-016
R-0071	Vol V, Validator SDK	Tested	CT-ASM-018
R-0072	Vol VI, Judge Architecture	Tested	CT-ASM-004
R-0073	Vol VI, Judge Architecture	Tested	CT-ASM-011
R-0074	Vol VI, Single Judge	Tested	CT-ASM-019; text revised in v0.37 or later
R-0075	Vol VI, Multi Judge	Tested	CT-MJG-001

ID	Source	Status	Tests or note
R-0076	Vol VI, Consensus	Tested	CT-ASM-013, CT-MJG-005, CT-MJG-006
R-0077	Vol VI, Confidence	Tested	CT-MJG-007
R-0078	Vol VI, Human Review	Tested	CT-HRV-003, CT-HRV-005; text revised in v0.37 or later
R-0079	Vol VI, Human Review	Tested	CT-ASM-008, CT-ASM-015, CT-DEC-013
R-0080	Vol VII	Not testable	definition of Quality Contract
R-0081	Vol VII, Contract Language	Not testable	property of the Contract language
R-0082	Vol VII, Constraints	Tested	CT-CFG-019
R-0083	Vol VII, Expectations	Tested	CT-ASM-011
R-0084	Vol VII, Contract Composition	Tested	CT-CFG-007
R-0085	Vol VIII, Test Design Process	Not testable	applies to Test Design, a human activity
R-0086	Vol VIII, AI Test Patterns	Not testable	describes a test pattern
R-0087	Vol VIII, Seeded Controls	Tested	CT-SCT-007
R-0088	Vol VIII, Seeded Controls	Tested	CT-DEC-016, CT-SCT-010
R-0089	Vol VIII, Seeded Controls	Tested	CT-SCT-001; text revised in v0.40 or later
R-0090	Vol VIII, Seeded Controls	Tested	CT-SCT-003, CT-SCT-004
R-0091	Vol VIII, Seeded Controls	Tested	CT-SCT-006
R-0092	Vol VIII, Seeded Controls	Tested	CT-SCT-005
R-0093	Vol VIII, Test Maintainability	Not testable	describes practice
R-0094	Vol X, Baselines	Tested	CT-REL-021

ID	Source	Status	Tests or note
R-0095	Vol X, Model Drift	Tested	CT-TTP-001
R-0096	Vol X, Dataset Drift	Restates	R-0063 (CT-CFG-017, CT-EXE-010, CT-DSR-003)
R-0097	Vol XI, Reports	Tested	CT-REL-035
R-0098	Vol XI, Quality Gates	Tested	CT-REL-003, CT-REL-029
R-0099	Vol XI, Quality Gates	Tested	CT-REL-029
R-0100	Vol XI, Quality Gates	Tested	CT-REL-029
R-0101	Vol XI, KPIs	Tested	CT-REL-036
R-0102	Vol XII, Permissions	Tested	CT-REL-032
R-0103	Vol XII, Principals	Tested	CT-REL-019
R-0104	Vol XII, Principals	Tested	CT-REL-033
R-0105	Vol XII, Principals	Tested	CT-REL-017
R-0106	Vol XII, Principals	Tested	CT-REL-020
R-0107	Vol XII, Principals	Tested	CT-REL-018
R-0108	Vol XII, Traceability	Tested	CT-REL-022
R-0109	Vol XII, Compliance	Not testable	scope of Compliance
R-0110	Vol XII, Approval Workflow	Not testable	describes Approval Workflow; its four cases are tested separately
R-0111	Vol XII, Approval Workflow	Tested	CT-REL-007
R-0112	Vol XIII, Azure DevOps	Tested	CT-REL-037
R-0113	Vol XIII, Quality Gates	Tested	CT-REL-024

ID	Source	Status	Tests or note
R-0114	Vol XIII, Quality Gates	Tested	CT-REL-025
R-0115	Vol XIV, CLI	Tested	CT-SDK-001
R-0116	Vol XV, Visual Test Designer	Tested	CT-RUI-001
R-0117	Vol XV, Validator Designer	Tested	CT-RUI-002
R-0118	Vol XV, Validator Designer	Tested	CT-RUI-002
R-0119	Vol XV, Judge Configuration	Tested	CT-RUI-003
R-0120	Vol XV, Reporting UI	Tested	CT-RUI-004
R-0121	Vol XVI	Tested	CT-PLG-005
R-0122	Vol XVI, Validator Plugins	Tested	CT-PLG-001
R-0123	Vol XVI, Validator Plugins	Tested	CT-PLG-002
R-0124	Vol XVI, Dataset Plugins	Tested	CT-PLG-006
R-0125	Vol XVI, Report Plugins	Tested	CT-PLG-007
R-0126	Vol XVI, Plugin Invocation	Tested	CT-PLG-003
R-0127	Vol XVI, Extension Lifecycle	Tested	CT-PLG-004
R-0128	App A	Tested	CT-CFG-001
R-0129	App A, A.1 — Project	Tested	CT-CFG-025.01
R-0130	App A, A.1 — Project	Tested	CT-CFG-020
R-0131	App A, A.1 — Project	Restates	R-0130 (CT-CFG-020)
R-0132	App A, A.1 — Project	Tested	CT-CFG-025.25; the source line is DefectClassConfig `name`, which the inventory labels Project.name

ID	Source	Status	Tests or note
R-0133	App A, A.1 — Project	Tested	CT-SCT-005
R-0134	App A, A.1 — Project	Tested	CT-SCT-005
R-0135	App A, A.1 — Project	Tested	CT-SCT-005
R-0136	App A, A.1 — Project	Tested	CT-SCT-006
R-0137	App A, A.2 — Environment	Tested	CT-CFG-025.02
R-0138	App A, A.2 — Environment	Tested	CT-CFG-025.03
R-0139	App A, A.2 — Environment	Tested	CT-CFG-025.04
R-0140	App A, A.2 — Environment	Tested	CT-CFG-025.05
R-0141	App A, A.2 — Environment	Tested	CT-ABD-001
R-0142	App A, A.2 — Environment	Tested	CT-CFG-025.06
R-0143	App A, A.2 — Environment	Tested	CT-ABD-002
R-0144	App A, A.2 — Environment	Tested	CT-ABD-002
R-0145	App A, A.2 — Environment	Tested	CT-ABD-002
R-0146	App A, A.2 — Environment	Tested	CT-ABD-002
R-0147	App A, A.2 — Environment	Tested	CT-ABD-006
R-0148	App A, A.2 — Environment	Tested	CT-ABD-002
R-0149	App A, A.2 — Environment	Tested	CT-ABD-002
R-0150	App A, A.2 — Environment	Tested	CT-ABD-002
R-0151	App A, A.3 — Suite	Tested	CT-CFG-025.07

ID	Source	Status	Tests or note
R-0152	App A, A.3 — Suite	Tested	CT-CFG-021
R-0153	App A, A.3 — Suite	Restates	R-0152 (CT-CFG-021)
R-0154	App A, A.3 — Suite	Tested	CT-CFG-009
R-0155	App A, A.4 — Scenario	Tested	CT-CFG-025.08
R-0156	App A, A.4 — Scenario	Tested	CT-CFG-002
R-0157	App A, A.4 — Scenario	Tested	CT-CFG-002
R-0158	App A, A.4 — Scenario	Tested	CT-CFG-022, CT-DSR-004
R-0159	App A, A.4 — Scenario	Not testable	states what is permitted; the prohibition is R-0160
R-0160	App A, A.4 — Scenario	Tested	CT-CFG-011
R-0161	App A, A.4 — Scenario	Tested	CT-CFG-022
R-0162	App A, A.4 — Scenario	Tested	CT-EXE-002
R-0163	App A, A.4 — Scenario	Tested	CT-CFG-008
R-0164	App A, A.4 — Scenario	Tested	CT-SCT-001
R-0165	App A, A.4 — Scenario	Tested	CT-SCT-005
R-0166	App A, A.4 — Scenario	Tested	CT-SCT-001, CT-SCT-009; text revised in v0.40 or later
R-0167	App A, A.4 — Scenario	Tested	CT-SCT-001
R-0168	App A, A.4 — Scenario	Tested	CT-SCT-002
R-0169	App A, A.4 — Scenario	Tested	CT-SCT-002
R-0170	App A, A.4 — Scenario	Tested	CT-SCT-002

ID	Source	Status	Tests or note
R-0171	App A, A.5 — Quality Contract	Restates	R-0080
R-0172	App A, A.5 — Quality Contract	Tested	CT-CFG-023
R-0173	App A, A.5 — Quality Contract	Tested	CT-CFG-003
R-0174	App A, A.5 — Quality Contract	Tested	CT-CFG-019
R-0175	App A, A.5 — Quality Contract	Tested	CT-CFG-025.09
R-0176	App A, A.5 — Quality Contract	Tested	CT-CFG-025.10
R-0177	App A, A.5 — Quality Contract	Tested	CT-CFG-025.11
R-0178	App A, A.5 — Quality Contract	Tested	CT-CFG-004
R-0179	App A, A.5 — Quality Contract	Tested	CT-CFG-004
R-0180	App A, A.5 — Quality Contract	Tested	CT-CFG-004
R-0181	App A, A.5 — Quality Contract	Tested	CT-CFG-005
R-0182	App A, A.5 — Quality Contract	Tested	CT-CFG-004
R-0183	App A, A.5 — Quality Contract	Tested	CT-HRV-001
R-0184	App A, A.5 — Quality Contract	Tested	CT-HRV-002
R-0185	App A, A.6 — JudgeConfig	Tested	CT-CFG-025.12
R-0186	App A, A.6 — JudgeConfig	Tested	CT-CFG-006
R-0187	App A, A.6 — JudgeConfig	Tested	CT-CFG-025.13
R-0188	App A, A.7 — MultiJudgeConfig	Tested	CT-MJG-001
R-0189	App A, A.7 — MultiJudgeConfig	Tested	CT-MJG-001

ID	Source	Status	Tests or note
R-0190	App A, A.7 — MultiJudgeConfig	Tested	CT-MJG-002
R-0191	App A, A.7 — MultiJudgeConfig	Tested	CT-MJG-002, CT-MJG-006
R-0192	App A, A.7 — MultiJudgeConfig	Tested	CT-MJG-002, CT-MJG-006
R-0193	App A, A.7 — MultiJudgeConfig	Tested	CT-MJG-003
R-0194	App A, A.7 — MultiJudgeConfig	Tested	CT-MJG-003
R-0195	App A, A.8 — Dataset	Tested	CT-CFG-025.19
R-0196	App A, A.8 — Dataset	Tested	CT-CFG-017
R-0197	App A, A.8 — Dataset	Restates	R-0063 (CT-CFG-017, CT-EXE-010, CT-DSR-003)
R-0198	App A, A.8 — Dataset	Tested	CT-CFG-025.20
R-0199	App A, A.8 — Dataset	Tested	CT-CFG-025.21
R-0200	App A, A.8 — Dataset	Tested	CT-DSR-001
R-0201	App A, A.8 — Dataset	Tested	CT-DSR-002
R-0202	App A, A.8 — Dataset	Tested	CT-CFG-025.22
R-0203	App A, A.8 — Dataset	Tested	CT-CFG-025.23
R-0204	App A, A.8 — Dataset	Tested	CT-DSR-001
R-0205	App A, A.8 — Dataset	Tested	CT-DSR-001
R-0206	App A, A.8 — Dataset	Tested	CT-DSR-002
R-0207	App A, A.8 — Dataset	Tested	CT-DSR-002
R-0208	App A, A.8 — Dataset	Tested	CT-DSR-002

ID	Source	Status	Tests or note
R-0209	App A, A.8 — Dataset	Tested	CT-DSR-002
R-0210	App A, A.8 — Dataset	Tested	CT-DSR-002
R-0211	App A, A.8 — Dataset	Tested	CT-CFG-025.24
R-0212	App A, A.9 — ExecutionConfig	Tested	CT-EXE-006
R-0213	App A, A.9 — ExecutionConfig	Tested	CT-CFG-025.18
R-0214	App A, A.9 — ExecutionConfig	Tested	CT-CFG-010
R-0215	App A, A.9 — ExecutionConfig	Tested	CT-CFG-010
R-0216	App A, A.10 — Quality Gates	Tested	CT-CFG-025.14
R-0217	App A, A.10 — Quality Gates	Tested	CT-CFG-025.15
R-0218	App A, A.10 — Quality Gates	Tested	CT-CFG-025.16
R-0219	App A, A.10 — Quality Gates	Tested	CT-CFG-025.17
R-0220	App A, A.10 — Quality Gates	Tested	CT-REL-002, CT-REL-003
R-0221	App A, A.10 — Quality Gates	Tested	CT-CFG-024
R-0222	App A, A.11 — Baseline	Tested	CT-REL-021
R-0223	App A, A.11 — Baseline	Tested	CT-REL-021
R-0224	App A, A.11 — Baseline	Tested	CT-REL-021
R-0225	App A, A.11 — Baseline	Tested	CT-REL-021
R-0226	App A, A.11 — Baseline	Tested	CT-REL-017, CT-REL-021
R-0227	App A, A.11 — Baseline	Tested	CT-REL-021

ID	Source	Status	Tests or note
R-0228	App A, A.11 — Baseline	Tested	CT-REL-021
R-0229	App A, A.11 — Baseline	Tested	CT-REL-021
R-0230	App A, A.12 — Release Decision	Tested	CT-REL-001
R-0231	App A, A.12 — Release Decision	Tested	CT-REL-001
R-0232	App A, A.12 — Release Decision	Tested	CT-REL-001
R-0233	App A, A.12 — Release Decision	Tested	CT-REL-001
R-0234	App A, A.12 — Release Decision	Tested	CT-REL-001
R-0235	App A, A.12 — Release Decision	Tested	CT-REL-001
R-0236	App A, A.12 — Release Decision	Tested	CT-REL-001
R-0237	App A, A.12 — Release Decision	Tested	CT-REL-001, CT-REL-018
R-0238	App A, A.12 — Release Decision	Tested	CT-REL-001
R-0239	App A, A.12 — Release Decision	Tested	CT-REL-001
R-0240	App A, A.12 — Release Decision	Tested	CT-REL-001
R-0241	App A, A.12 — Release Decision	Tested	CT-REL-001
R-0242	App A, A.12 — Release Decision	Tested	CT-REL-010
R-0243	App A, A.12 — Release Decision	Tested	CT-DEC-001
R-0244	App A, A.12 — Release Decision	Tested	CT-DEC-001
R-0245	App A, A.12 — Release Decision	Tested	CT-DEC-001
R-0246	App A, A.12 — Release Decision	Tested	CT-DEC-001, CT-DEC-017

ID	Source	Status	Tests or note
R-0247	App A, A.12 — Release Decision	Tested	CT-DEC-001, CT-REL-014
R-0248	App A, A.12 — Release Decision	Tested	CT-DEC-001
R-0249	App A, A.12 — Release Decision	Tested	CT-DEC-001, CT-REL-006
R-0250	App A, A.13 — Snapshot	Tested	CT-REL-013
R-0251	App A, A.13 — Snapshot	Tested	CT-REL-013
R-0252	App A, A.13 — Snapshot	Tested	CT-REL-013
R-0253	App A, A.13 — Snapshot	Tested	CT-REL-013
R-0254	App A, A.13 — Snapshot	Tested	CT-REL-013
R-0255	App A, A.13 — Snapshot	Tested	CT-REL-013
R-0256	App A, A.13 — Snapshot	Tested	CT-REL-013
R-0257	App A, A.13 — Snapshot	Tested	CT-REL-013
R-0258	App B	Not testable	introduces B.2
R-0259	App B	Not testable	introduces B.2
R-0260	App B, B.1 — Schema Structure Overview	Not testable	introduces B.1
R-0261	App B, B.2 — Property Constraints	Not testable	introduces B.2
R-0262	App B, B.2 — Property Constraints	Tested	CT-SCT-005
R-0263	App B, B.2 — Property Constraints	Tested	CT-SCT-005
R-0264	App B, B.2 — Property Constraints	Tested	CT-SCT-002
R-0265	App B, B.2 — Property Constraints	Tested	CT-SCT-002

ID	Source	Status	Tests or note
R-0266	App B, B.2 — Property Constraints	Tested	CT-CFG-003
R-0267	App B, B.2 — Property Constraints	Tested	CT-CFG-004
R-0268	App B, B.2 — Property Constraints	Tested	CT-MJG-002
R-0269	App B, B.2 — Property Constraints	Tested	CT-MJG-002
R-0270	App B, B.2 — Property Constraints	Tested	CT-MJG-004
R-0271	App B, B.2 — Property Constraints	Tested	CT-CFG-010
R-0272	App B, B.2 — Property Constraints	Restates	R-0221 (CT-CFG-024)
R-0273	App B, B.2 — Property Constraints	Tested	CT-DEC-005, CT-DEC-008
R-0274	App B, B.3 — Cross-Type Validation Rules	Not testable	introduces B.3
R-0275	App B, B.3 — Cross-Type Validation Rules	Tested	CT-CFG-006
R-0276	App B, B.3 — Cross-Type Validation Rules	Tested	CT-CFG-002
R-0277	App B, B.3 — Cross-Type Validation Rules	Tested	CT-CFG-007, CT-CFG-012, CT-CFG-013; text revised in v0.37 or later
R-0278	App B, B.3 — Cross-Type Validation Rules	Tested	CT-CFG-009
R-0279	App B, B.3 — Cross-Type Validation Rules	Tested	CT-CFG-022, CT-DSR-004
R-0280	App B, B.3 — Cross-Type Validation Rules	Tested	CT-CFG-008
R-0281	App B, B.3 — Cross-Type Validation Rules	Tested	CT-CFG-004, CT-CFG-005
R-0282	App B, B.3 — Cross-Type Validation Rules	Tested	CT-CFG-018, CT-DSR-003
R-0283	App B, B.3 — Cross-Type Validation Rules	Tested	CT-CFG-010, CT-EXE-004
R-0284	App B, B.3 — Cross-Type Validation Rules	Tested	CT-REL-034

ID	Source	Status	Tests or note
R-0285	App B, B.3 — Cross-Type Validation Rules	Tested	CT-CFG-007, CT-CFG-027; text revised in v0.37 or later; text revised in v0.41.2 or later
R-0286	App B, B.3 — Cross-Type Validation Rules	Tested	CT-ASM-006, CT-ASM-007
R-0287	App B, B.3 — Cross-Type Validation Rules	Tested	CT-ASM-006
R-0288	App B, B.3 — Cross-Type Validation Rules	Tested	CT-SCT-006
R-0289	App B, B.3 — Cross-Type Validation Rules	Tested	CT-SCT-003, CT-SCT-004, CT-SCT-012, CT-SCT-013; text revised in v0.40 or later
R-0290	App B, B.3 — Cross-Type Validation Rules	Tested	CT-DEC-015
R-0291	App B, B.3 — Cross-Type Validation Rules	Tested	CT-ASM-013, CT-MJG-008
R-0292	App B, B.3 — Cross-Type Validation Rules	Tested	CT-MJG-008, CT-MJG-009
R-0293	App B, B.3 — Cross-Type Validation Rules	Tested	CT-DEC-002, CT-DEC-024; text revised in v0.41.2 or later
R-0294	App B, B.3 — Cross-Type Validation Rules	Tested	CT-DEC-004
R-0295	App B, B.3 — Cross-Type Validation Rules	Tested	CT-ABD-001
R-0296	App B, B.3 — Cross-Type Validation Rules	Tested	CT-ABD-003, CT-ABD-004
R-0297	App B, B.3 — Cross-Type Validation Rules	Tested	CT-ABD-006
R-0298	App B, B.3 — Cross-Type Validation Rules	Tested	CT-REL-022
R-0299	App B, B.3 — Cross-Type Validation Rules	Tested	CT-DEC-012, CT-DEC-014, CT-DEC-018, CT-SCT-008, CT-SCT-014
R-0300	App B, B.3 — Cross-Type Validation Rules	Tested	CT-DEC-012
R-0301	App B, B.3 — Cross-Type Validation Rules	Tested	CT-DEC-010
R-0302	App B, B.3 — Cross-Type Validation Rules	Tested	CT-DEC-015, CT-SCT-011
R-0303	App B, B.3 — Cross-Type Validation Rules	Tested	CT-ASM-014

ID	Source	Status	Tests or note
R-0304	App B, B.3 — Cross-Type Validation Rules	Tested	CT-DEC-005, CT-DEC-008, CT-ADM-007
R-0305	App B, B.3 — Cross-Type Validation Rules	Tested	CT-DEC-006, CT-DEC-007, CT-SCT-011; text revised in v0.40 or later
R-0306	App B, B.3 — Cross-Type Validation Rules	Tested	CT-REL-006, CT-REL-008; text revised in v0.37 or later
R-0307	App B, B.3 — Cross-Type Validation Rules	Tested	CT-REL-007
R-0308	App B, B.3 — Cross-Type Validation Rules	Tested	CT-REL-004
R-0309	App B, B.3 — Cross-Type Validation Rules	Tested	CT-REL-010, CT-REL-011, CT-REL-012
R-0310	App B, B.3 — Cross-Type Validation Rules	Tested	CT-REL-005
R-0311	App B, B.3 — Cross-Type Validation Rules	Tested	CT-REL-001, CT-REL-016, CT-REL-021
R-0312	App B, B.3 — Cross-Type Validation Rules	Tested	CT-REL-017
R-0313	App B, B.3 — Cross-Type Validation Rules	Tested	CT-REL-020
R-0314	App B, B.3 — Cross-Type Validation Rules	Tested	CT-REL-018
R-0315	App B, B.3 — Cross-Type Validation Rules	Tested	CT-REL-013
R-0316	App C	Tested	CT-SDK-001, CT-SDK-002
R-0317	App C, C.1 — Design Principles	Tested	CT-EXE-007
R-0318	App C, C.3 — Operations by Resource	Tested	CT-CFG-017
R-0319	App C, C.3 — Operations by Resource	Tested	CT-DEC-015, CT-SCT-010, CT-SCT-011
R-0320	App C, C.3 — Operations by Resource	Tested	CT-DEC-017, CT-SCT-015
R-0321	App C, C.3 — Operations by Resource	Tested	CT-SCT-007
R-0322	App C, C.3 — Operations by Resource	Tested	CT-SCT-007

ID	Source	Status	Tests or note
R-0323	App C, C.3 — Operations by Resource	Tested	CT-REL-008; text revised in v0.37 or later
R-0324	App C, C.3 — Operations by Resource	Tested	CT-DEC-001, CT-DEC-002
R-0325	App C, C.3 — Operations by Resource	Tested	CT-DEC-003
R-0326	App C, C.3 — Operations by Resource	Tested	CT-REL-005, CT-REL-010
R-0327	App C, C.4 — Authentication and Authorization	Tested	CT-REL-028
R-0328	App C, C.4 — Authentication and Authorization	Tested	CT-REL-019, CT-REL-028
R-0329	App C, C.4 — Authentication and Authorization	Tested	CT-REL-008, CT-REL-009, CT-REL-015, CT-REL-016, CT-REL-030
R-0330	App C, C.4 — Authentication and Authorization	Tested	CT-REL-008, CT-REL-009, CT-REL-015, CT-REL-016, CT-REL-030
R-0331	App C, C.6 — Pagination and Filtering	Tested	CT-API-001
R-0332	App C, C.6 — Pagination and Filtering	Tested	CT-API-001
R-0333	App C, C.7 — Result Response Shape	Tested	CT-ASM-006
R-0334	App C, C.7 — Result Response Shape	Tested	CT-ASM-006
R-0335	App C, C.7 — Result Response Shape	Tested	CT-ASM-006
R-0336	App C, C.7 — Result Response Shape	Tested	CT-ASM-006
R-0337	App C, C.7 — Result Response Shape	Tested	CT-ASM-006, CT-ASM-012
R-0338	App C, C.7 — Result Response Shape	Tested	CT-ASM-006
R-0339	App C, C.7 — Result Response Shape	Tested	CT-ASM-006
R-0340	App C, C.7 — Result Response Shape	Tested	CT-ASM-006
R-0341	App C, C.7 — Result Response Shape	Tested	CT-ASM-006

ID	Source	Status	Tests or note
R-0342	App C, C.7 — Result Response Shape	Tested	CT-ASM-013
R-0343	App C, C.7 — Result Response Shape	Tested	CT-ASM-008, CT-DEC-007
R-0344	App C, C.7 — Result Response Shape	Tested	CT-DEC-006, CT-DEC-007, CT-DEC-009
R-0345	App C, C.8 — Deletion and Referential Integrity	Tested	CT-PLG-004
R-0346	App C, C.8 — Deletion and Referential Integrity	Tested	CT-REL-022
R-0347	App C, C.8 — Deletion and Referential Integrity	Tested	CT-REL-022
R-0348	App C, C.8 — Deletion and Referential Integrity	Tested	CT-API-002
R-0349	App C, C.8 — Deletion and Referential Integrity	Tested	CT-API-002
R-0350	App D	Restates	R-0036 (CT-DEC-002)
R-0351	App D	Restates	R-0042 (CT-REL-001, CT-REL-002)
R-0352	App D	Tested	CT-SDK-002
R-0353	App D	Restates	R-0016 (CT-REL-027)
R-0354	App D	Restates	R-0084 (CT-CFG-007)
R-0355	App D	Restates	R-0081
R-0356	App D	Restates	R-0007 (CT-ASM-004)
R-0357	App D	Restates	R-0075 (CT-MJG-001)
R-0358	App D	Restates	R-0080
R-0359	App D	Tested	CT-SDK-002
R-0360	App D	Restates	R-0007 (CT-ASM-004)

ID	Source	Status	Tests or note
R-0361	App D	Restates	R-0316 (CT-SDK-001, CT-SDK-002)
R-0362	App D	Restates	R-0090 (CT-SCT-003, CT-SCT-004); text revised in v0.40 or later
R-0363	App D	Restates	R-0006 (CT-ASM-003, CT-ASM-009)
R-0364	App H, H.4 — Contract Composition and Reusable Contracts	Restates	R-0084 (CT-CFG-007); text revised in v0.37 or later
R-0365	App H, H.5 — Multi-Judge: Independence vs. Stability	Restates	R-0189 (CT-MJG-001)
R-0366	App H, H.5 — Multi-Judge: Independence vs. Stability	Restates	R-0189 (CT-MJG-001)
R-0367	App H, H.9 — Human Reviewer Binding and Non-Actionable Dispositions	Restates	R-0045 (CT-DEC-006, CT-DEC-007, CT-DEC-011)
R-0368	App I, I.1 — Contract Design	Restates	R-0081
R-0369	App I, I.1 — Contract Design	Restates	R-0083 (CT-ASM-011)
R-0370	App I, I.1 — Contract Design	Restates	R-0084 (CT-CFG-007)
R-0371	App I, I.2 — Validators and Judges	Restates	R-0022 (CT-CFG-006)
R-0372	App I, I.2 — Validators and Judges	Restates	R-0075 (CT-MJG-001)
R-0373	App I, I.3 — Dataset Management	Restates	R-0063 (CT-CFG-017, CT-EXE-010, CT-DSR-003)
R-0374	App I, I.3 — Dataset Management	Not testable	describes practice
R-0375	App I, I.3 — Dataset Management	Restates	R-0062
R-0376	App I, I.4 — Execution and Runtime	Restates	R-0023 (CT-EXE-007, CT-EXE-008)
R-0377	App I, I.4 — Execution and Runtime	Restates	R-0054 (CT-EXE-006)
R-0378	App I, I.4 — Execution and Runtime	Restates	R-0056 (CT-EXE-004, CT-EXE-005)
R-0379	App I, I.5 — Regression and Baselines	Tested	CT-TTP-001

ID	Source	Status	Tests or note
R-0380	App I, I.6 — Governance	Restates	R-0110
R-0381	App I, I.6 — Governance	Not testable	applies to people
R-0382	App I, I.7 — CI/CD Integration	Restates	R-0113 (CT-REL-024)
R-0383	App I, I.7 — CI/CD Integration	Tested	CT-SDK-001, CT-SDK-002
R-0384	App I, I.8 — Extensibility	Tested	CT-PLG-001
R-0385	App I, I.8 — Extensibility	Restates	R-0282 (CT-CFG-018, CT-DSR-003)
R-0386	App I, I.9 — Common Mistakes	Restates	R-0056 (CT-EXE-004, CT-EXE-005)
R-0387	App I, I.9 — Common Mistakes	Restates	R-0064 (CT-ASM-009)
R-0388	App I, I.9 — Common Mistakes	Restates	R-0022 (CT-CFG-006)
R-0389	App I, I.9 — Common Mistakes	Restates	R-0043 (CT-REL-002, CT-REL-006)
R-0390	App I, I.9 — Common Mistakes	Restates	R-0189 (CT-MJG-001)
R-0391	App K, Step 6 — Evaluation (Semantic Check)	Restates	R-0022 (CT-CFG-006)
R-0392	App A, A.4 — Scenario	Tested	CT-CFG-014; added in v0.37
R-0393	App A, A.5 — Quality Contract	Tested	CT-CFG-016; added in v0.37
R-0394	App A, A.12 — Release Decision	Tested	CT-REL-001; added in v0.37
R-0395	App A, A.12 — Release Decision	Tested	CT-REL-018; added in v0.37
R-0396	App A, A.12 — Release Decision	Tested	CT-DEC-001; added in v0.37
R-0397	App A, A.14 — Release Policy	Tested	CT-REL-031; added in v0.37
R-0398	App A, A.14 — Release Policy	Tested	CT-REL-031; added in v0.37

ID	Source	Status	Tests or note
R-0399	App A, A.14 — Release Policy	Tested	CT-REL-031; added in v0.37
R-0400	App A, A.14 — Release Policy	Tested	CT-REL-031; added in v0.37
R-0401	App A, A.14 — Release Policy	Tested	CT-REL-031; added in v0.37
R-0402	App A, A.14 — Release Policy	Tested	CT-REL-031; added in v0.37
R-0403	App A, A.14 — Release Policy	Tested	CT-REL-031; added in v0.37
R-0404	App A, A.14 — Release Policy	Tested	CT-REL-031; added in v0.37
R-0405	App A, A.14 — Release Policy	Tested	CT-REL-031; added in v0.37
R-0406	App A, A.14 — Release Policy	Tested	CT-REL-031; added in v0.37
R-0407	App A, A.14 — Release Policy	Tested	CT-REL-018; added in v0.37
R-0408	App B, B.3 — Cross-Type Validation Rules	Tested	CT-CFG-015; added in v0.37
R-0409	App B, B.3 — Cross-Type Validation Rules	Tested	CT-CFG-011, CT-CFG-014; added in v0.37
R-0410	App B, B.3 — Cross-Type Validation Rules	Tested	CT-CFG-016; added in v0.37
R-0411	App B, B.3 — Cross-Type Validation Rules	Tested	CT-DEC-022, CT-DEC-023; text revised in v0.37 or later; added in v0.37
R-0412	App B, B.3 — Cross-Type Validation Rules	Tested	CT-REL-018; added in v0.37
R-0413	App C, C.3 — Operations by Resource	Tested	CT-DEC-012, CT-DEC-013, CT-DEC-018; added in v0.37
R-0414	App C, C.3 — Operations by Resource	Tested	CT-DEC-012, CT-HRV-002, CT-HRV-005; text revised in v0.37 or later; added in v0.37
R-0415	App C, C.3 — Operations by Resource	Tested	CT-DEC-013, CT-DEC-018, CT-SCT-008, CT-SCT-014; text revised in v0.40 or later; added in v0.37
R-0416	App C, C.3 — Operations by Resource	Tested	CT-REL-031; added in v0.37
R-0417	App C, C.3 — Operations by Resource	Tested	CT-REL-031; added in v0.37

ID	Source	Status	Tests or note
R-0418	App C, C.7 — Result Response Shape	Tested	CT-ASM-006; added in v0.37
R-0419	Vol I, Quality Gates	Tested	CT-DEC-022, CT-DEC-023, CT-REL-012; added in v0.37
R-0420	App A, A.5 — Quality Contract	Tested	CT-HRV-006; added in v0.38
R-0421	App A, A.5 — Quality Contract	Tested	CT-HRV-006; added in v0.38
R-0422	App B, B.2 — Property Constraints	Tested	CT-HRV-006; added in v0.38
R-0423	App C, C.3 — Operations by Resource	Tested	CT-SCT-004, CT-SCT-013; text revised in v0.40 or later; added in v0.38
R-0424	App C, C.3 — Operations by Resource	Tested	CT-SCT-008, CT-SCT-014; text revised in v0.40 or later; added in v0.38
R-0425	App C, C.7 — Result Response Shape	Tested	CT-ASM-019; added in v0.38
R-0426	App C, C.10 — Report Response Shape	Tested	CT-REL-029; added in v0.38
R-0427	App C, C.10 — Report Response Shape	Tested	CT-REL-029; added in v0.38
R-0428	App C, C.10 — Report Response Shape	Tested	CT-REL-029; added in v0.38
R-0429	App C, C.10 — Report Response Shape	Tested	CT-REL-029; added in v0.38
R-0430	App C, C.10 — Report Response Shape	Tested	CT-REL-029; added in v0.38
R-0431	App C, C.10 — Report Response Shape	Tested	CT-REL-029, CT-REL-035; added in v0.38
R-0432	App A, A.4 — Scenario	Tested	CT-SCT-009; added in v0.40 to v0.41.1
R-0433	App B, B.2 — Property Constraints	Tested	CT-SCT-009; added in v0.40 to v0.41.1
R-0434	App B, B.2 — Property Constraints	Tested	CT-SCT-009; added in v0.40 to v0.41.1
R-0435	Vol VI, Admission	Tested	CT-ADM-001; added in v0.40 to v0.41.1
R-0436	App B, B.3 — Cross-Type Validation Rules	Tested	CT-ADM-001, CT-ADM-002, CT-ADM-003, CT-ADM-004; added in v0.40 to v0.41.1

ID	Source	Status	Tests or note
R-0437	App C, C.3 — Operations by Resource	Tested	CT-ADM-001; added in v0.40 to v0.41.1
R-0438	App C, C.3 — Operations by Resource	Tested	CT-ADM-003; added in v0.40 to v0.41.1
R-0439	App C, C.3 — Operations by Resource	Tested	CT-ADM-001, CT-ADM-002; added in v0.40 to v0.41.1
R-0440	App C, C.3 — Operations by Resource	Tested	CT-ADM-002, CT-ADM-003; added in v0.40 to v0.41.1
R-0441	App B, B.3 — Cross-Type Validation Rules	Tested	CT-ADM-004, CT-ADM-005, CT-ADM-006, CT-ADM-007; added in v0.40 to v0.41.1
R-0442	App A, A.5 — Quality Contract	Tested	CT-ASM-020; added in v0.41.2 to v0.41.4
R-0443	App A, A.8 — Dataset	Tested	CT-CFG-028; added in v0.41.2 to v0.41.4
R-0444	App A, A.10 — Quality Gates	Tested	CT-CFG-026; added in v0.41.2 to v0.41.4
R-0445	App B, B.3 — Cross-Type Validation Rules	Tested	CT-CFG-026; added in v0.41.2 to v0.41.4
R-0446	Vol I, Chapter 6 — Quality Gates	Restates	R-0445 (CT-CFG-026); added in v0.41.2 to v0.41.4
R-0447	Vol I, Chapter 6 — Quality Gates	Tested	CT-DEC-024; added in v0.41.2 to v0.41.4
R-0448	Vol XII, Teams	Tested	CT-REL-027; added in v0.41.2 to v0.41.4
R-0449	Vol XII, Teams	Tested	CT-REL-039; added in v0.41.2 to v0.41.4
R-0450	Vol XII, Audit	Tested	CT-REL-028, CT-REL-038, CT-API-002; added in v0.41.2 to v0.41.4
R-0451	§10, Conformance Statement	Restates	R-0448 (CT-REL-027); added in v0.41.2 to v0.41.4

90.4 Not yet covered

None. Every testable requirement in the inventory, core and scoped, has at least one test.

This is a statement about the tests written, not about any implementation. A requirement whose tests or cases are all `NOT_RUN` or `NOT_APPLICABLE` in a test run was not verified in that run; the test report lists it (1.7).

90.5 Test → requirement

Test	Requirements
CT-CFG-001	R-0128
CT-CFG-002	R-0156, R-0157, R-0276
CT-CFG-003	R-0173, R-0266
CT-CFG-004	R-0178, R-0179, R-0180, R-0182, R-0267, R-0281
CT-CFG-005	R-0181, R-0281
CT-CFG-006	R-0022, R-0186, R-0275
CT-CFG-007	R-0084, R-0277, R-0285
CT-CFG-008	R-0163, R-0280
CT-CFG-009	R-0154, R-0278
CT-CFG-010	R-0214, R-0215, R-0271, R-0283
CT-CFG-011	R-0160, R-0409
CT-CFG-012	R-0277
CT-CFG-013	R-0277

Test	Requirements
CT-CFG-014	R-0392, R-0409
CT-CFG-015	R-0408
CT-CFG-016	R-0393, R-0410
CT-CFG-017	R-0063, R-0196, R-0318
CT-CFG-018	R-0282
CT-CFG-019	R-0082, R-0174
CT-CFG-020	R-0130
CT-CFG-021	R-0152
CT-CFG-022	R-0158, R-0279, R-0161
CT-CFG-023	R-0172
CT-CFG-024	R-0221
CT-CFG-025	By case: .01 R-0129; .02 R-0137; .03 R-0138; .04 R-0139; .05 R-0140; .06 R-0142; .07 R-0151; .08 R-0155; .09 R-0175; .10 R-0176; .11 R-0177; .12 R-0185; .13 R-0187; .14 R-0216; .15 R-0217; .16 R-0218; .17 R-0219; .18 R-0213; .19 R-0195; .20 R-0198; .21 R-0199; .22 R-0202; .23 R-0203; .24 R-0211; .25 R-0132
CT-CFG-026	R-0444, R-0445
CT-CFG-	R-0285

Test	Requirements
027	
CT-CFG-028	R-0443
CT-EXE-001	R-0048
CT-EXE-002	R-0057, R-0162
CT-EXE-003	R-0055
CT-EXE-004	R-0056, R-0283
CT-EXE-005	R-0056
CT-EXE-006	R-0054, R-0212
CT-EXE-007	R-0023, R-0024, R-0029, R-0049, R-0317
CT-EXE-008	R-0023
CT-EXE-009	R-0053
CT-EXE-010	R-0063
CT-EXE-011	R-0058
CT-EXE-012	R-0059

Test	Requirements
CT-ASM-001	R-0021, R-0067
CT-ASM-002	R-0021, R-0030
CT-ASM-003	R-0006, R-0025, R-0026, R-0027, R-0066
CT-ASM-004	R-0007, R-0072
CT-ASM-005	R-0032
CT-ASM-006	R-0286, R-0287, R-0333, R-0334, R-0335, R-0336, R-0337, R-0338, R-0339, R-0340, R-0341, R-0418
CT-ASM-007	R-0286
CT-ASM-008	R-0079, R-0343
CT-ASM-009	R-0006, R-0064
CT-ASM-010	R-0065
CT-ASM-011	R-0073, R-0083
CT-ASM-012	R-0005, R-0337
CT-ASM-013	R-0076, R-0291, R-0342
CT-ASM-	R-0303

Test	Requirements
014	
CT-ASM-015	R-0079
CT-ASM-016	R-0068, R-0070
CT-ASM-017	R-0069
CT-ASM-018	R-0071
CT-ASM-019	R-0074, R-0425
CT-ASM-020	R-0442
CT-DEC-001	R-0243, R-0244, R-0245, R-0246, R-0247, R-0248, R-0249, R-0324, R-0396
CT-DEC-002	R-0036, R-0037, R-0293, R-0324
CT-DEC-003	R-0325
CT-DEC-004	R-0041, R-0294
CT-DEC-005	R-0273, R-0304
CT-DEC-006	R-0031, R-0045, R-0305, R-0344
CT-DEC-007	R-0045, R-0305, R-0343, R-0344

Test	Requirements
CT-DEC-008	R-0273, R-0304
CT-DEC-009	R-0344
CT-DEC-010	R-0301
CT-DEC-011	R-0045
CT-DEC-012	R-0299, R-0300, R-0413, R-0414
CT-DEC-013	R-0079, R-0413, R-0415
CT-DEC-014	R-0299
CT-DEC-015	R-0040, R-0290, R-0302, R-0319
CT-DEC-016	R-0088
CT-DEC-017	R-0040, R-0246, R-0320
CT-DEC-018	R-0299, R-0413, R-0415
CT-DEC-019	R-0035
CT-DEC-020	R-0034
CT-DEC-	R-0038

Test	Requirements
021	
CT-DEC-022	R-0411, R-0419
CT-DEC-023	R-0411, R-0419
CT-DEC-024	R-0293, R-0447
CT-REL-001	R-0039, R-0042, R-0311, R-0230, R-0231, R-0232, R-0233, R-0234, R-0235, R-0236, R-0237, R-0238, R-0239, R-0240, R-0241, R-0394
CT-REL-002	R-0042, R-0043, R-0220
CT-REL-003	R-0039, R-0098, R-0220
CT-REL-004	R-0308
CT-REL-005	R-0310, R-0326
CT-REL-006	R-0043, R-0249, R-0306
CT-REL-007	R-0044, R-0111, R-0307
CT-REL-008	R-0323, R-0329, R-0330, R-0306
CT-REL-009	R-0329, R-0330
CT-REL-010	R-0046, R-0242, R-0309, R-0326

Test	Requirements
CT-REL-011	R-0309
CT-REL-012	R-0309, R-0419
CT-REL-013	R-0250, R-0251, R-0252, R-0253, R-0254, R-0255, R-0256, R-0257, R-0315
CT-REL-014	R-0247
CT-REL-015	R-0329, R-0330
CT-REL-016	R-0311, R-0329, R-0330
CT-REL-017	R-0105, R-0226, R-0312
CT-REL-018	R-0107, R-0237, R-0314, R-0395, R-0412, R-0407
CT-REL-019	R-0103, R-0328
CT-REL-020	R-0106, R-0313
CT-REL-021	R-0094, R-0311, R-0222, R-0223, R-0224, R-0225, R-0226, R-0227, R-0228, R-0229
CT-REL-022	R-0108, R-0298, R-0346, R-0347
CT-REL-023	None in the inventory: Volume XII, Traceability, states this without MUST
CT-REL-	R-0015, R-0113

Test	Requirements
024	
CT-REL-025	R-0114
CT-REL-026	R-0017, R-0018
CT-REL-027	R-0016, R-0448
CT-REL-028	R-0327, R-0328, R-0450
CT-REL-029	R-0098, R-0099, R-0100, R-0426, R-0427, R-0428, R-0429, R-0430, R-0431
CT-REL-030	R-0329, R-0330
CT-REL-031	R-0397, R-0398, R-0399, R-0400, R-0401, R-0402, R-0403, R-0404, R-0405, R-0406, R-0416, R-0417
CT-REL-032	R-0102
CT-REL-033	R-0104
CT-REL-034	R-0284
CT-REL-035	R-0097, R-0431
CT-REL-036	R-0101
CT-REL-037	R-0112

Test	Requirements
CT-REL-038	R-0450
CT-REL-039	R-0449
CT-ADM-001	R-0435, R-0436, R-0437, R-0439
CT-ADM-002	R-0436, R-0439, R-0440
CT-ADM-003	R-0436, R-0438, R-0440
CT-ADM-004	R-0436, R-0441
CT-ADM-005	R-0441
CT-ADM-006	R-0441
CT-ADM-007	R-0304, R-0441
CT-API-001	R-0331, R-0332
CT-API-002	R-0348, R-0349, R-0450
CT-HRV-001	R-0183
CT-HRV-002	R-0184, R-0414
CT-HRV-003	R-0078

Test	Requirements
CT-HRV-004	R-0032
CT-HRV-005	R-0078, R-0414
CT-HRV-006	R-0420, R-0421, R-0422
CT-MJG-001	R-0075, R-0188, R-0189
CT-MJG-002	R-0190, R-0191, R-0192, R-0268, R-0269
CT-MJG-003	R-0193, R-0194
CT-MJG-004	R-0270
CT-MJG-005	R-0076
CT-MJG-006	R-0076, R-0191, R-0192
CT-MJG-007	R-0077
CT-MJG-008	R-0291, R-0292
CT-MJG-009	R-0292
CT-SCT-001	R-0089, R-0164, R-0166, R-0167
CT-SCT-002	R-0168, R-0169, R-0170, R-0264, R-0265

Test	Requirements
CT-SCT-003	R-0090, R-0289
CT-SCT-004	R-0090, R-0289, R-0423
CT-SCT-005	R-0092, R-0133, R-0134, R-0135, R-0165, R-0262, R-0263
CT-SCT-006	R-0091, R-0136, R-0288
CT-SCT-007	R-0087, R-0321, R-0322
CT-SCT-008	R-0299, R-0415, R-0424
CT-SCT-009	R-0166, R-0432, R-0433, R-0434
CT-SCT-010	R-0088, R-0319
CT-SCT-011	R-0040, R-0302, R-0305, R-0319
CT-SCT-012	R-0289
CT-SCT-013	R-0289, R-0423
CT-SCT-014	R-0299, R-0415, R-0424
CT-SCT-015	R-0040, R-0320
CT-ABD-001	R-0050, R-0141, R-0295
CT-ABD-002	R-0143, R-0144, R-0145, R-0146, R-0148, R-0149, R-0150
CT-ABD-003	R-0051, R-0060, R-0296
CT-ABD-004	R-0060, R-0296

Test	Requirements
CT-ABD-005	R-0050
CT-ABD-006	R-0147, R-0297
CT-ABD-007	R-0052
CT-DSR-001	R-0200, R-0204, R-0205
CT-DSR-002	R-0201, R-0206, R-0207, R-0208, R-0209, R-0210
CT-DSR-003	R-0063, R-0282
CT-DSR-004	R-0158, R-0279
CT-PLG-001	R-0122, R-0384
CT-PLG-002	R-0123
CT-PLG-003	R-0126
CT-PLG-004	R-0127, R-0345
CT-PLG-005	R-0121
CT-PLG-006	R-0124
CT-PLG-	R-0125

Test	Requirements
007	
CT-SDK-001	R-0115, R-0316, R-0383
CT-SDK-002	R-0316, R-0352, R-0359, R-0383
CT-RUI-001	R-0116
CT-RUI-002	R-0117, R-0118
CT-RUI-003	R-0119
CT-RUI-004	R-0120
CT-TTP-001	R-0095, R-0379
CT-TTP-002	R-0047